

ECE/CS 757 Advanced Computer Architecture II

Spring 2025

Interconnects (Part 5)

Instructor: Prof. Joshua San Miguel

Some slides adopted from Natalie Enright Jerger

Last Lecture

- Adaptive routing
- Routing deadlocks
- Circuit-switching

This Lecture

- Packet-based flow control
- Flit-based flow control
- Virtual channels
- Deadlocks
- Router microarchitecture

Network-On-Chip

Key design decisions:

Topology

- Where to place routers/nodes and how to link them together

Routing

- Which set of links to traverse between source-destination pairs

Flow Control

- What granularity to move messages through links and routers

Router Microarchitecture

- How to pipeline and arbitrate incoming messages in routers

Flow Control – Switching

Message-based:

Allocation made at message granularity

- e.g., circuit-switching

Packet-based:

Allocation made at packet granularity

- e.g., store and forward, virtual cut through

Flit-based:

Allocation made at flit granularity

- e.g., wormhole

Packet-Based Flow Control

Break messages into packets and interleave packets on links

- Better link utilization
- Requires per-hop buffering to store in-flight packets
- In shared-memory CMPs, each packet is coherence message
- Two types of packet-based: store and forward, virtual cut through

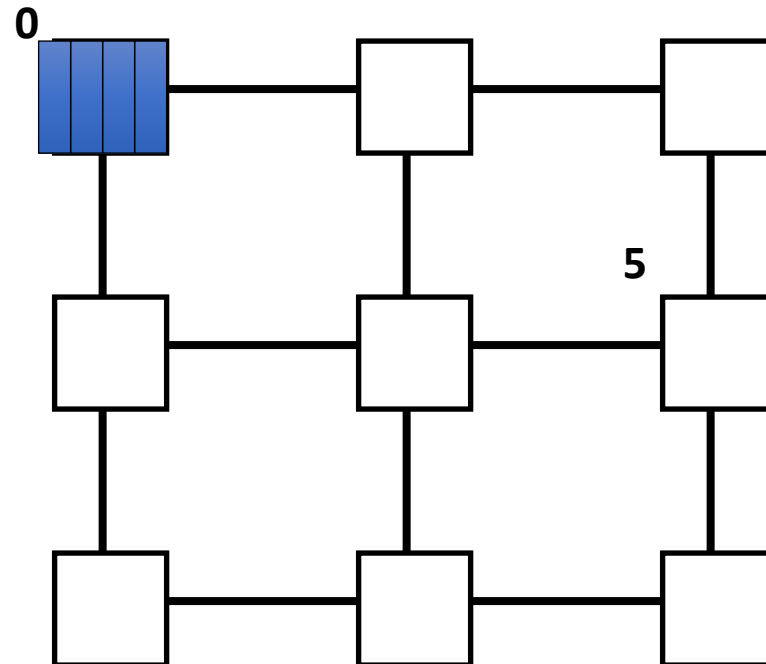
Store And Forward

Links and buffers are allocated to entire packet one hop at a time

- Head flit waits at router until entire packet is received before being forwarded to the next hop
- Requires enough buffering at each router to store entire packet
- High latency; pays serialization delay at each hop

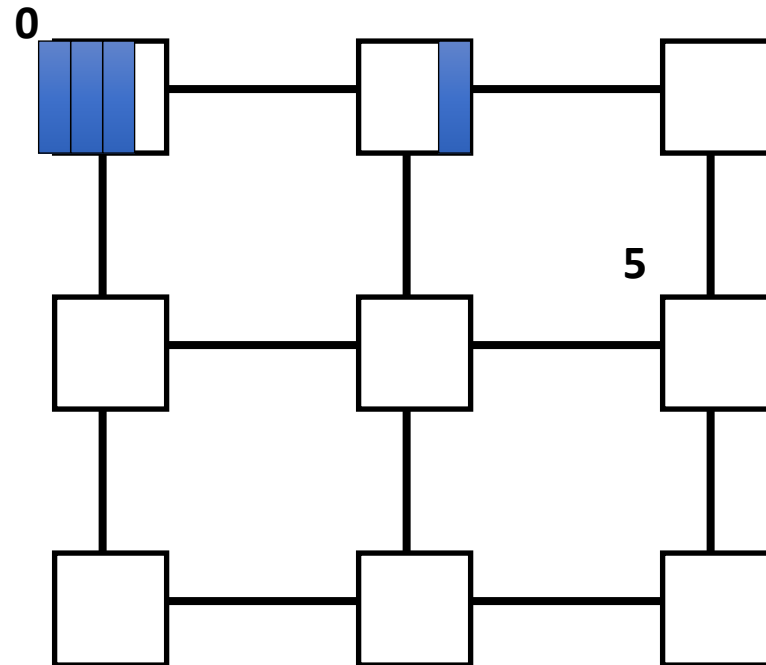
Store And Forward

e.g., Packet from 0 to 5



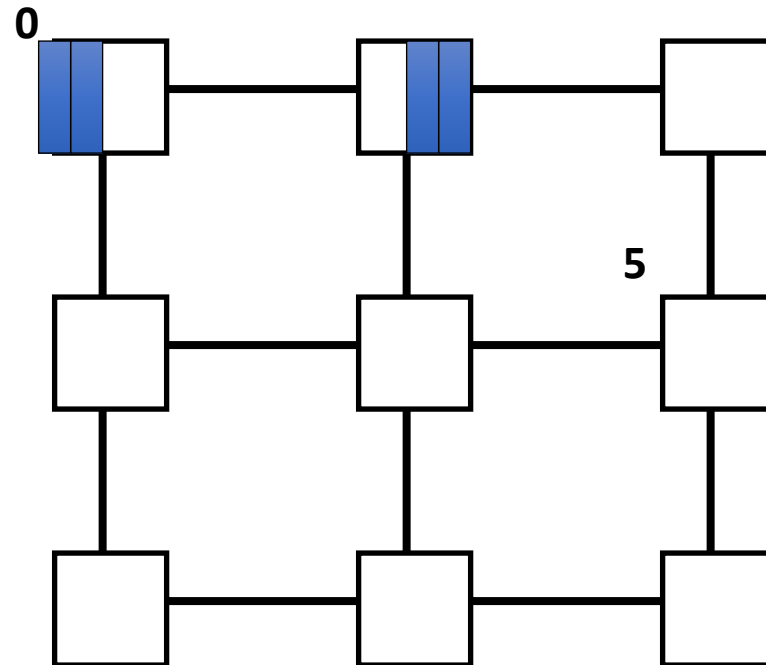
Store And Forward

e.g., Packet from 0 to 5



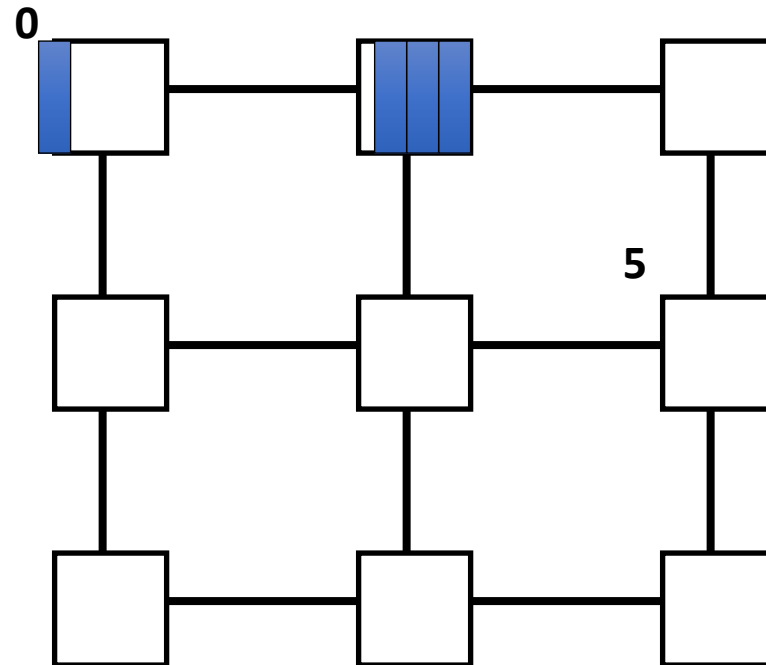
Store And Forward

e.g., Packet from 0 to 5



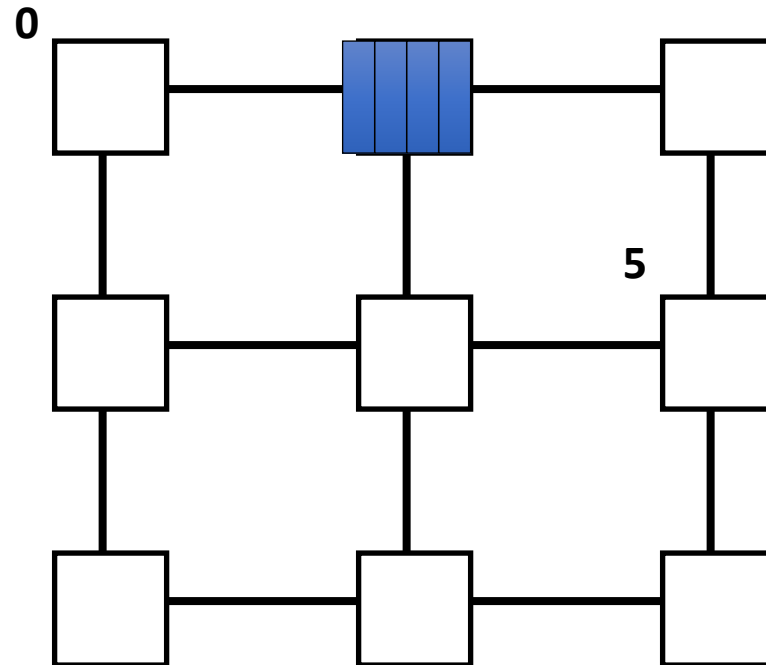
Store And Forward

e.g., Packet from 0 to 5



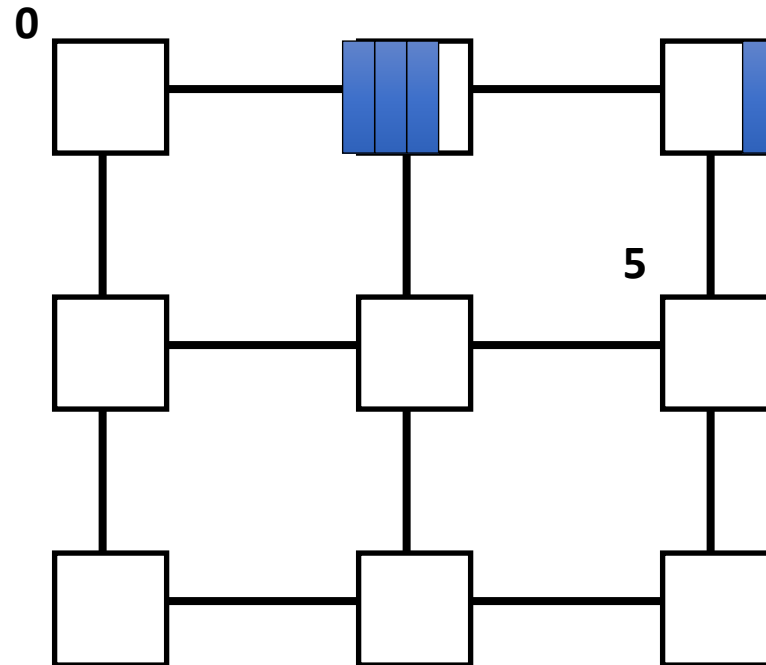
Store And Forward

e.g., Packet from 0 to 5



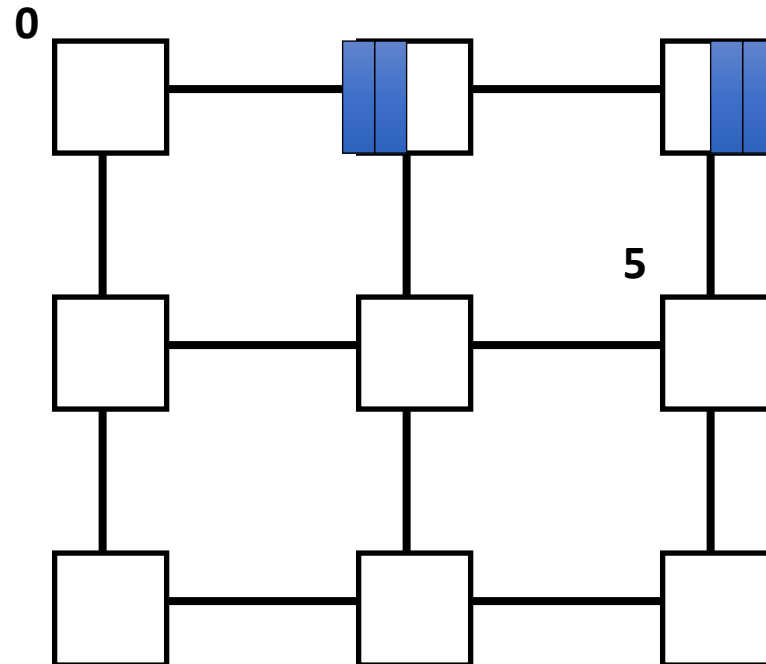
Store And Forward

e.g., Packet from 0 to 5



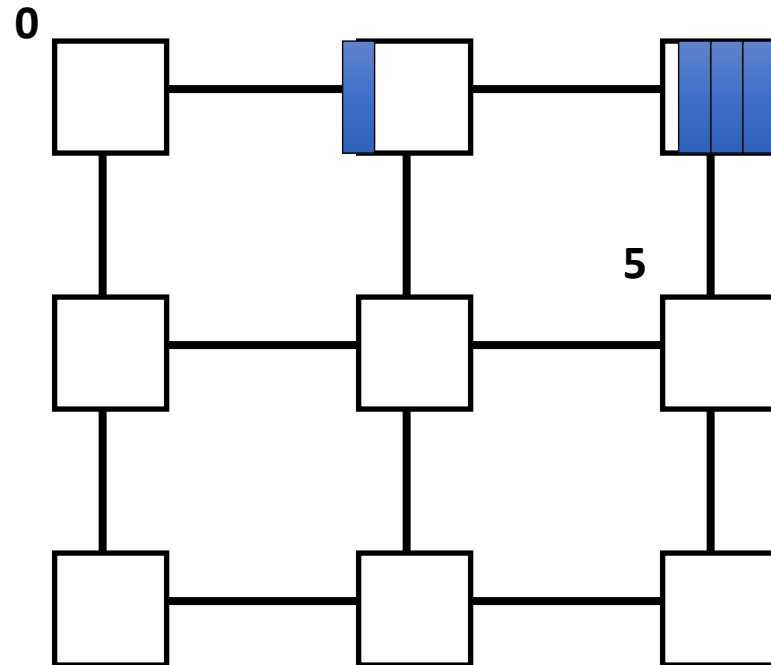
Store And Forward

e.g., Packet from 0 to 5



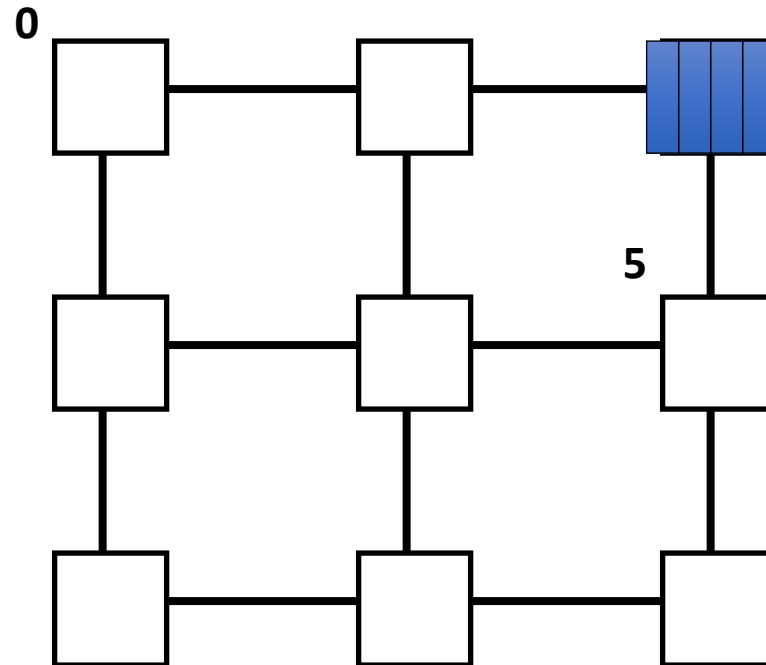
Store And Forward

e.g., Packet from 0 to 5



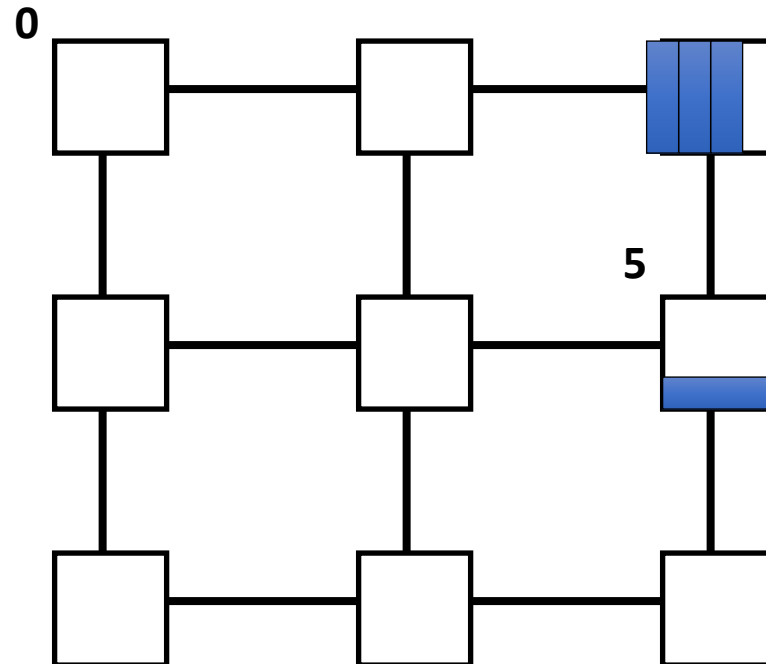
Store And Forward

e.g., Packet from 0 to 5



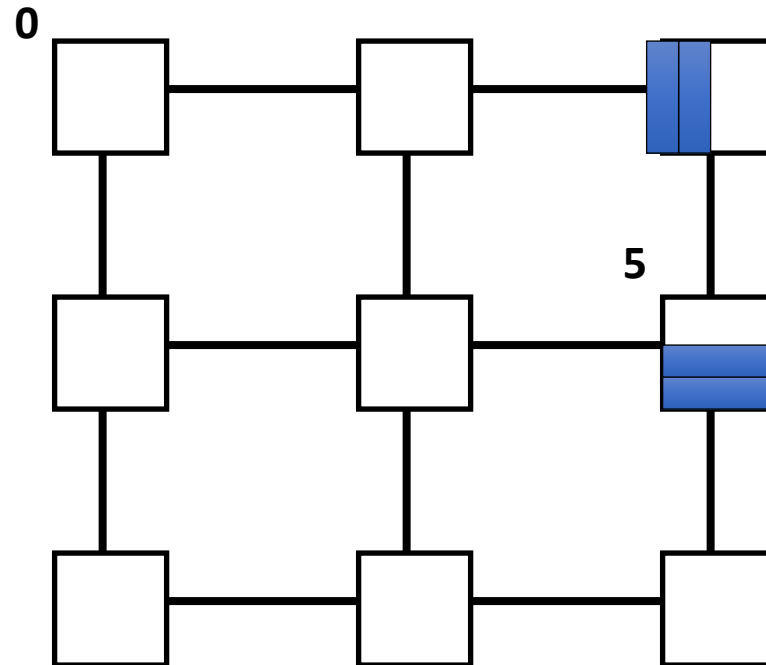
Store And Forward

e.g., Packet from 0 to 5



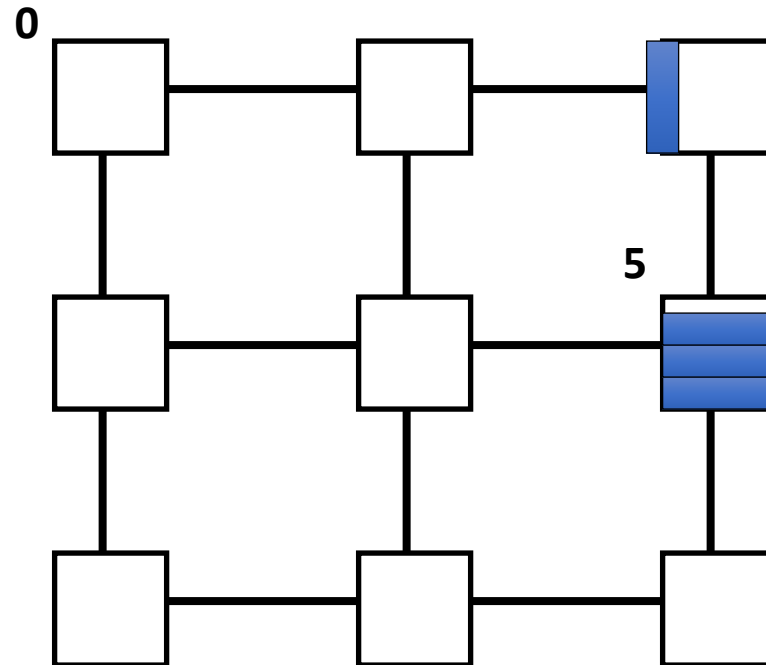
Store And Forward

e.g., Packet from 0 to 5



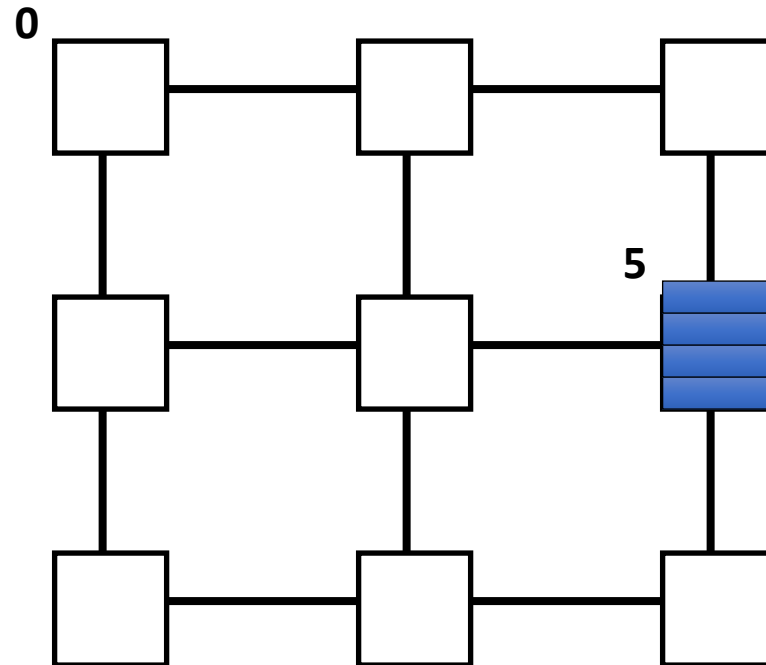
Store And Forward

e.g., Packet from 0 to 5



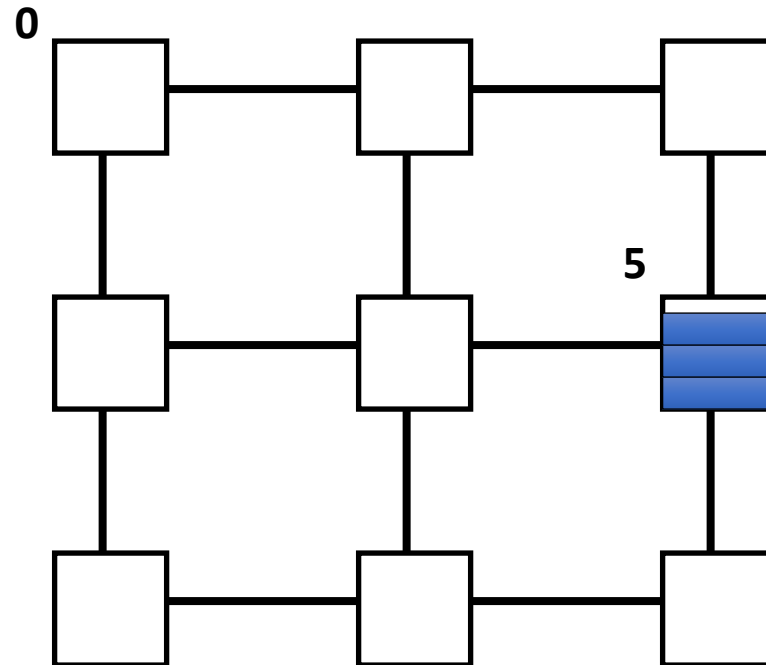
Store And Forward

e.g., Packet from 0 to 5



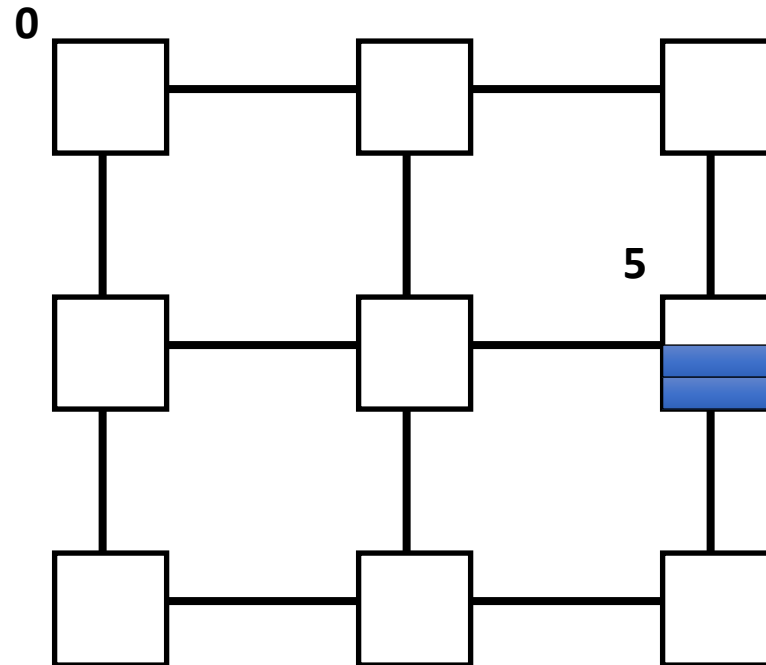
Store And Forward

e.g., Packet from 0 to 5



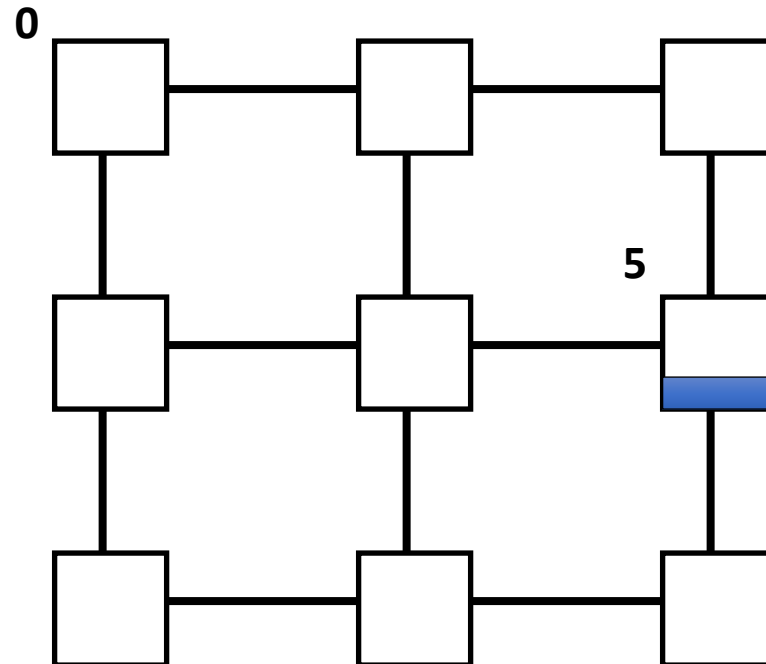
Store And Forward

e.g., Packet from 0 to 5



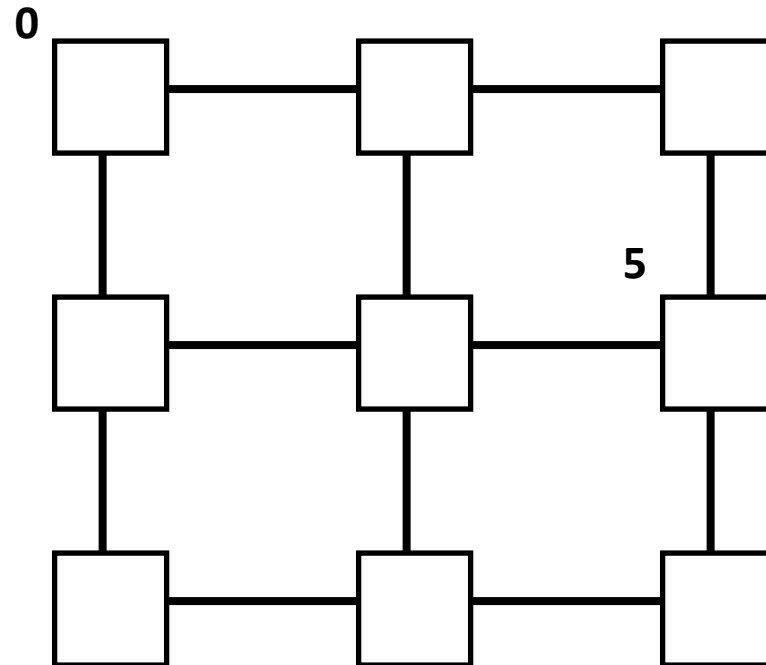
Store And Forward

e.g., Packet from 0 to 5



Store And Forward

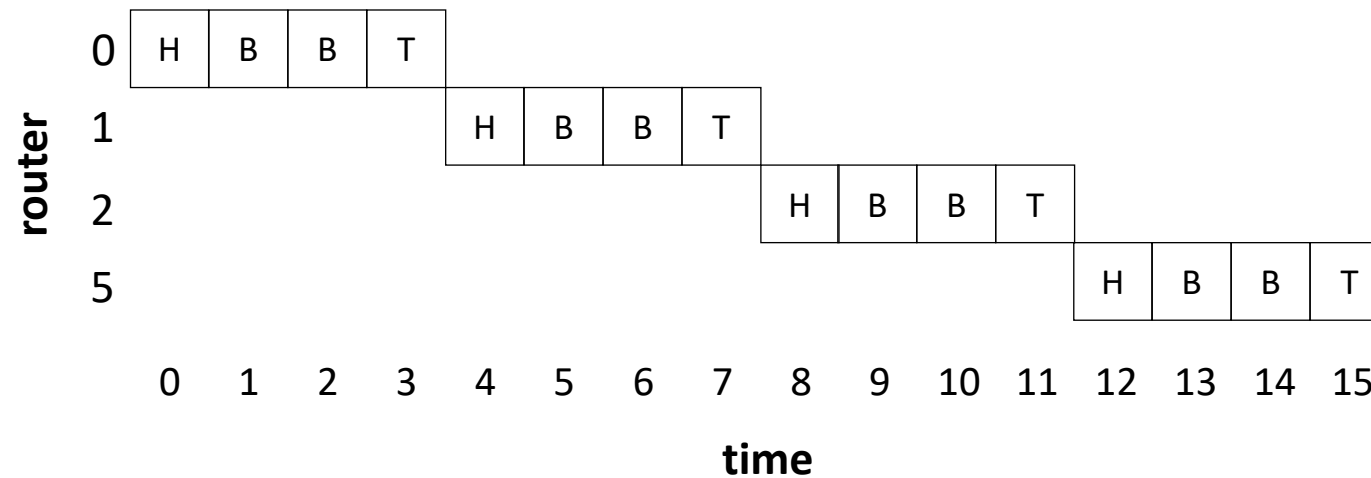
e.g., Packet from 0 to 5



Store And Forward

e.g., Packet from 0 to 5

- Serialization delay = # flits per packet / link bandwidth (i.e., flits/cycle)
- Packet latency = (# hops + 1) × (serialization delay) = (3 hops + 1) × (4/1 cycles) = 16 cycles



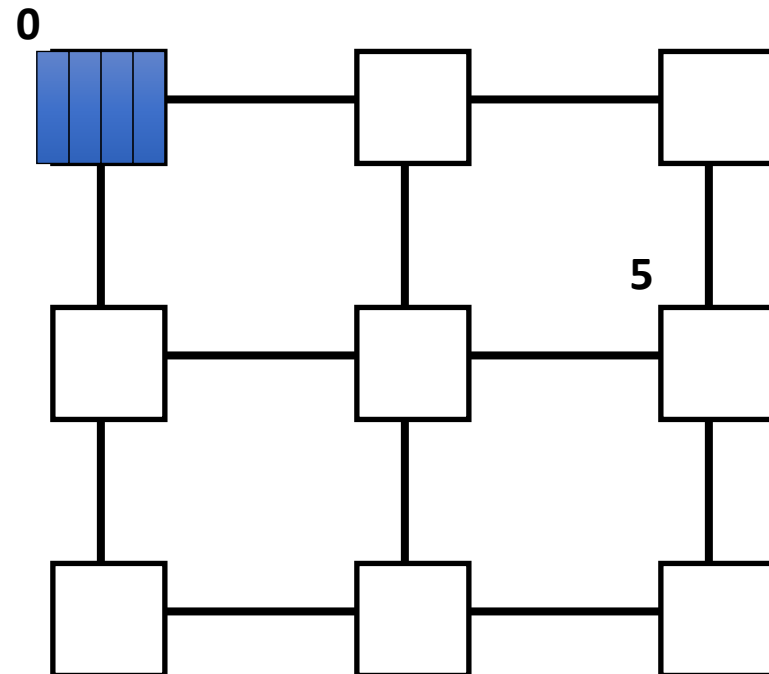
Virtual Cut Through

Links and buffers are allocated to entire packet one hop at a time, but flits can proceed to next hop before tail flit has been received at current router

- Flits can proceed to next hop only if next router has enough buffer space for entire packet
- Improves latency of store and forward
- Still requires large buffers at each router to store entire packet

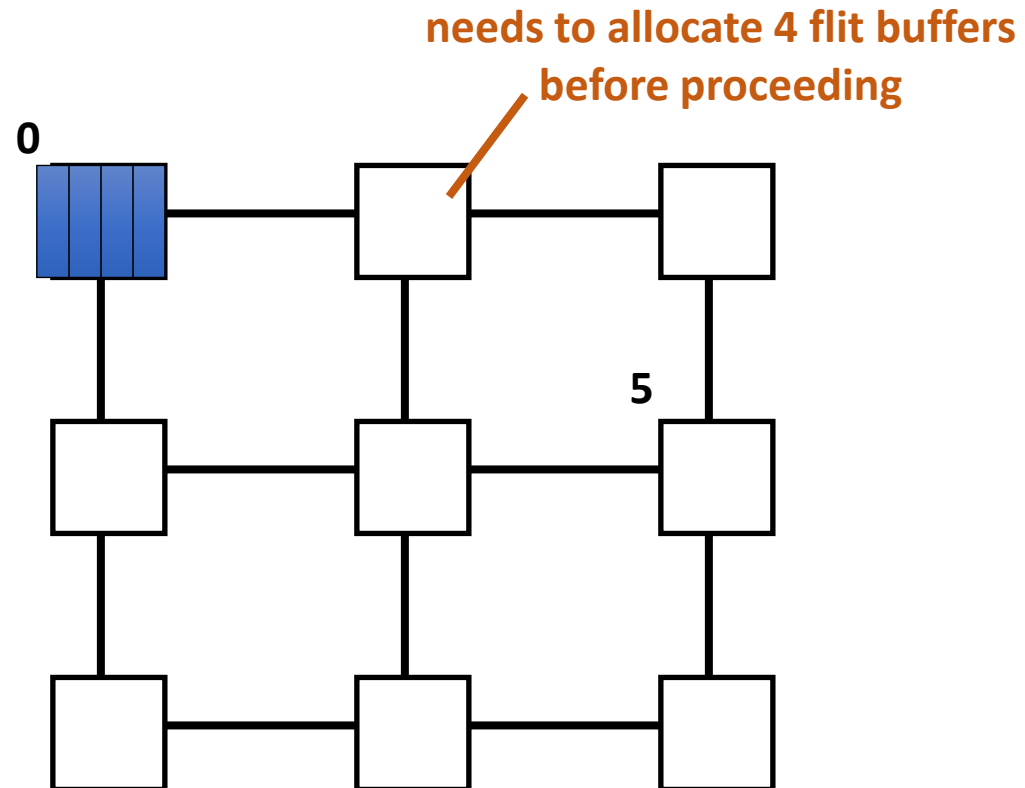
Virtual Cut Through

e.g., Packet from 0 to 5



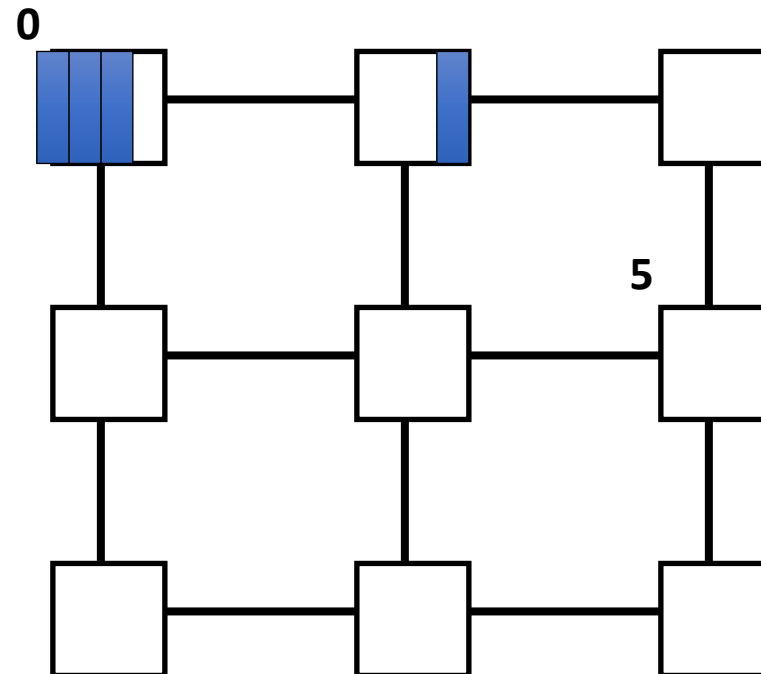
Virtual Cut Through

e.g., Packet from 0 to 5



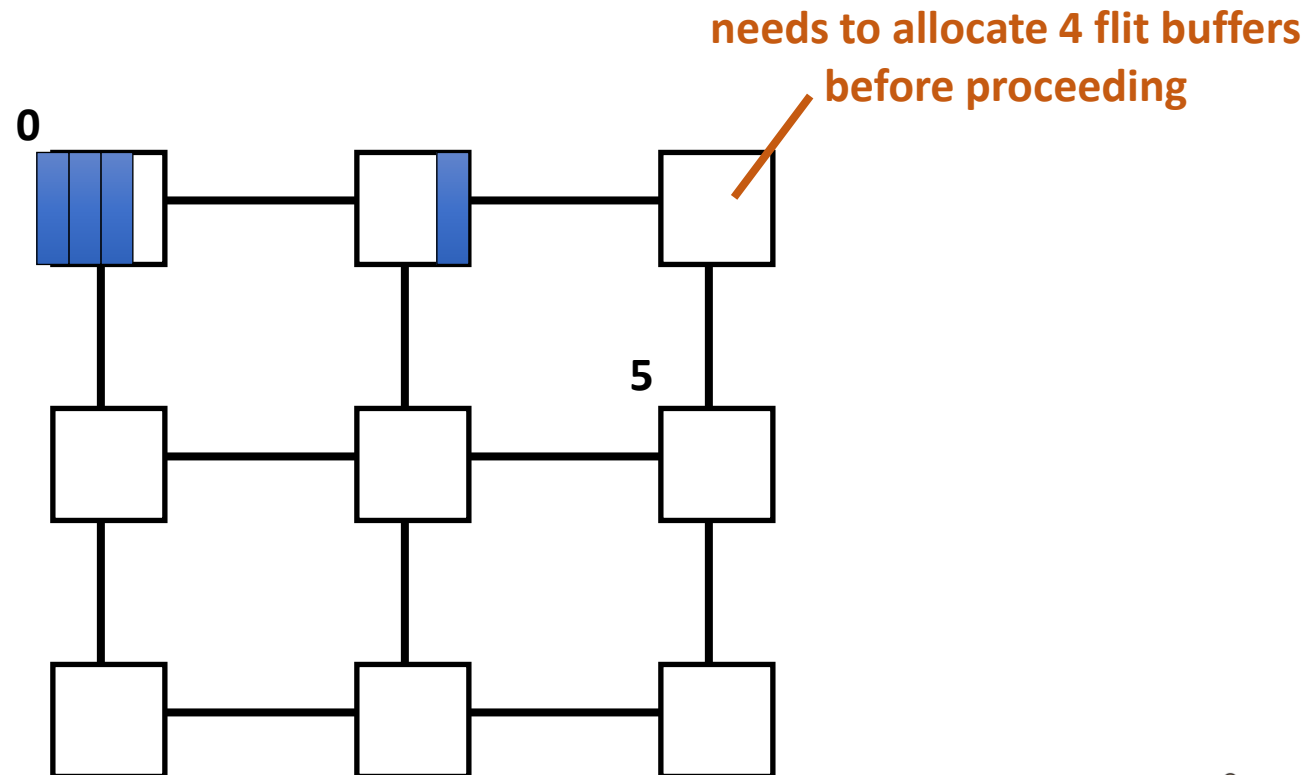
Virtual Cut Through

e.g., Packet from 0 to 5



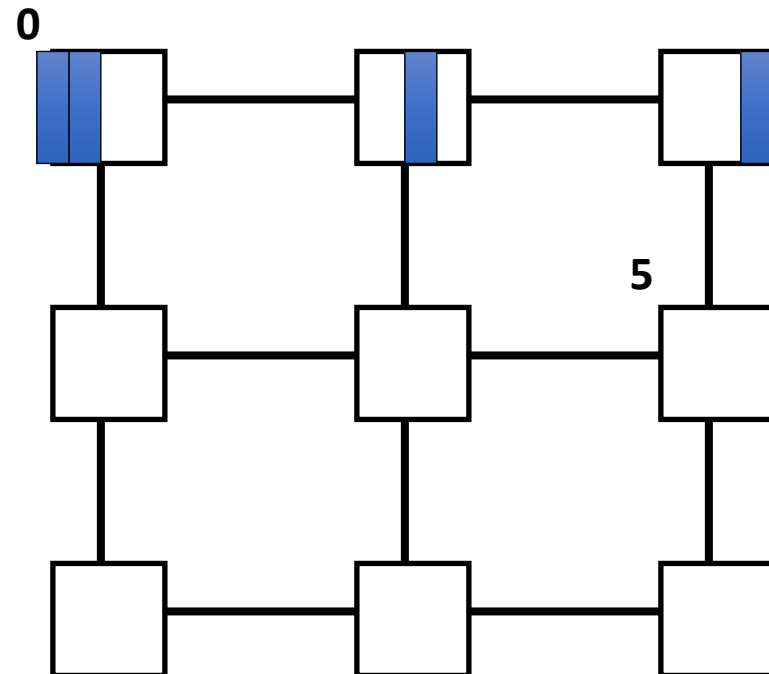
Virtual Cut Through

e.g., Packet from 0 to 5



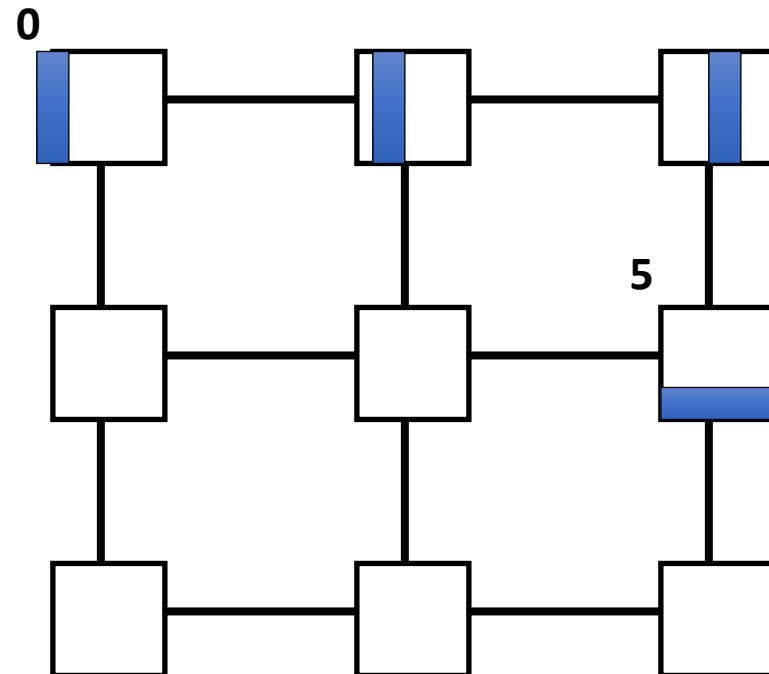
Virtual Cut Through

e.g., Packet from 0 to 5



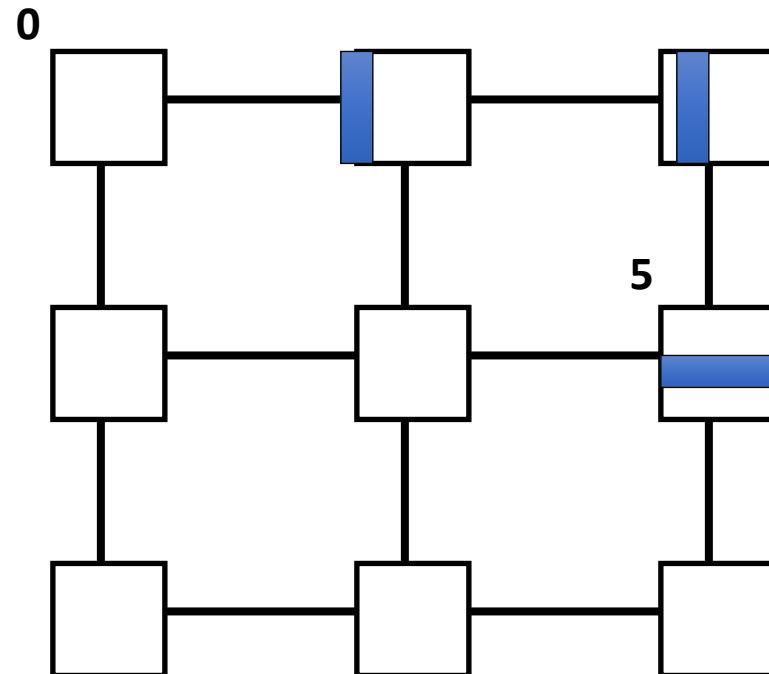
Virtual Cut Through

e.g., Packet from 0 to 5



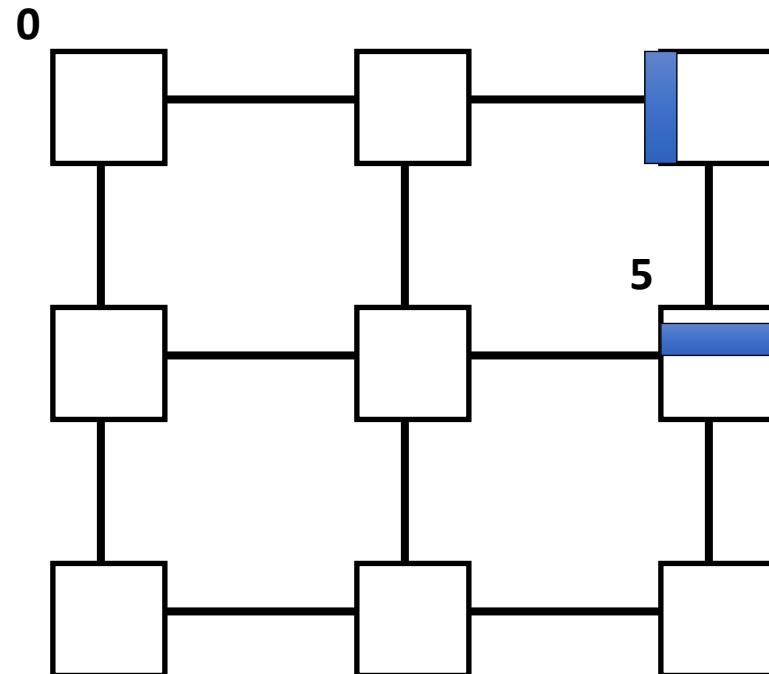
Virtual Cut Through

e.g., Packet from 0 to 5



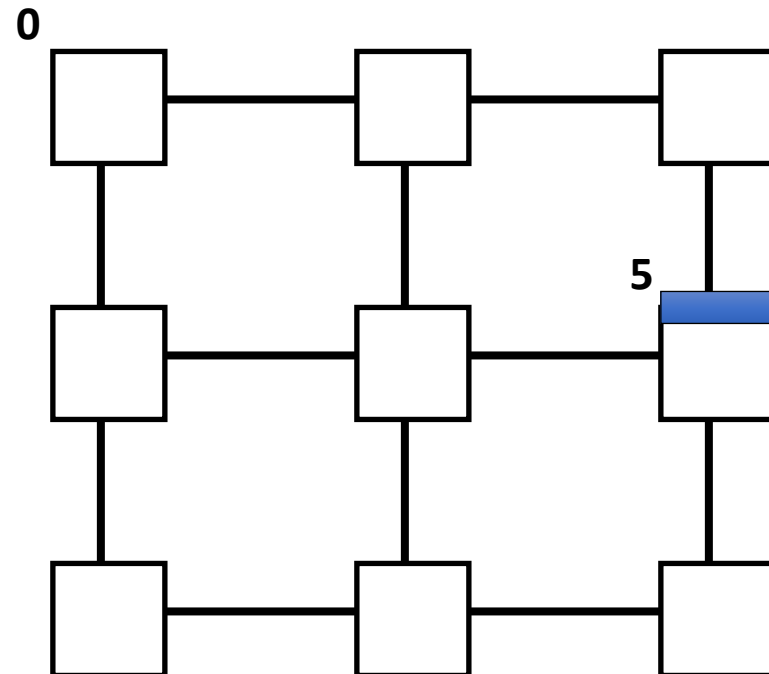
Virtual Cut Through

e.g., Packet from 0 to 5



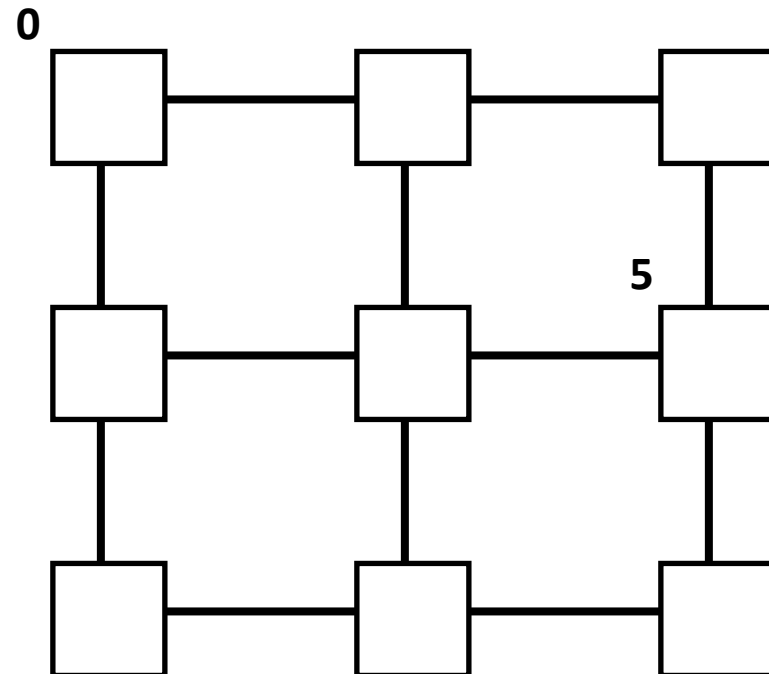
Virtual Cut Through

e.g., Packet from 0 to 5



Virtual Cut Through

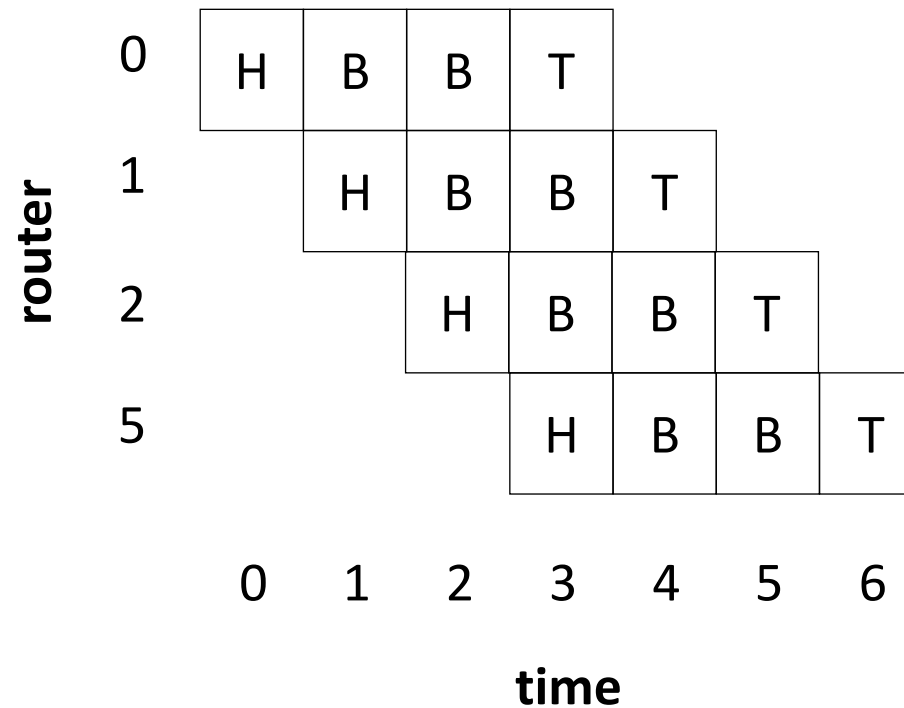
e.g., Packet from 0 to 5



Virtual Cut Through

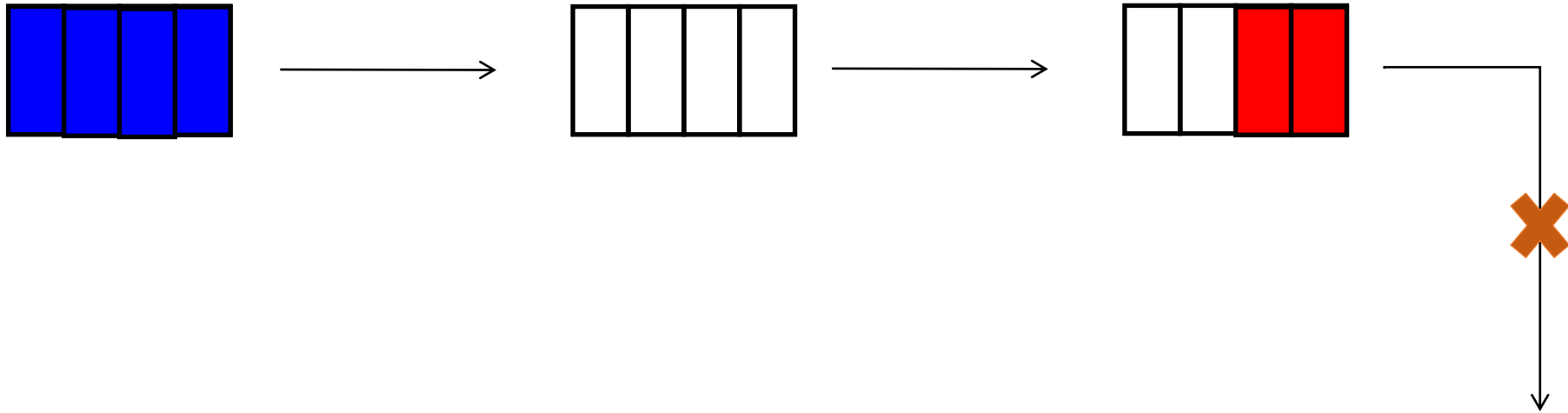
e.g., Packet from 0 to 5

➤ Packet latency = (# hops) + (serialization delay) = (3 hops) + (4/1 cycles) = 7 cycles



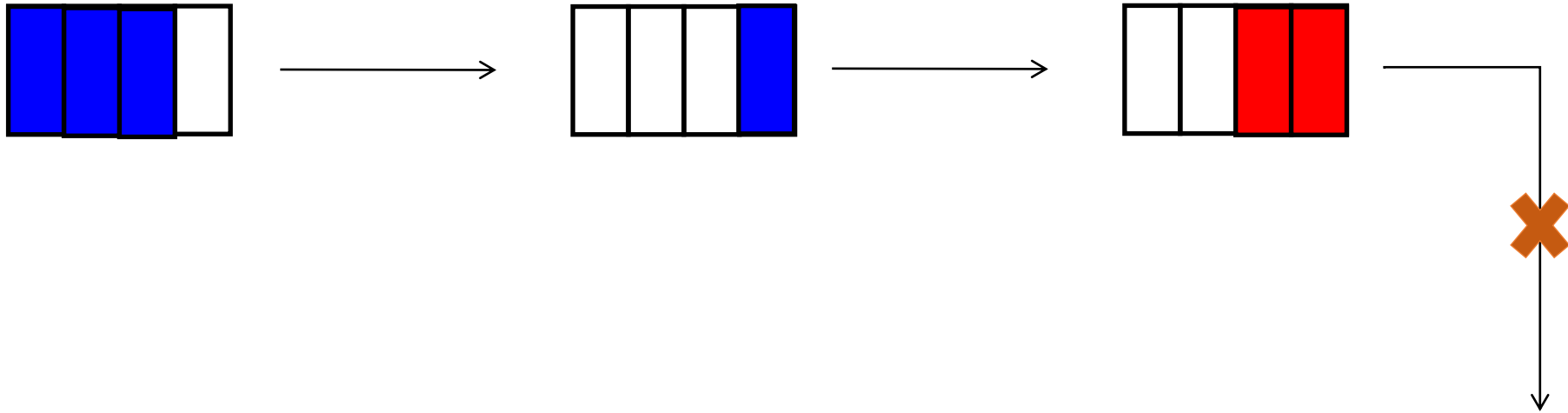
Virtual Cut Through

e.g., Packet from 0 to 5, with contention



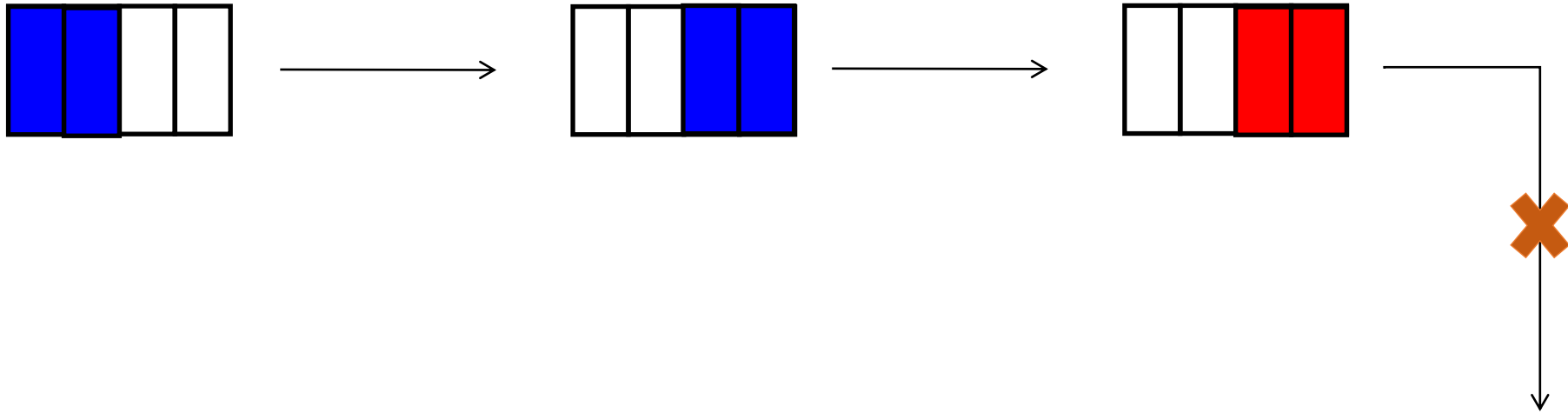
Virtual Cut Through

e.g., Packet from 0 to 5, with contention



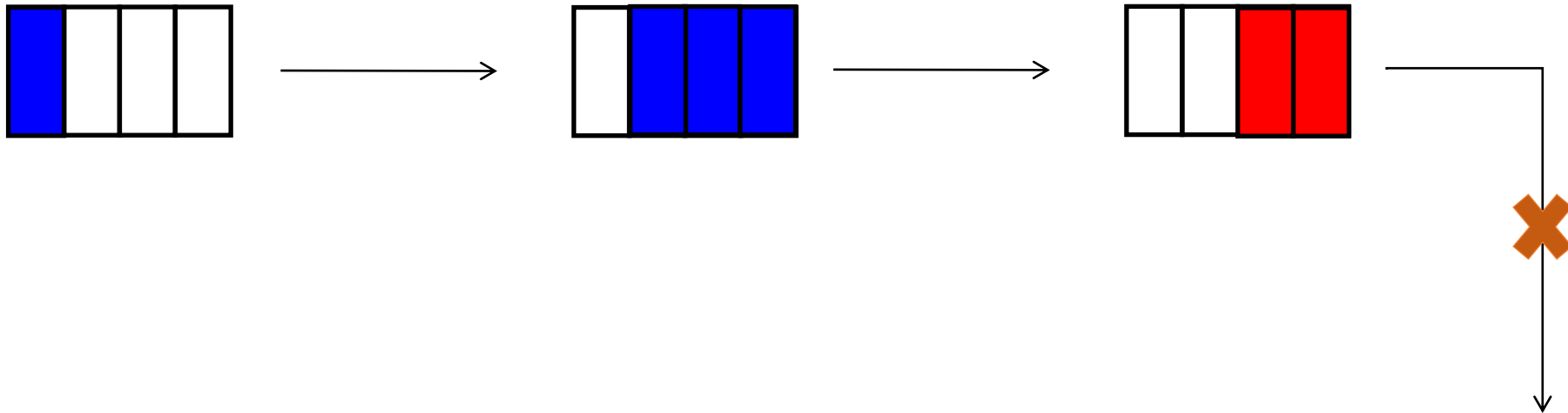
Virtual Cut Through

e.g., Packet from 0 to 5, with contention



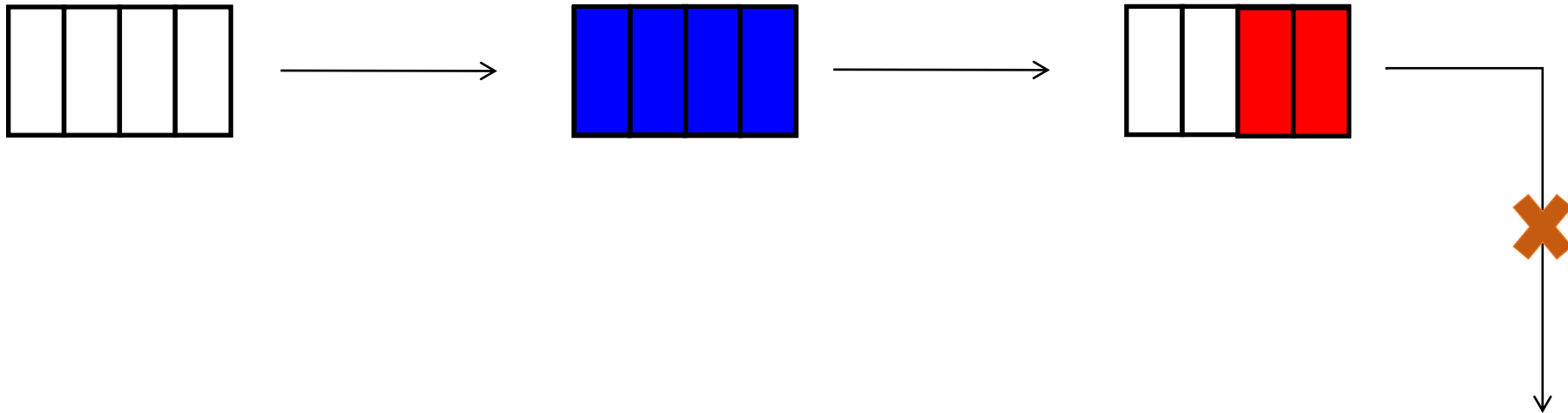
Virtual Cut Through

e.g., Packet from 0 to 5, with contention



Virtual Cut Through

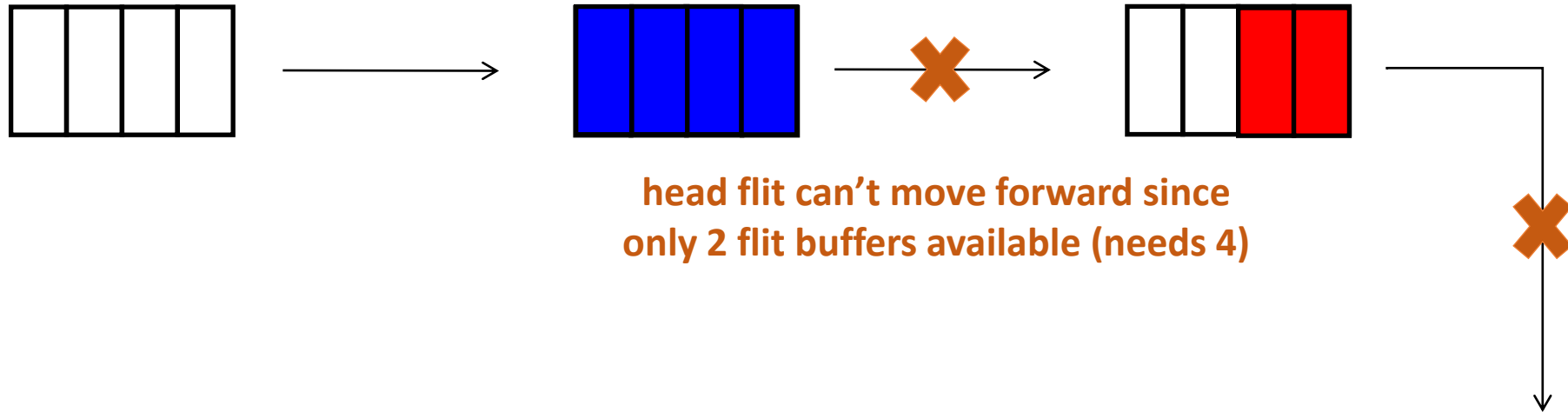
e.g., Packet from 0 to 5, with contention



Virtual Cut Through

e.g., Packet from 0 to 5, with contention

- Throughput suffers due to inefficient buffer utilization



Flit-Based Flow Control

Flit can proceed to next router when there is buffer space available for that flit

- Allocates buffers on a flit-by-flit basis
- Helps routers meet tight area and power constraints

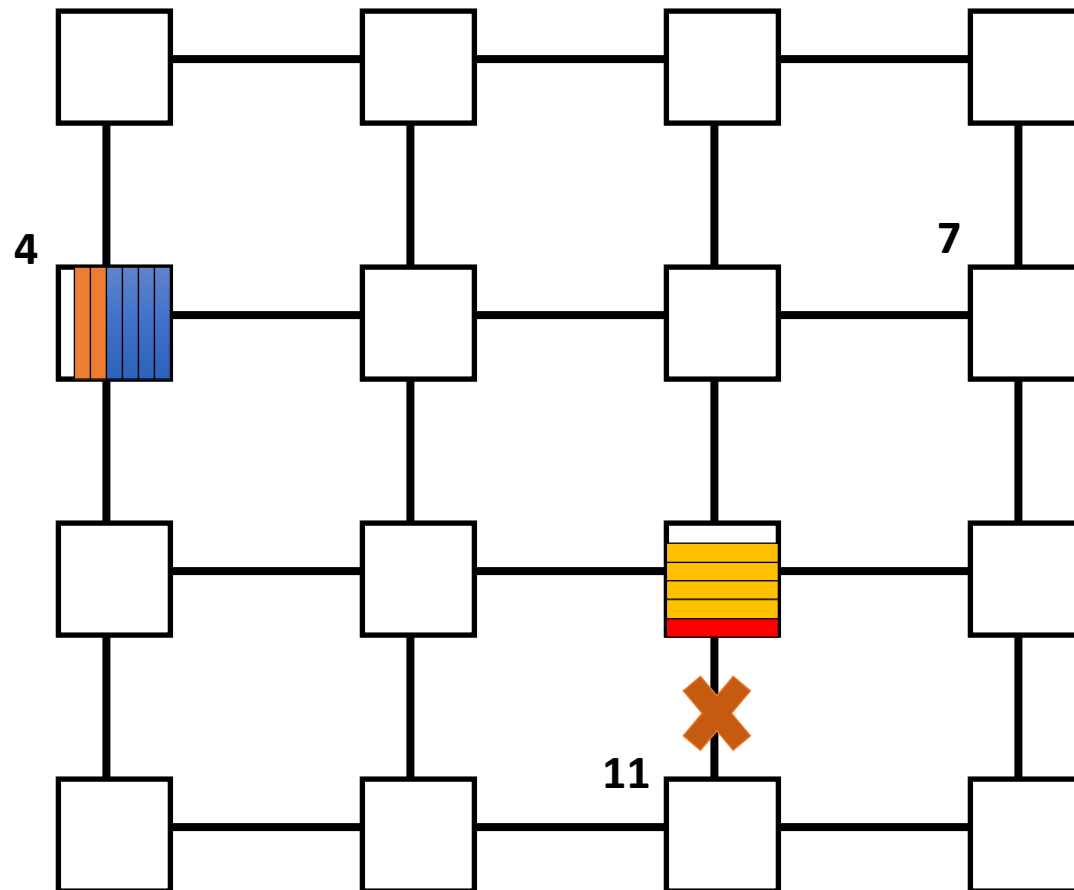
Wormhole Flow Control

Links and buffers are allocated one flit at a time, but flits of different packets cannot be interleaved

- Routing computation is only performed on head flit, so all flits must still convoy together (i.e., other packets' flits can't squeeze in between)
- More efficient buffer utilization; don't need packet-sized buffers per router
- Inefficient link utilization; if head flit becomes blocked, all links between head and tail flits are idle (i.e., cannot be re-allocated to different packet)
- Suffers from head-of-line blocking

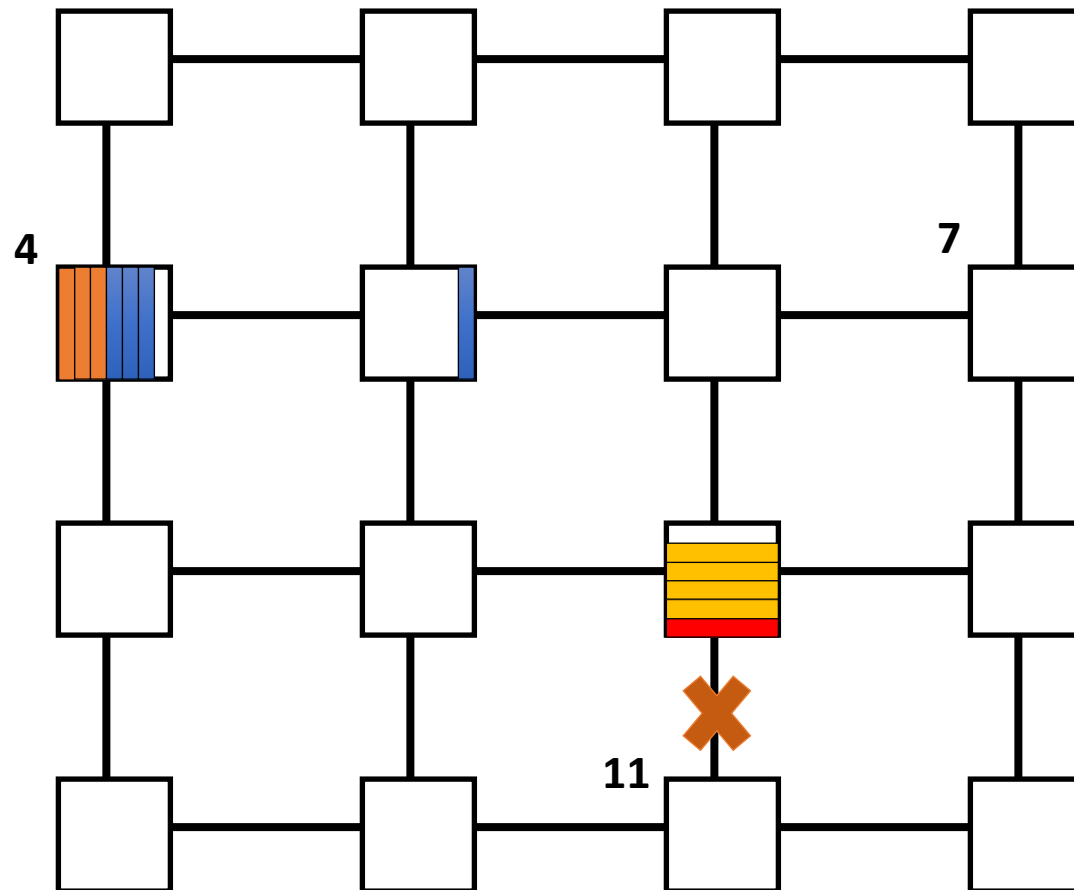
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



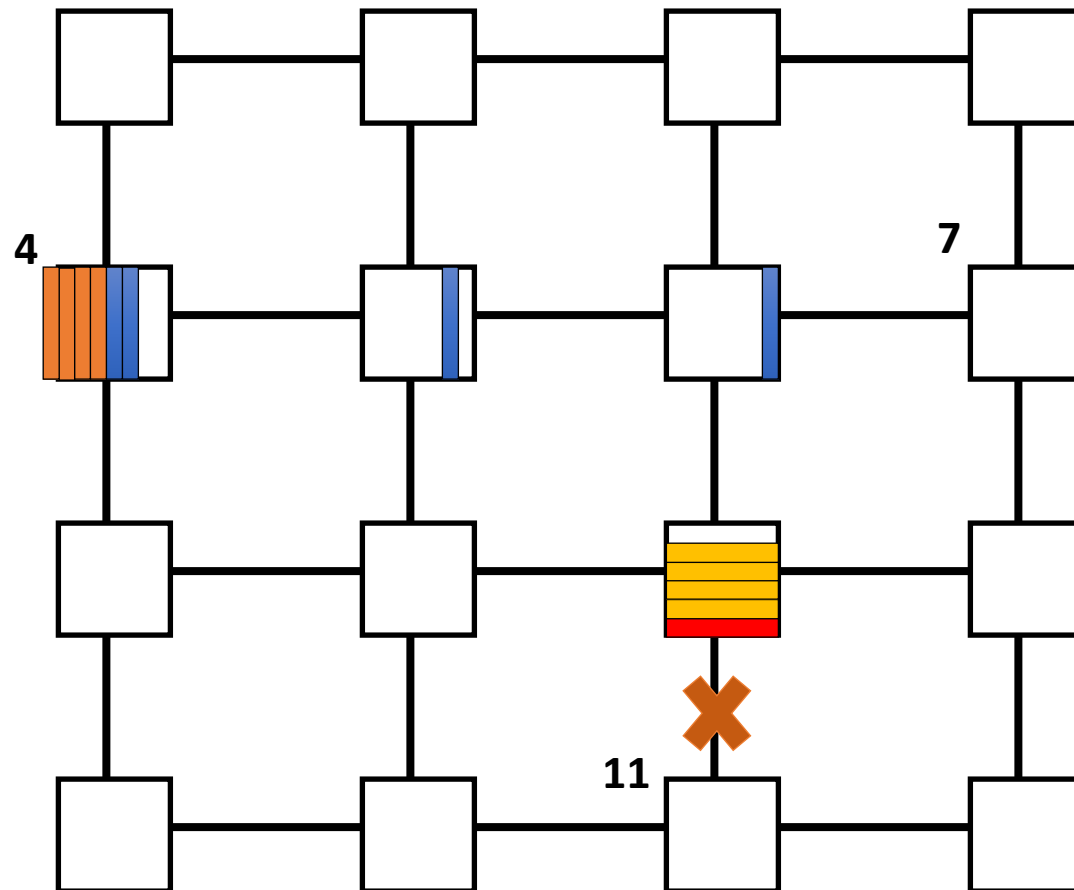
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



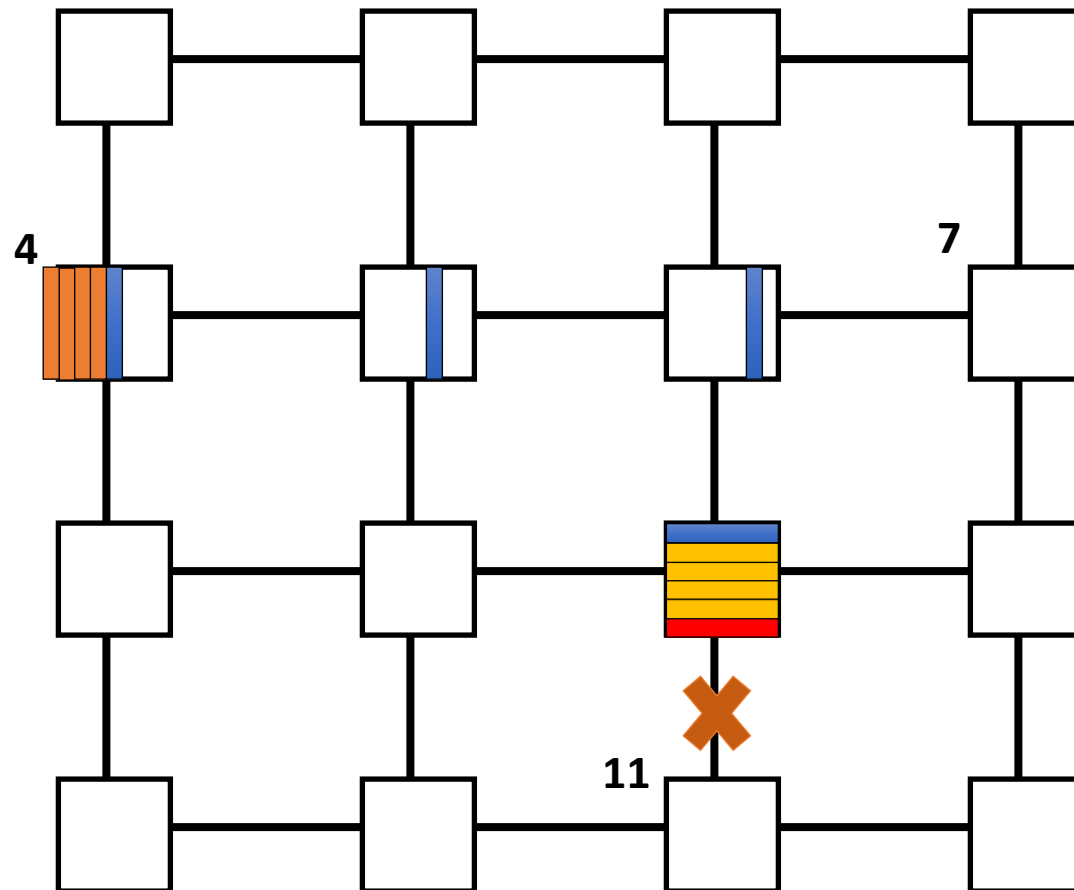
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



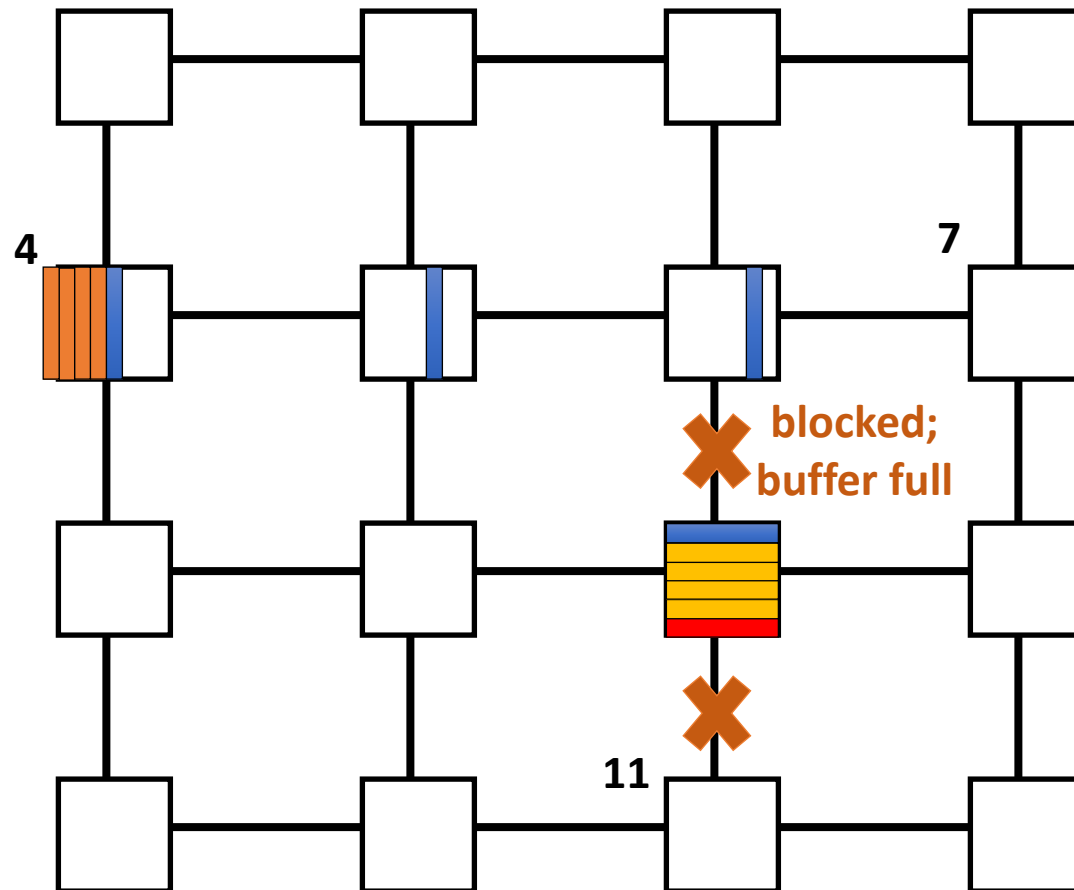
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



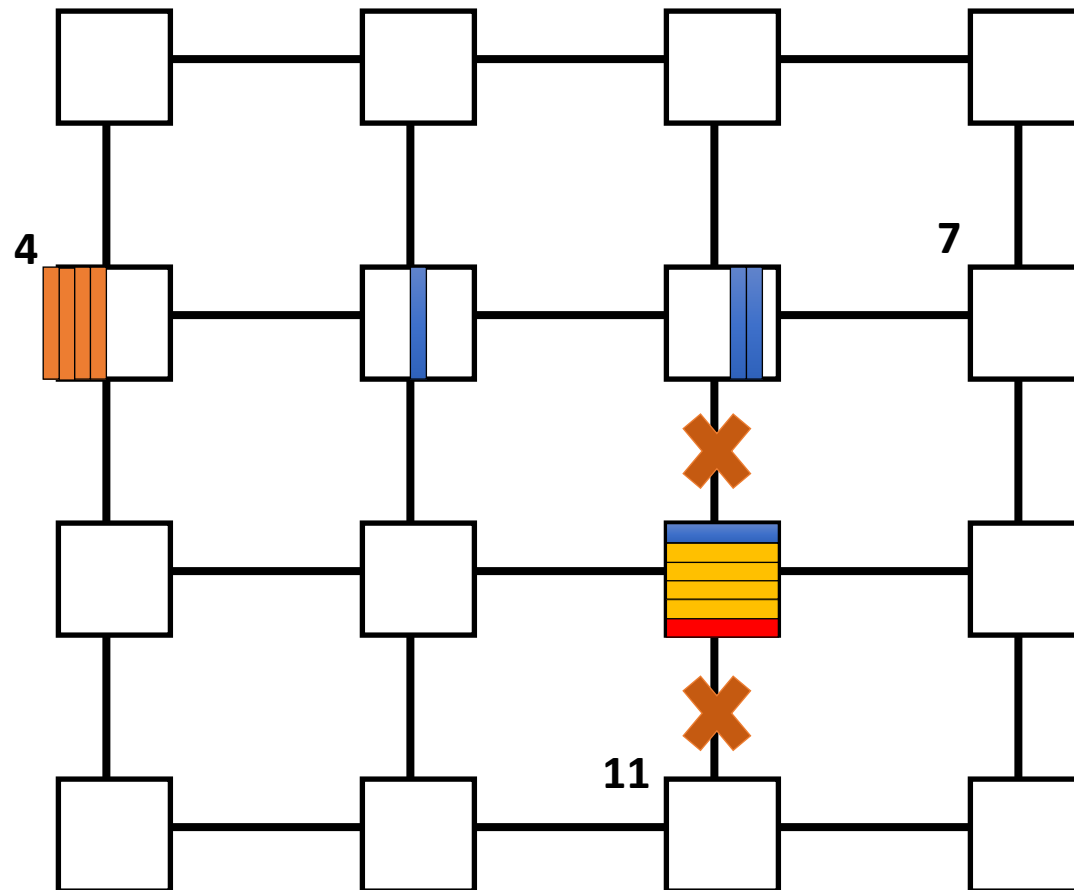
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



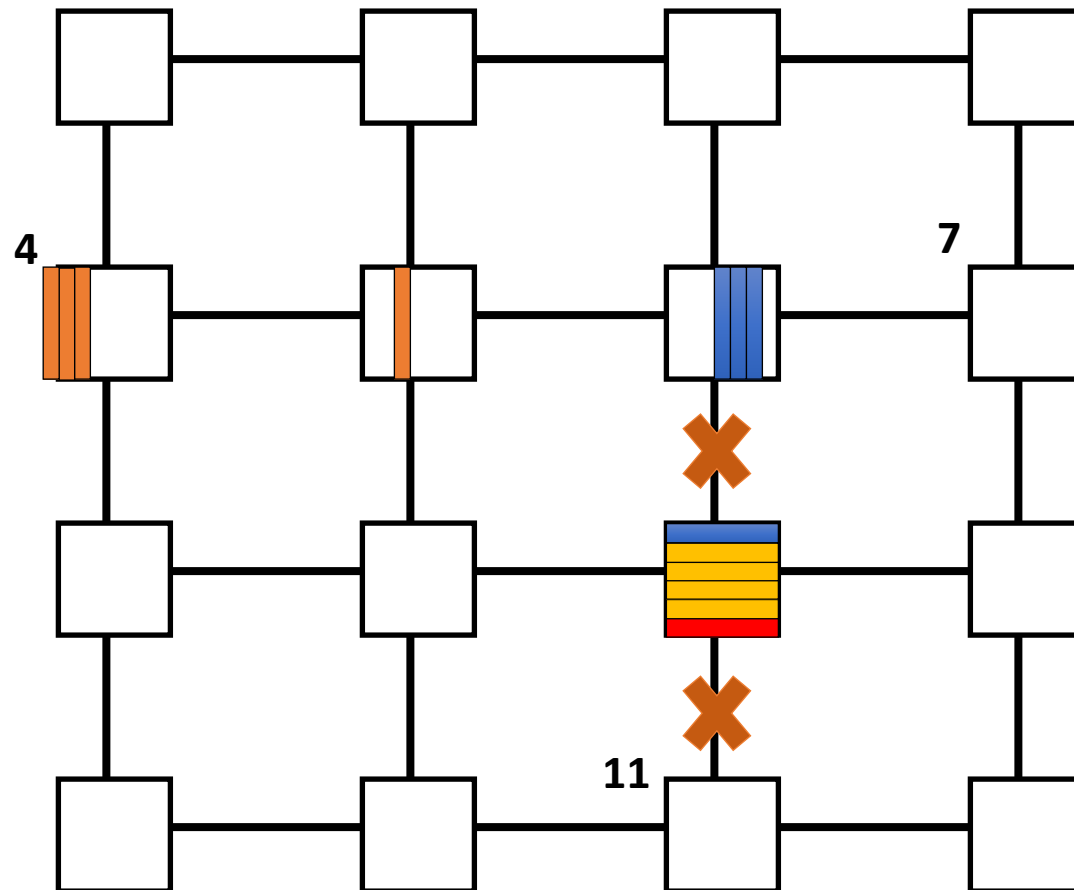
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



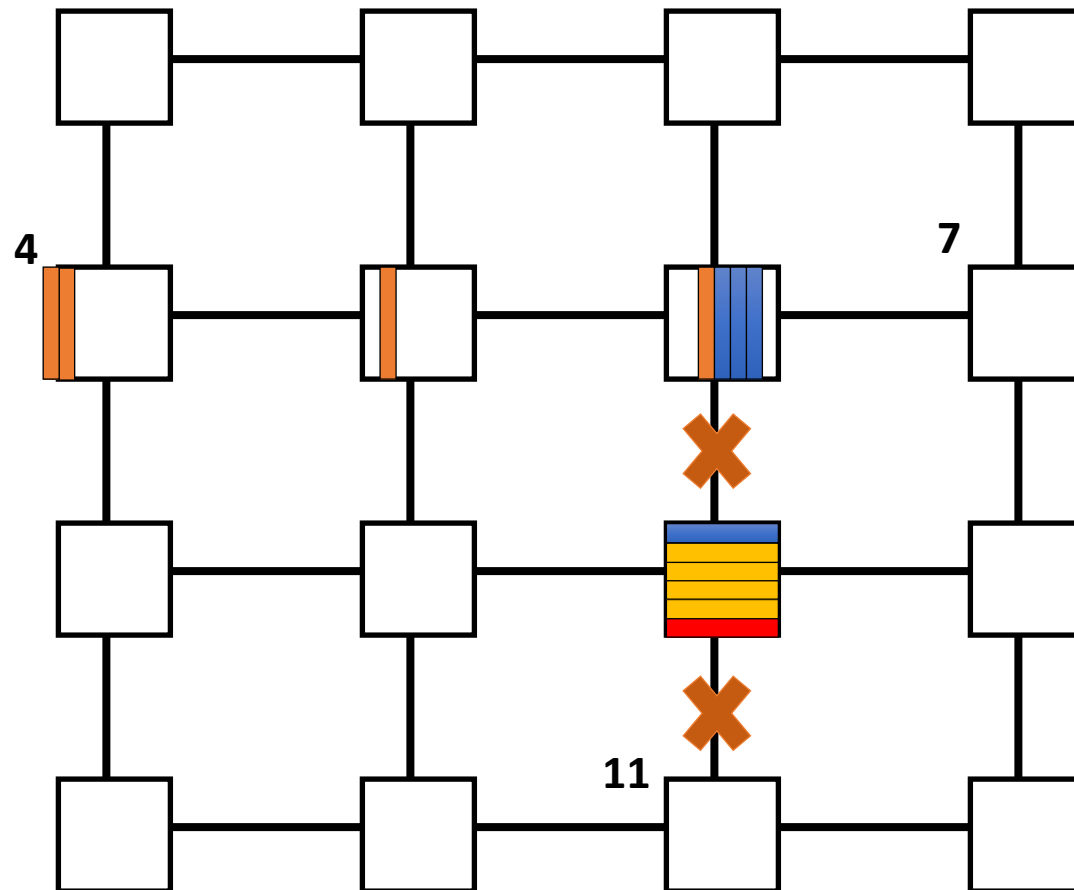
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



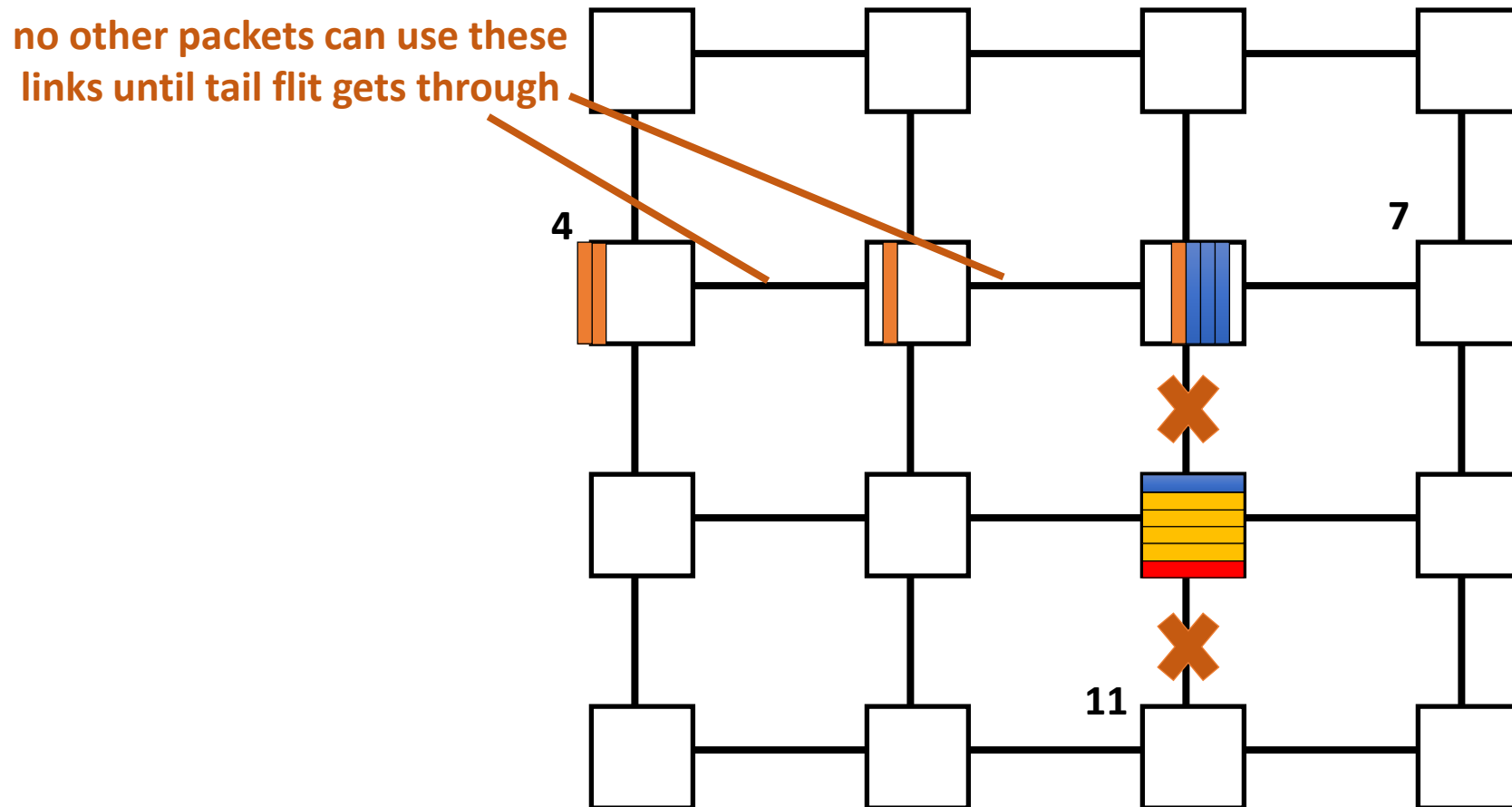
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



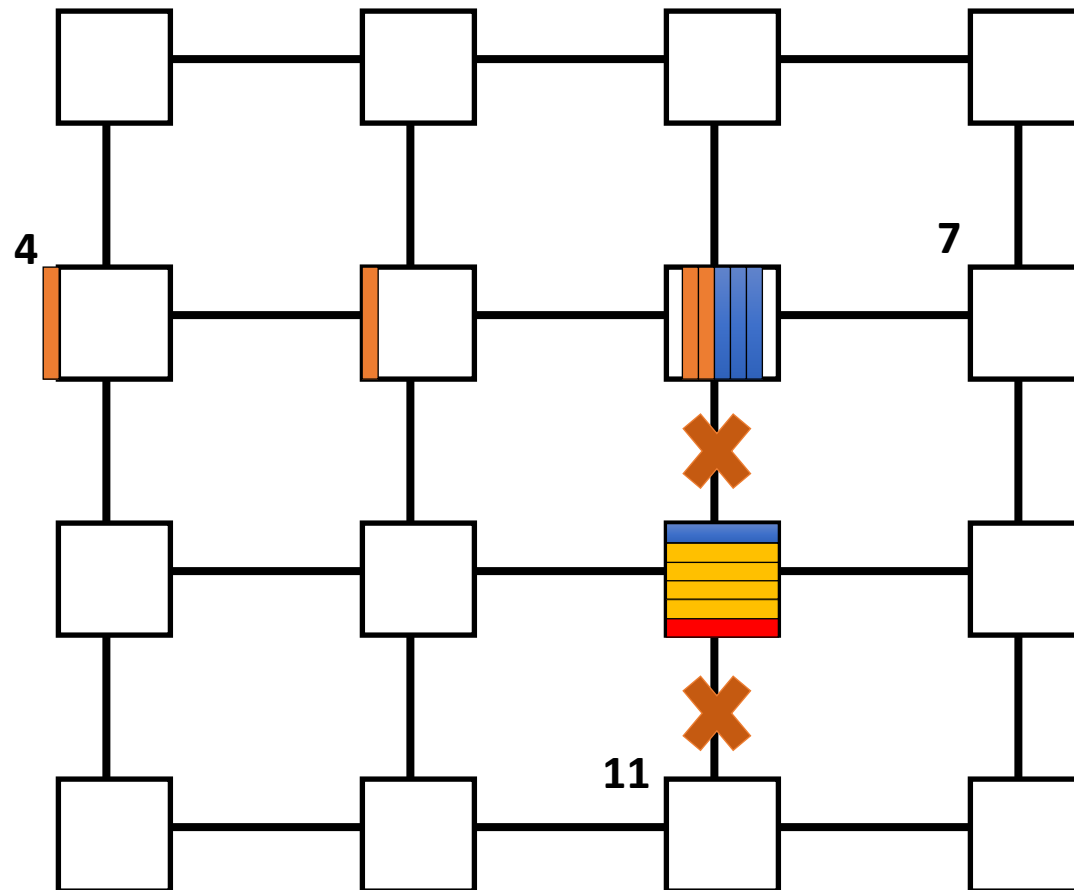
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



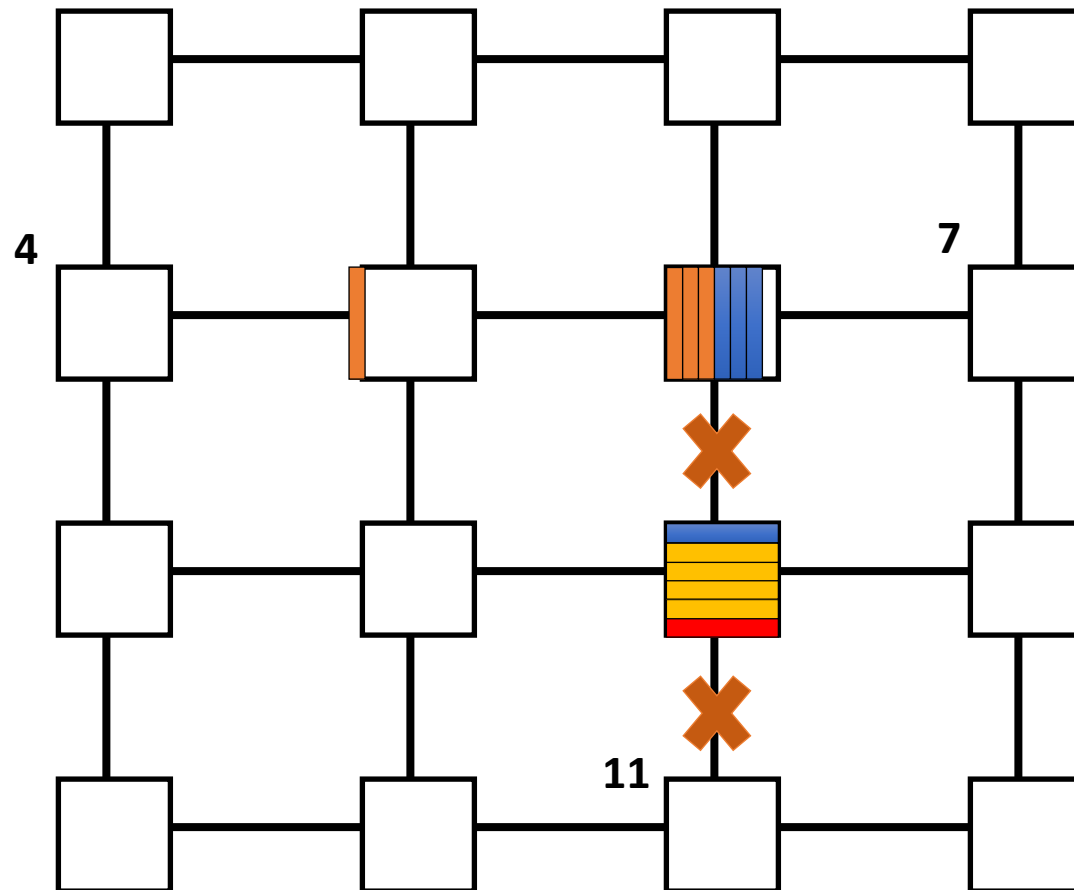
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



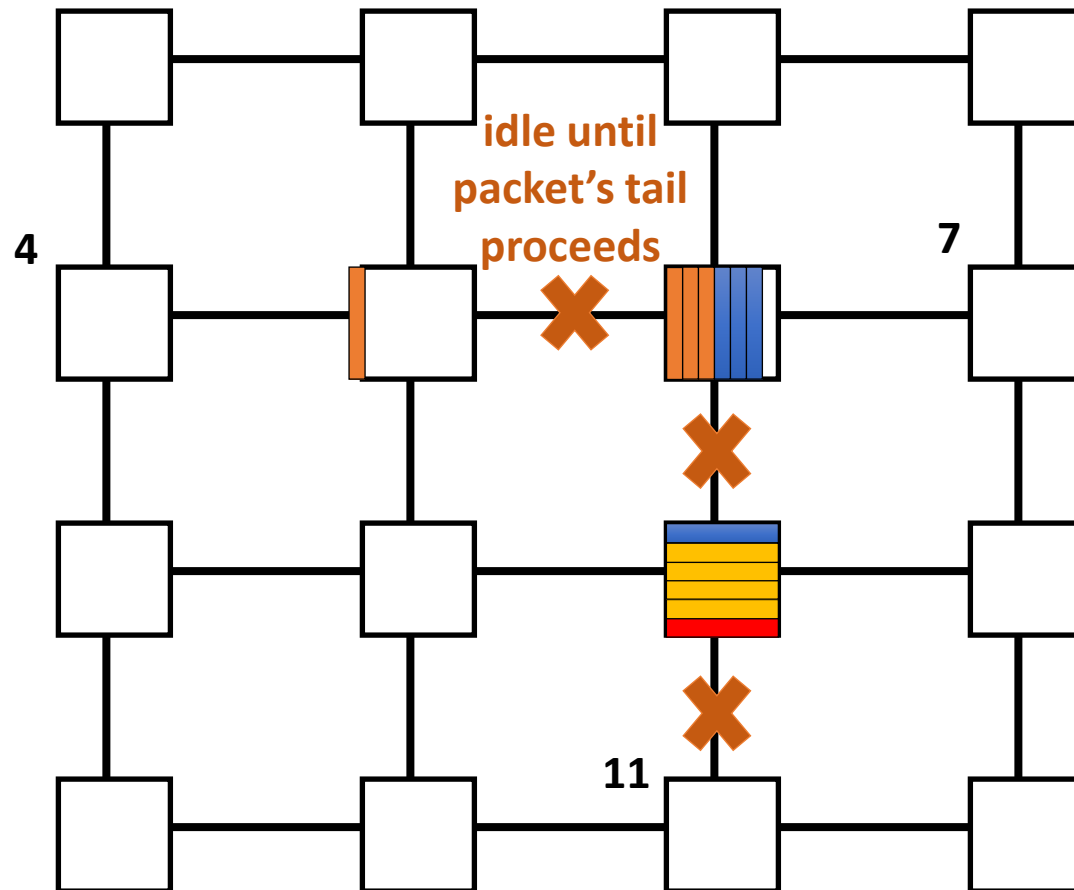
Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



Wormhole Flow Control

e.g., Assuming 6-flit buffers, **packet from 4 to 11** and **packet from 4 to 7**



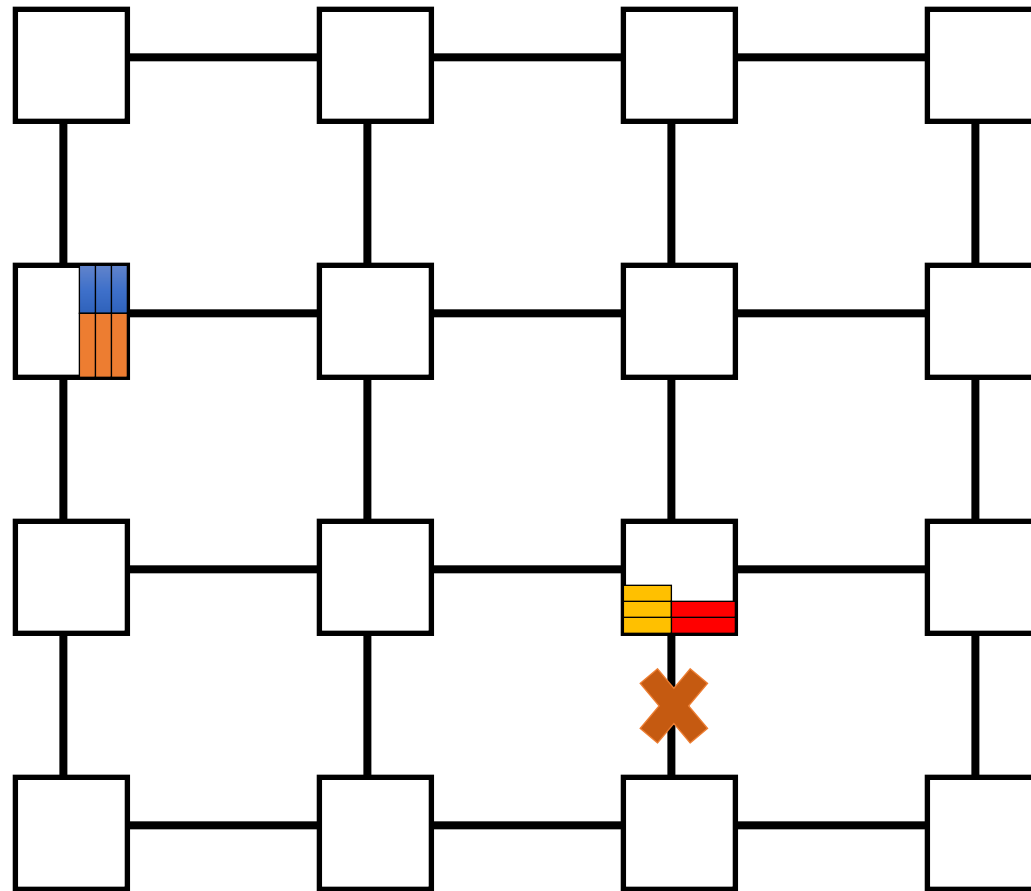
Virtual Channels

Multiple buffers per input port (i.e., virtual channels) sharing one physical link

- Time-multiplex flits of different VCs on link; packets compete for physical link on flit-by-flit basis
- Can be applied to any flow control mechanism; initially proposed for deadlock-freedom
- Can combat head-of-line blocking
- Improves link utilization; packets can bypass other packets on different VCs

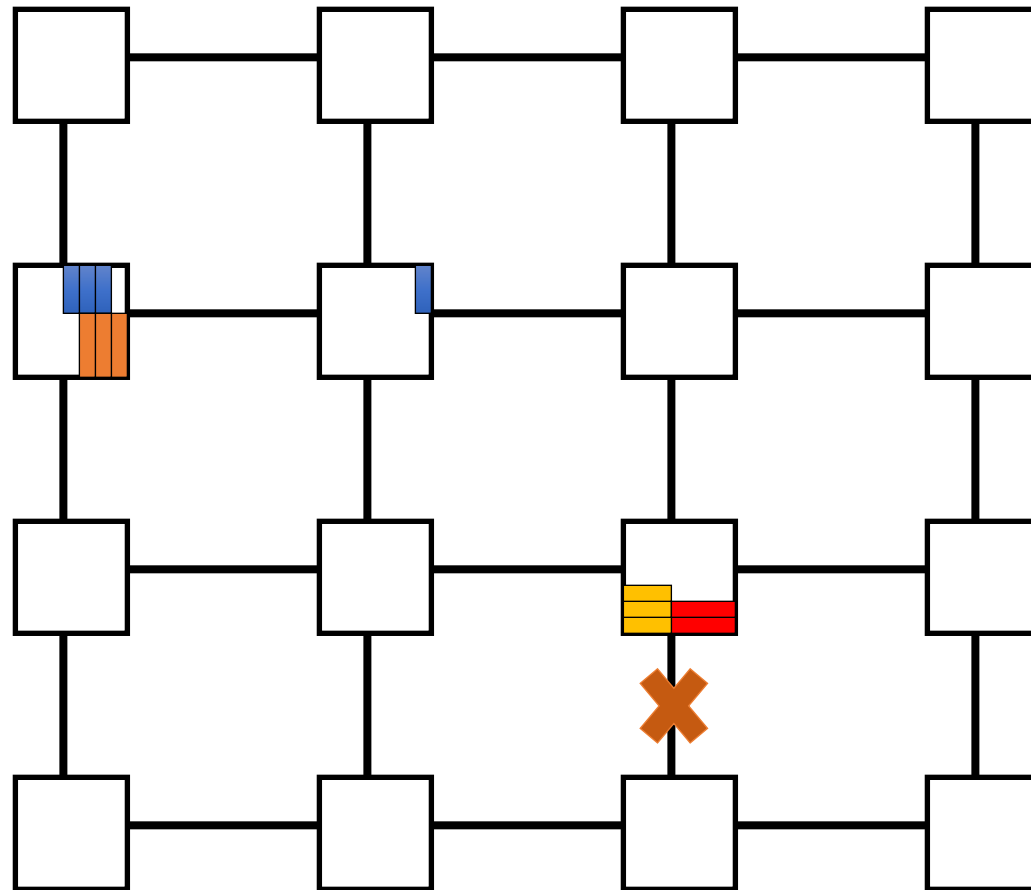
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



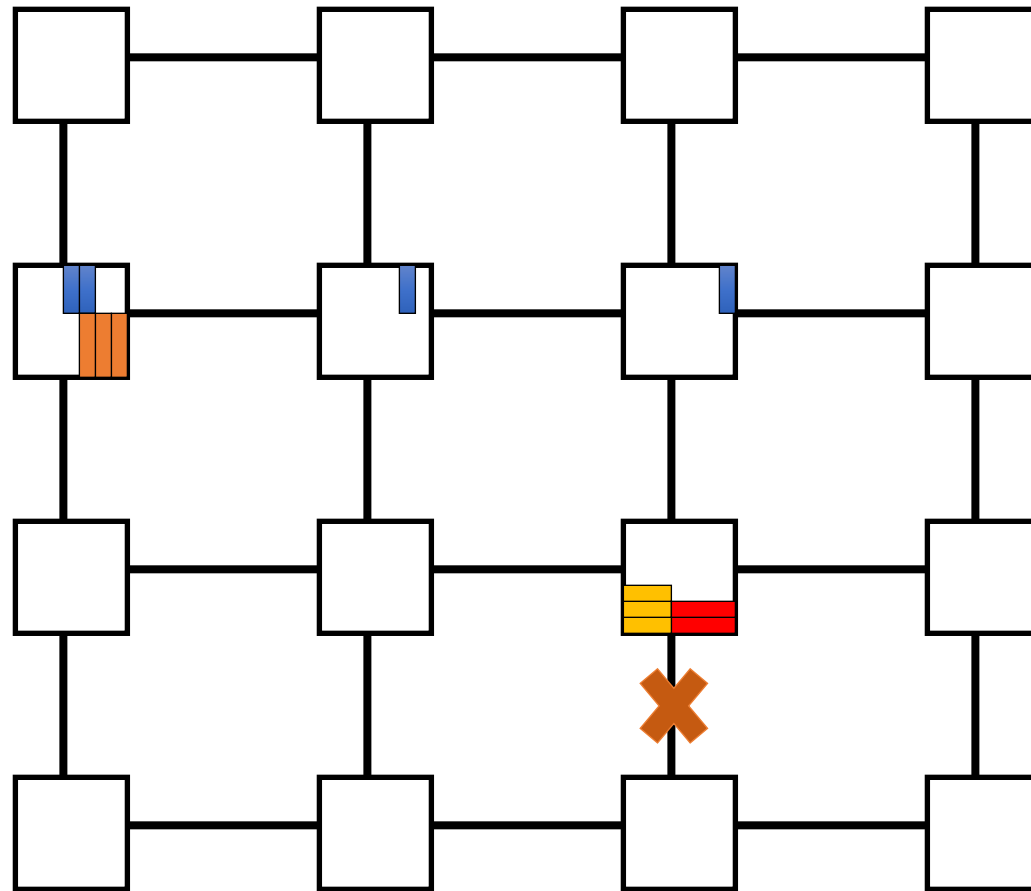
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



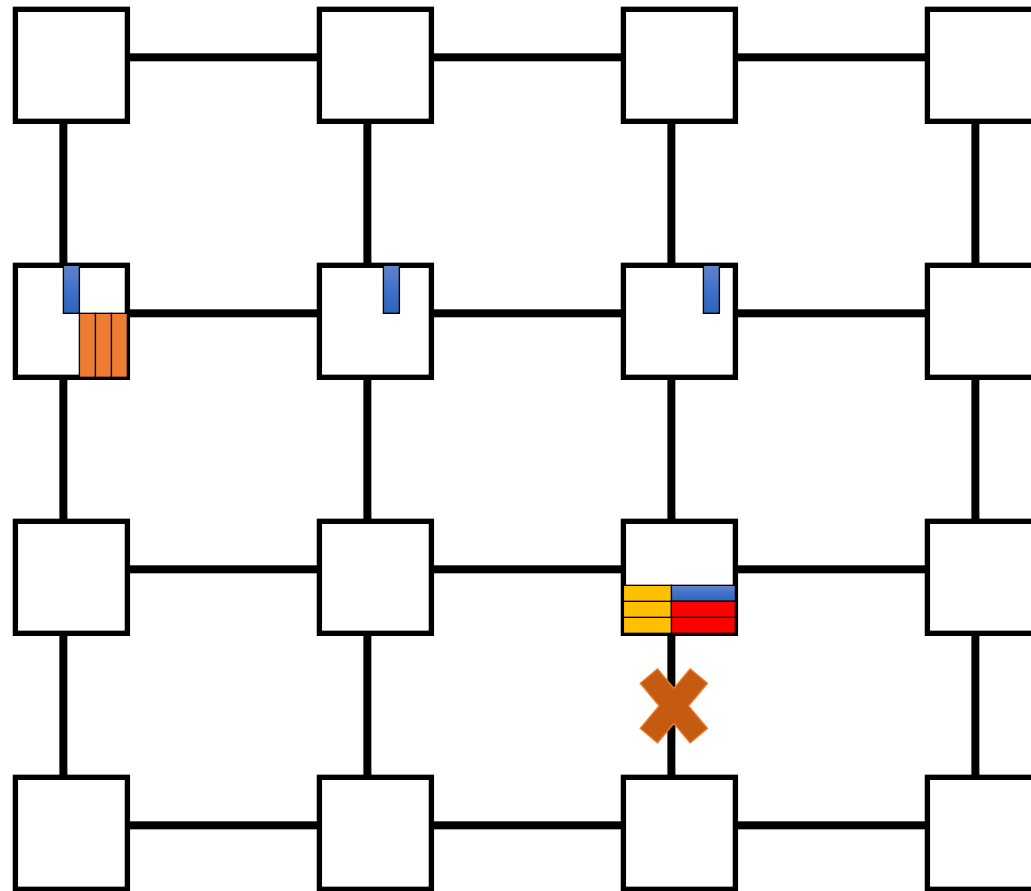
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



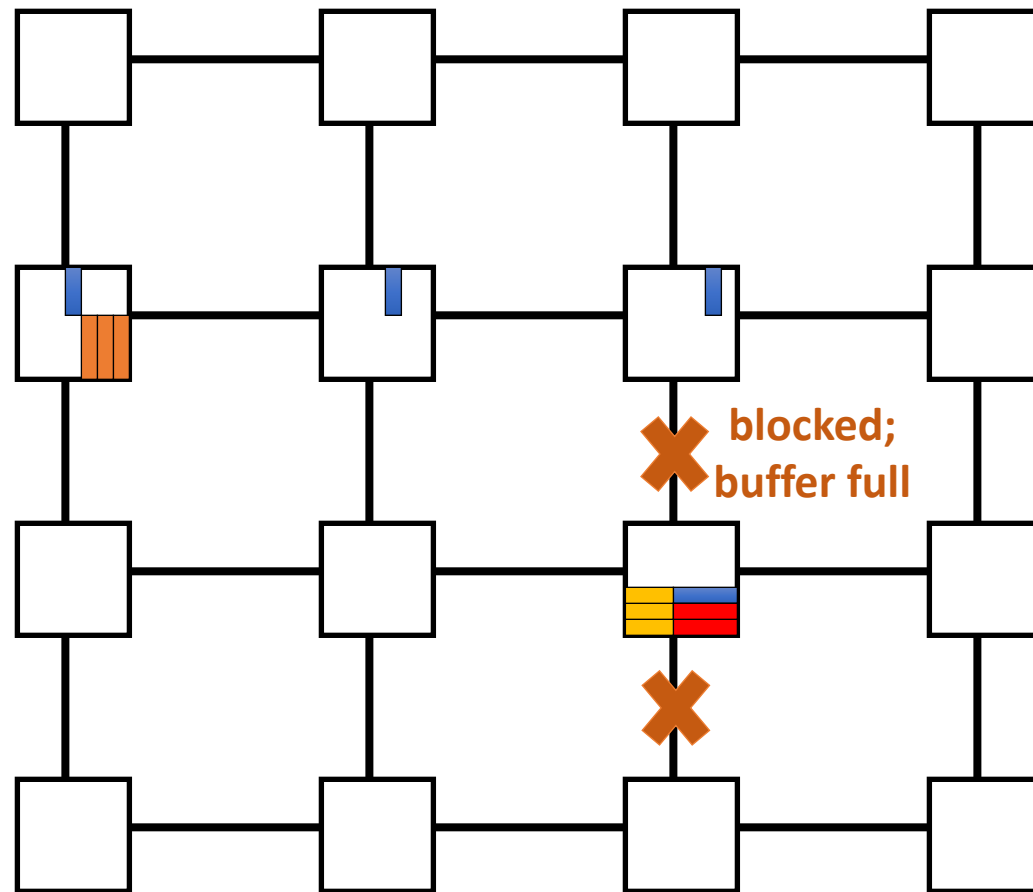
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



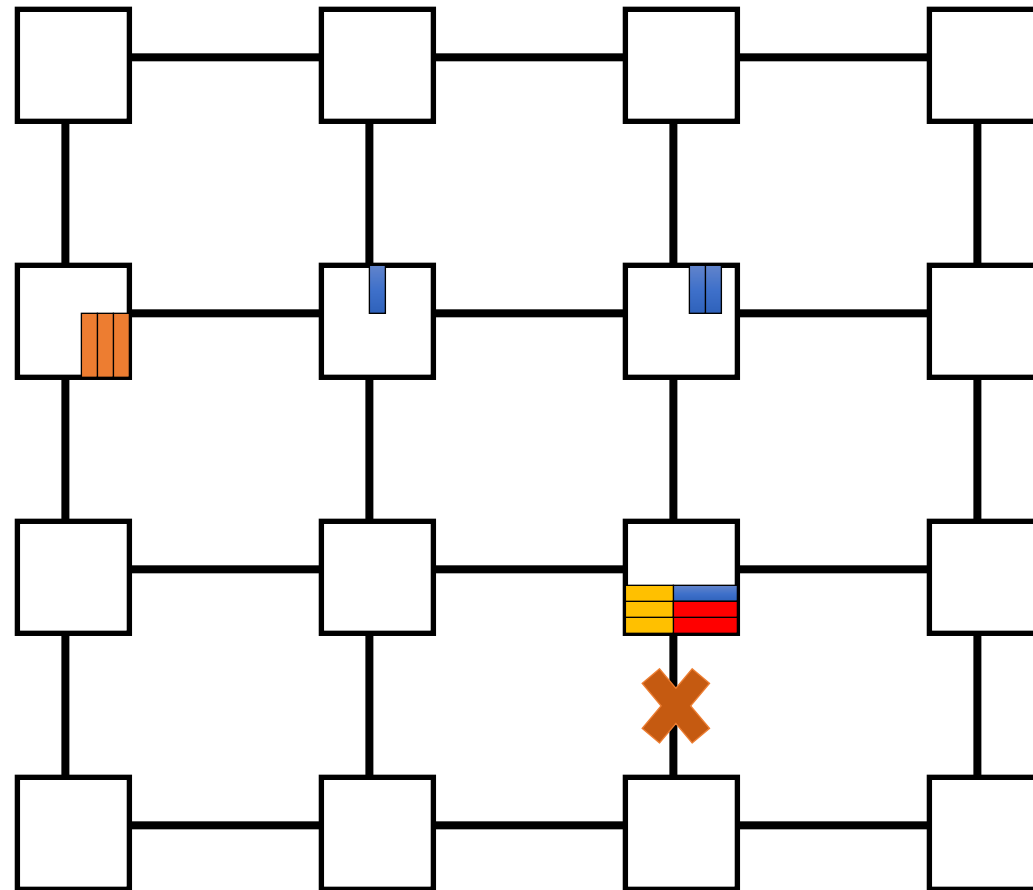
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



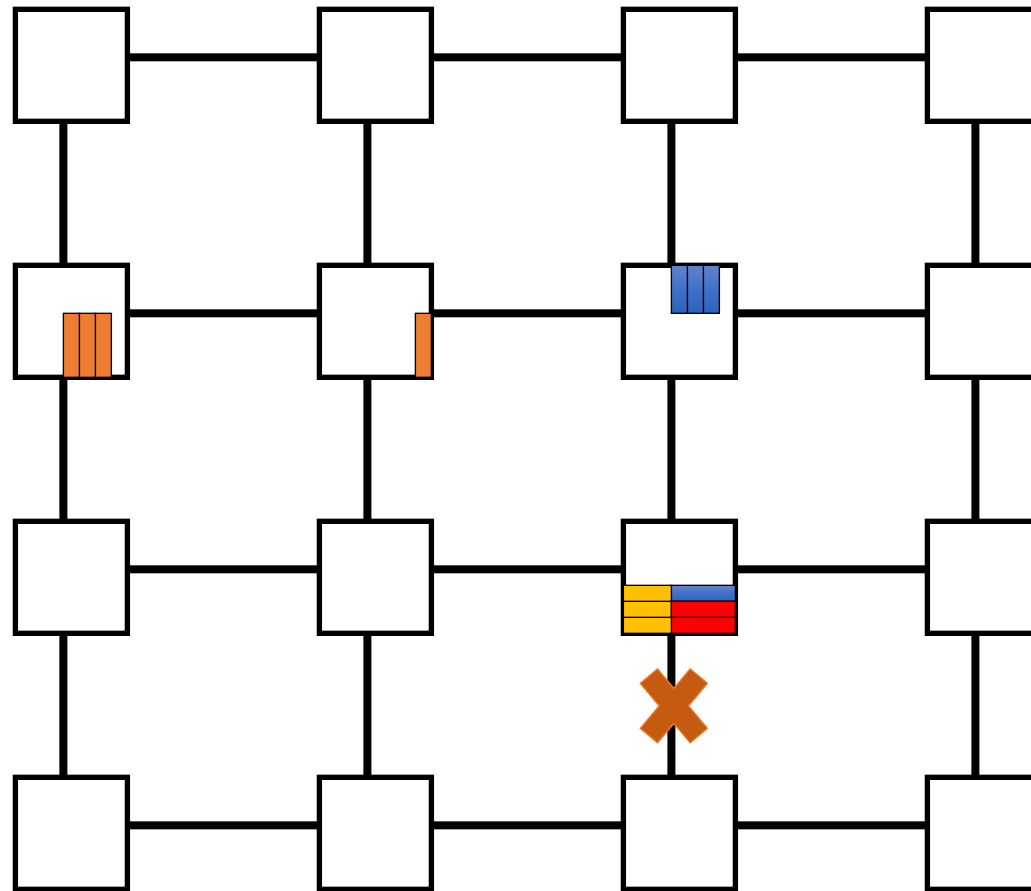
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



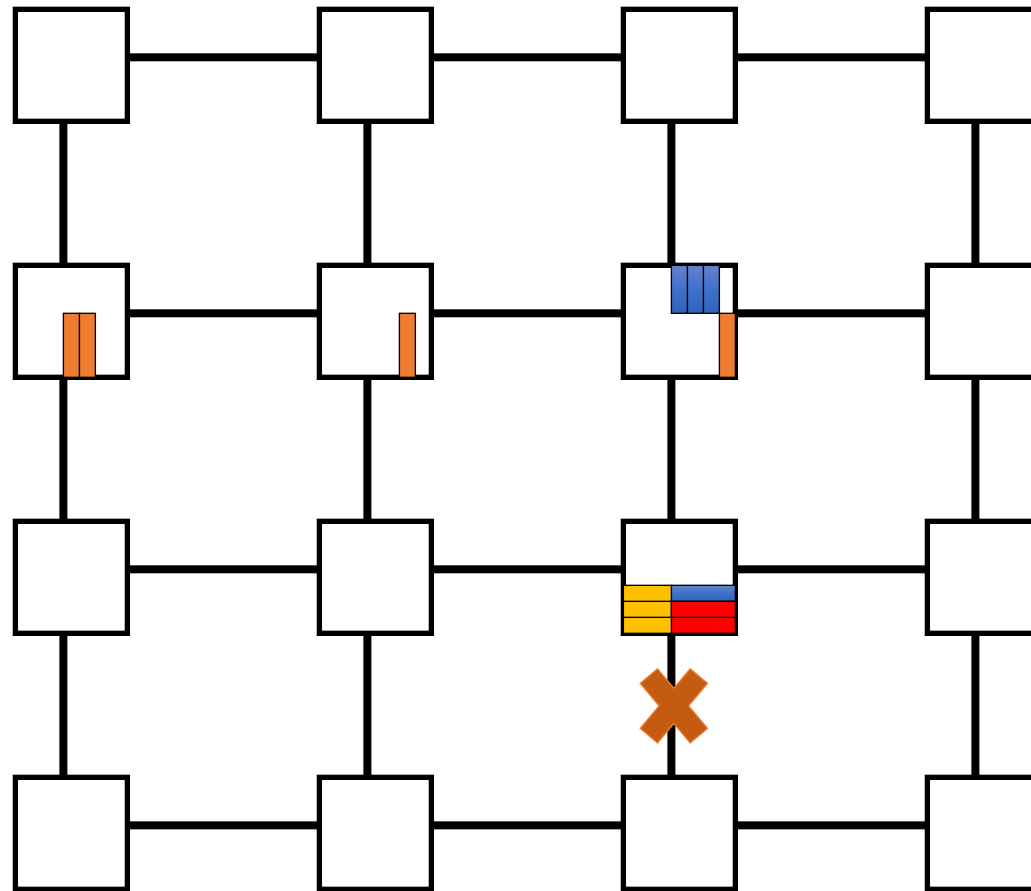
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



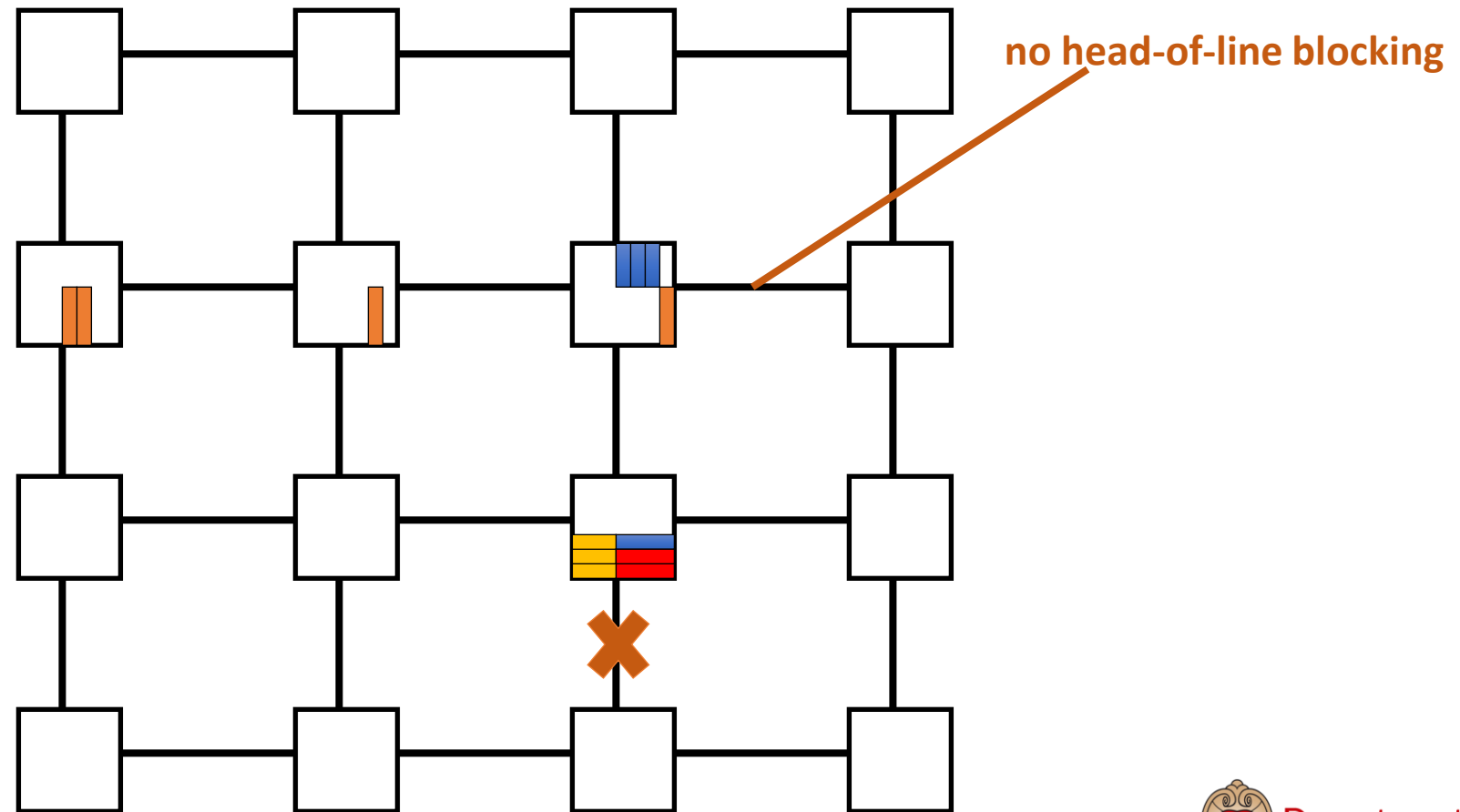
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



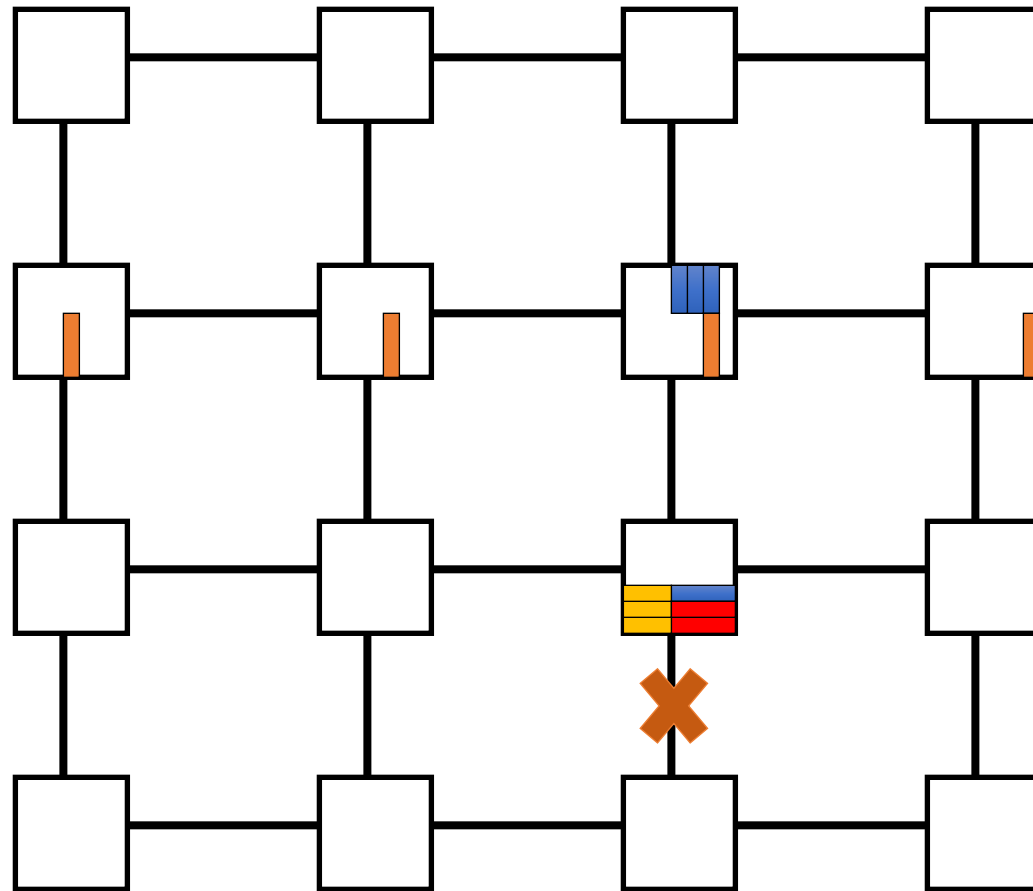
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



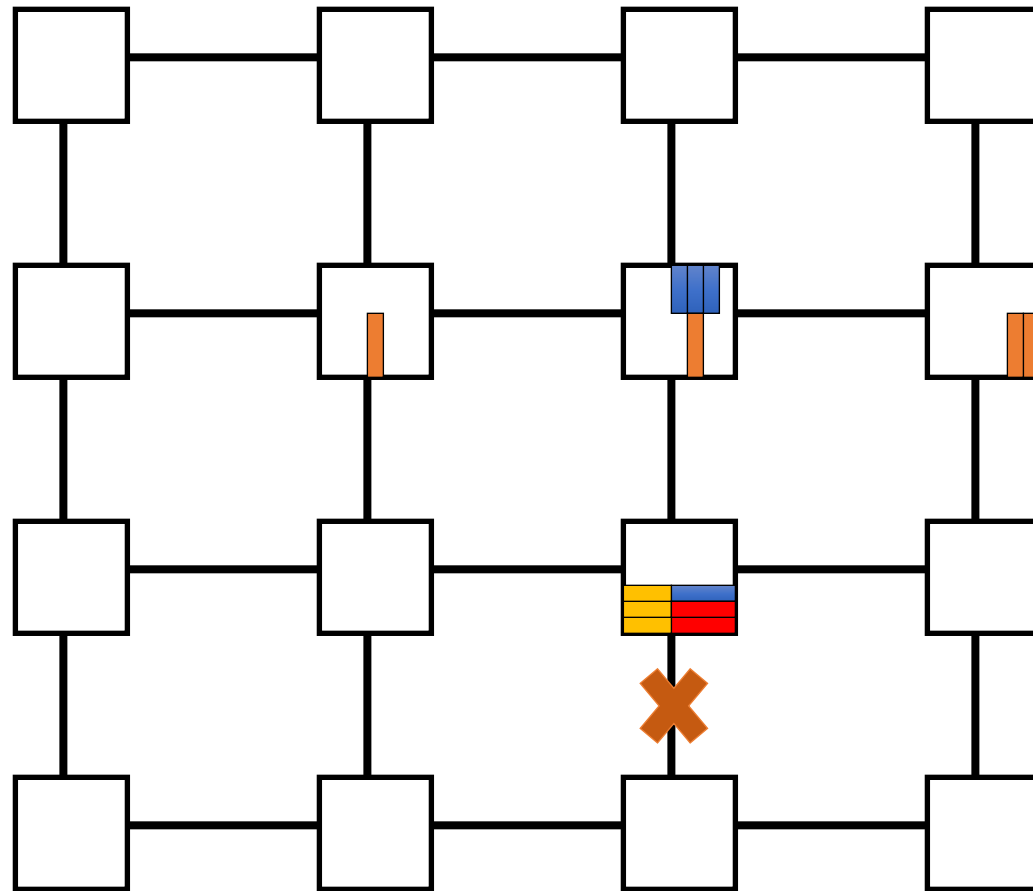
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



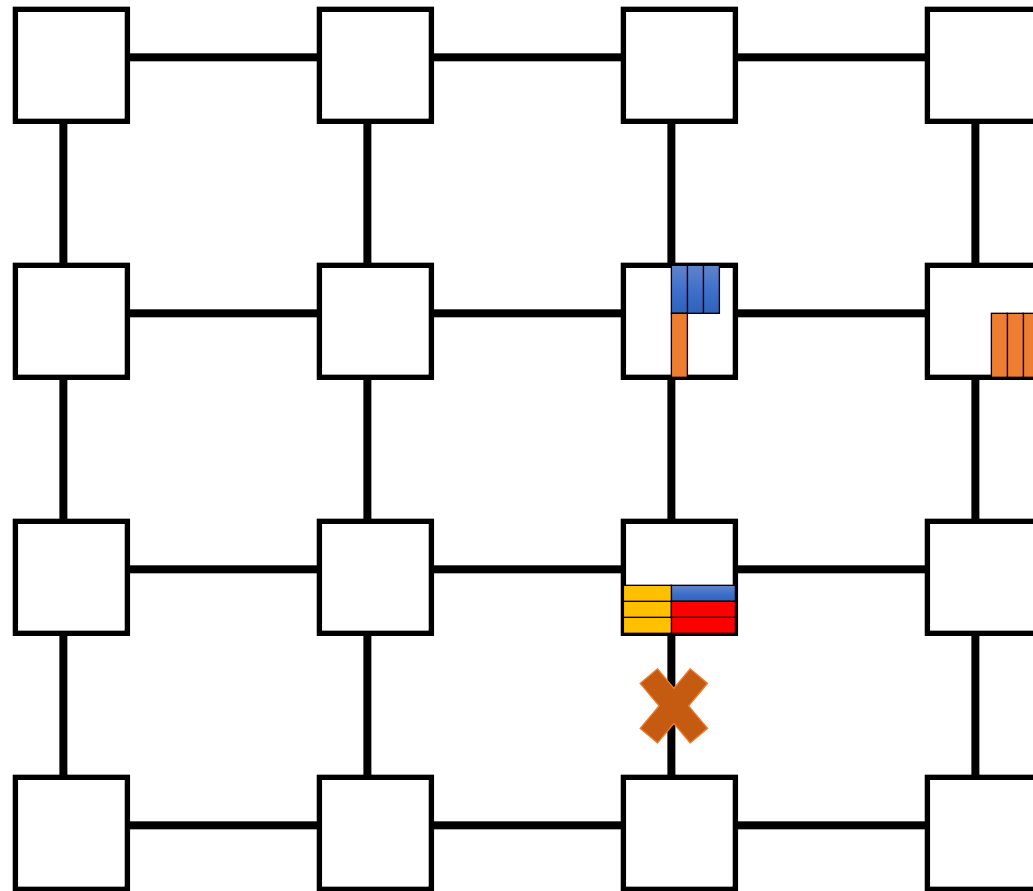
Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



Virtual Channels

e.g., Assuming two VCs with 3-flit buffers each



Virtual Channels

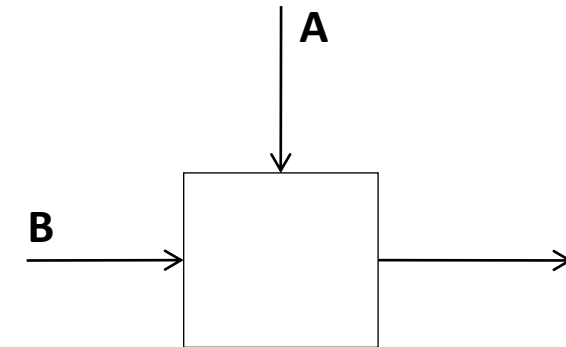
e.g., Assuming 3-flit buffers, packets A and B on separate VCs

in (N)

in (W)

out (E)

time



Virtual Channels

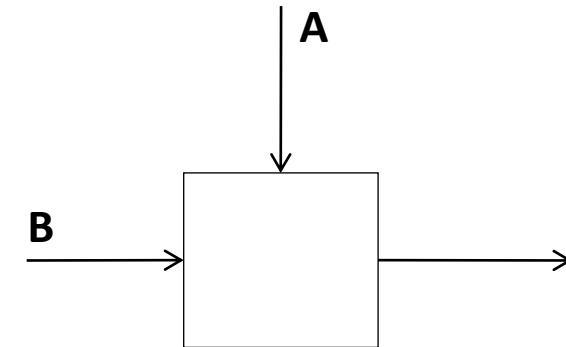
e.g., Assuming 3-flit buffers, packets A and B on separate VCs

in (N) AH

in (W) BH

out (E)

time



Virtual Channels

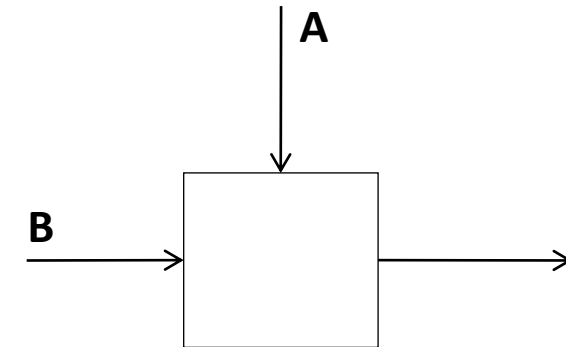
e.g., Assuming 3-flit buffers, packets A and B on separate VCs

in (N) AH

in (W) BH

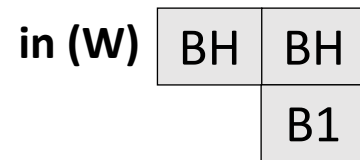
out (E) AH

time

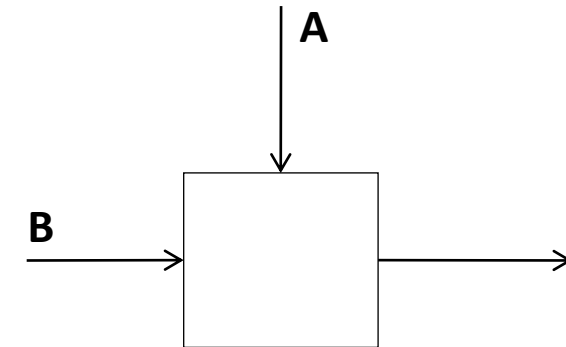


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

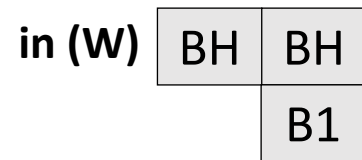


time

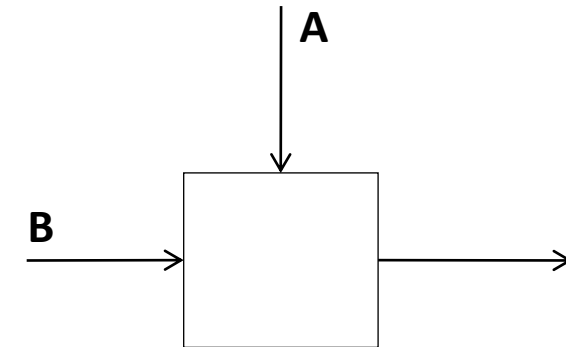


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

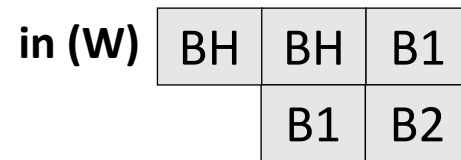
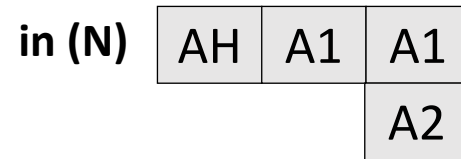


time

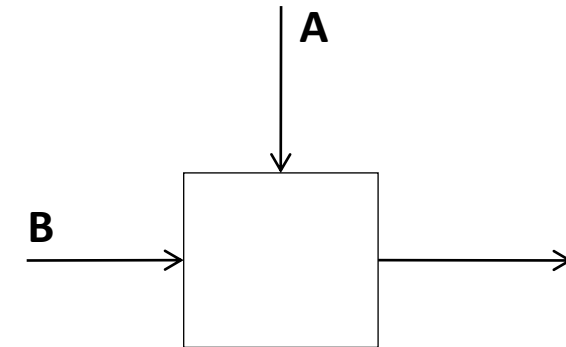


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

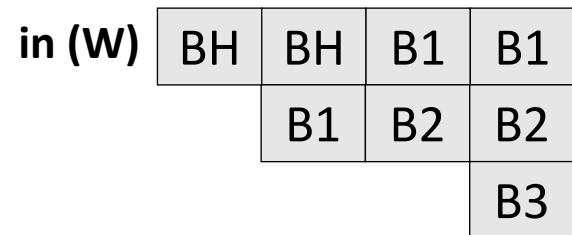
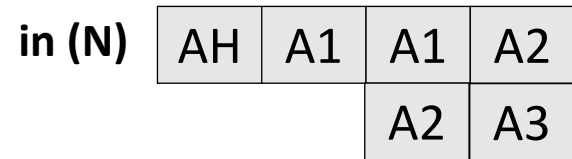


time

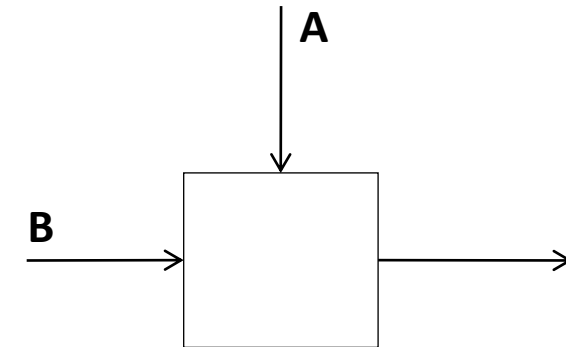


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

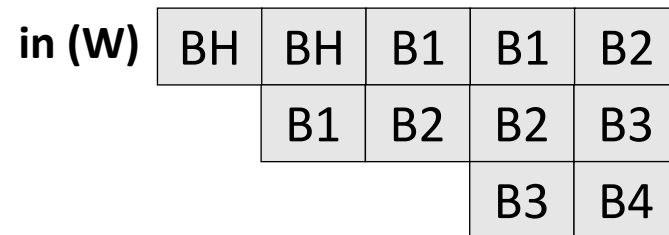
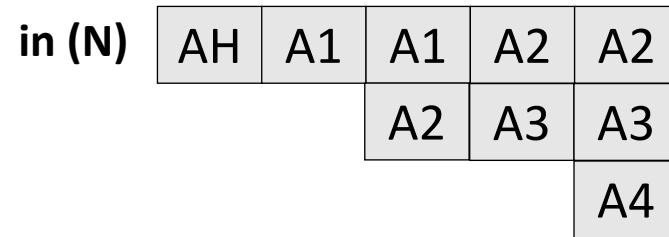


time

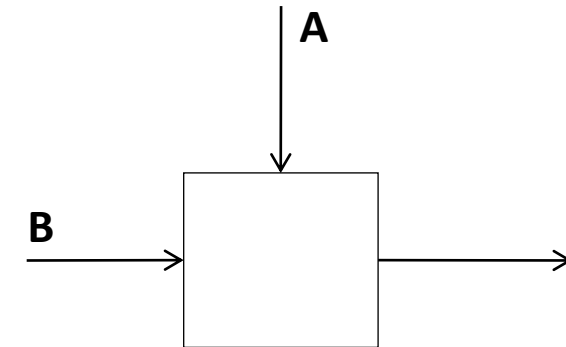


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

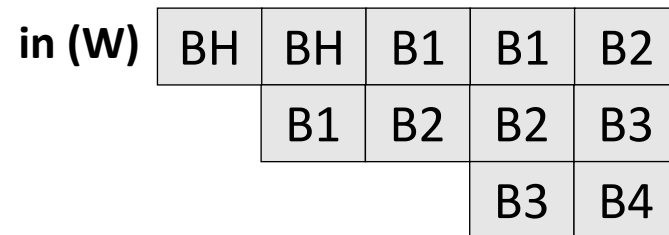
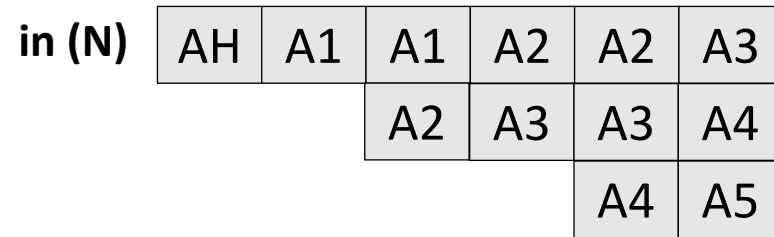


time

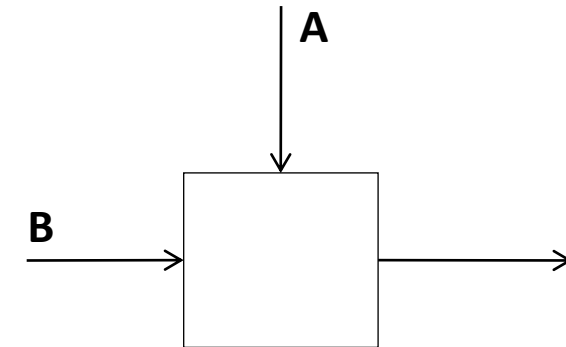


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

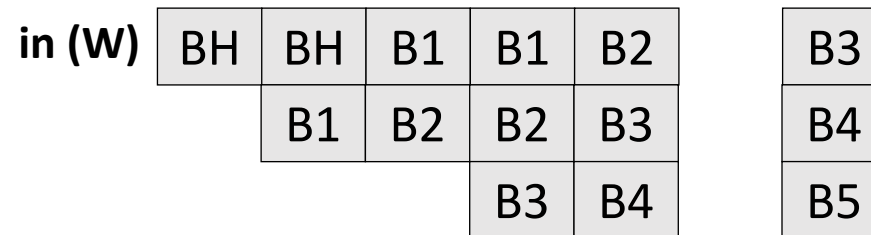
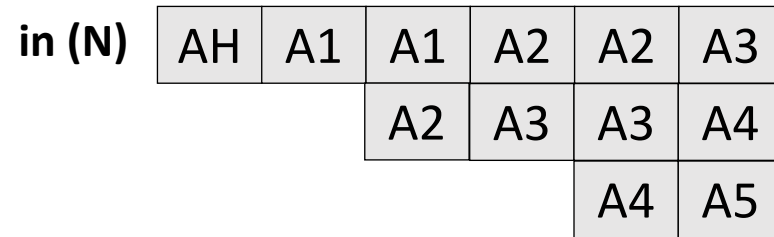


time

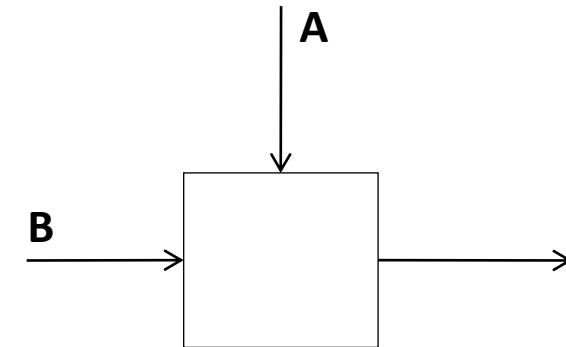


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

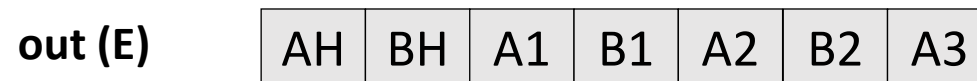
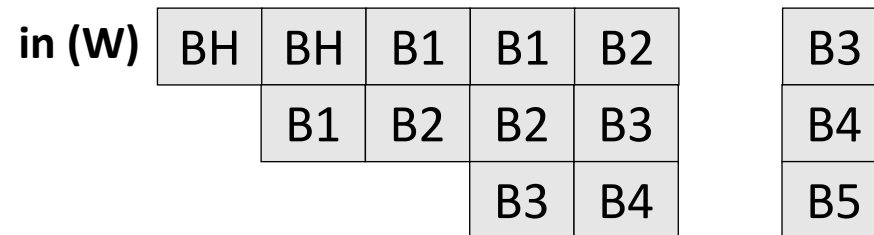
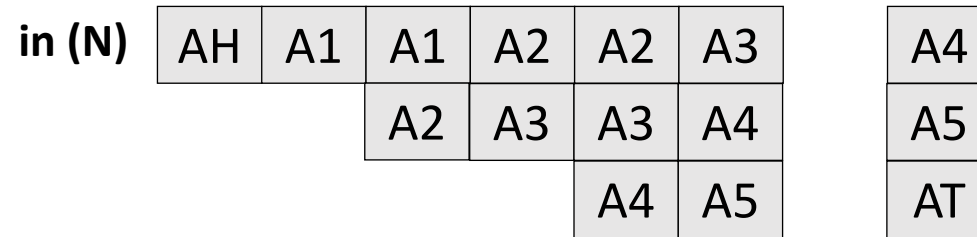


time

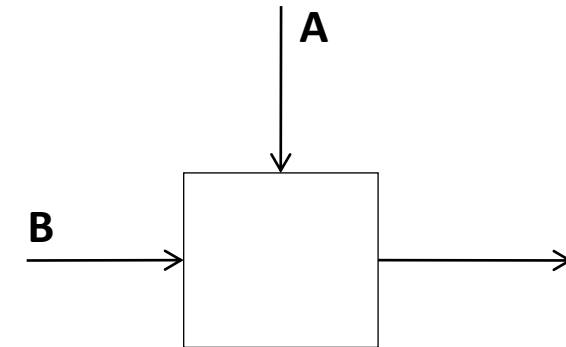


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

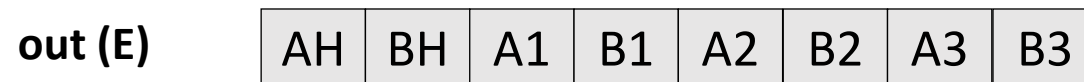
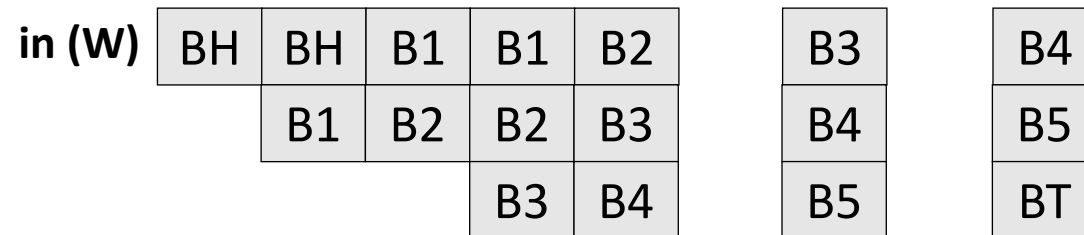
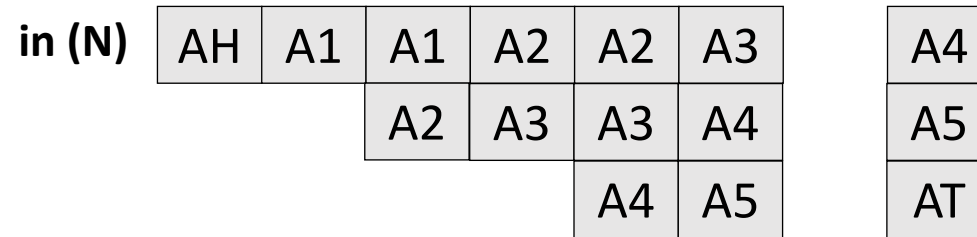


time

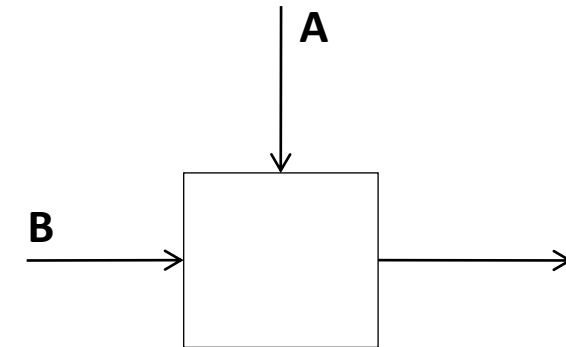


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

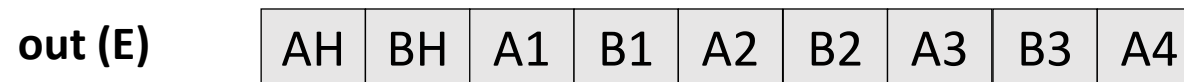
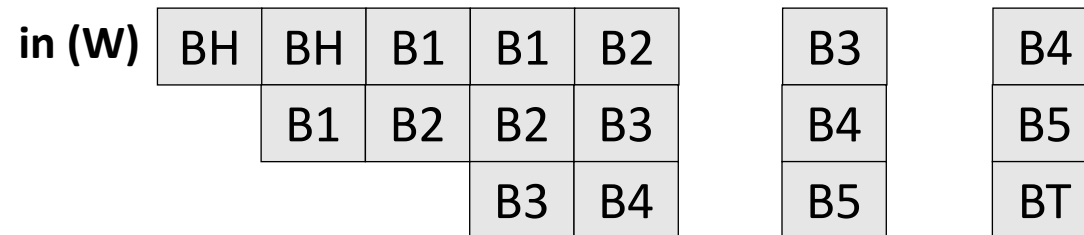
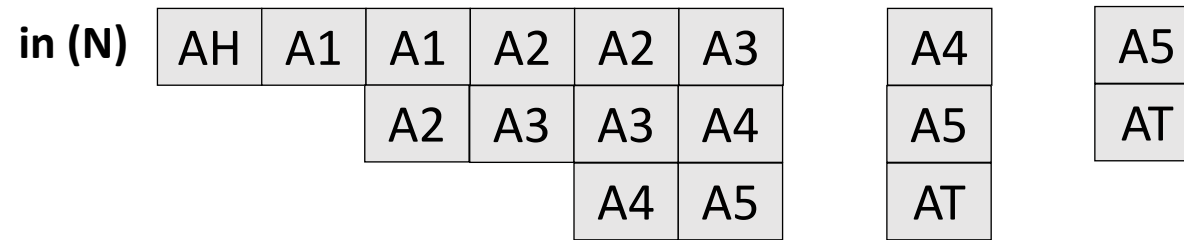


time

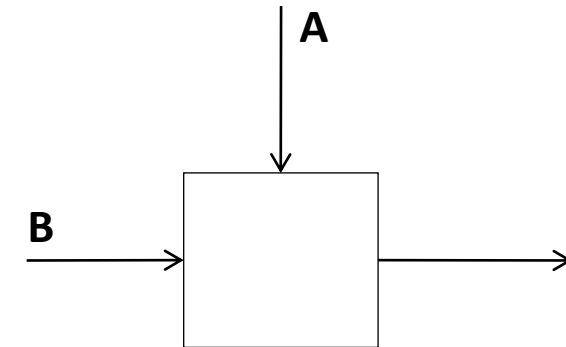


Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs

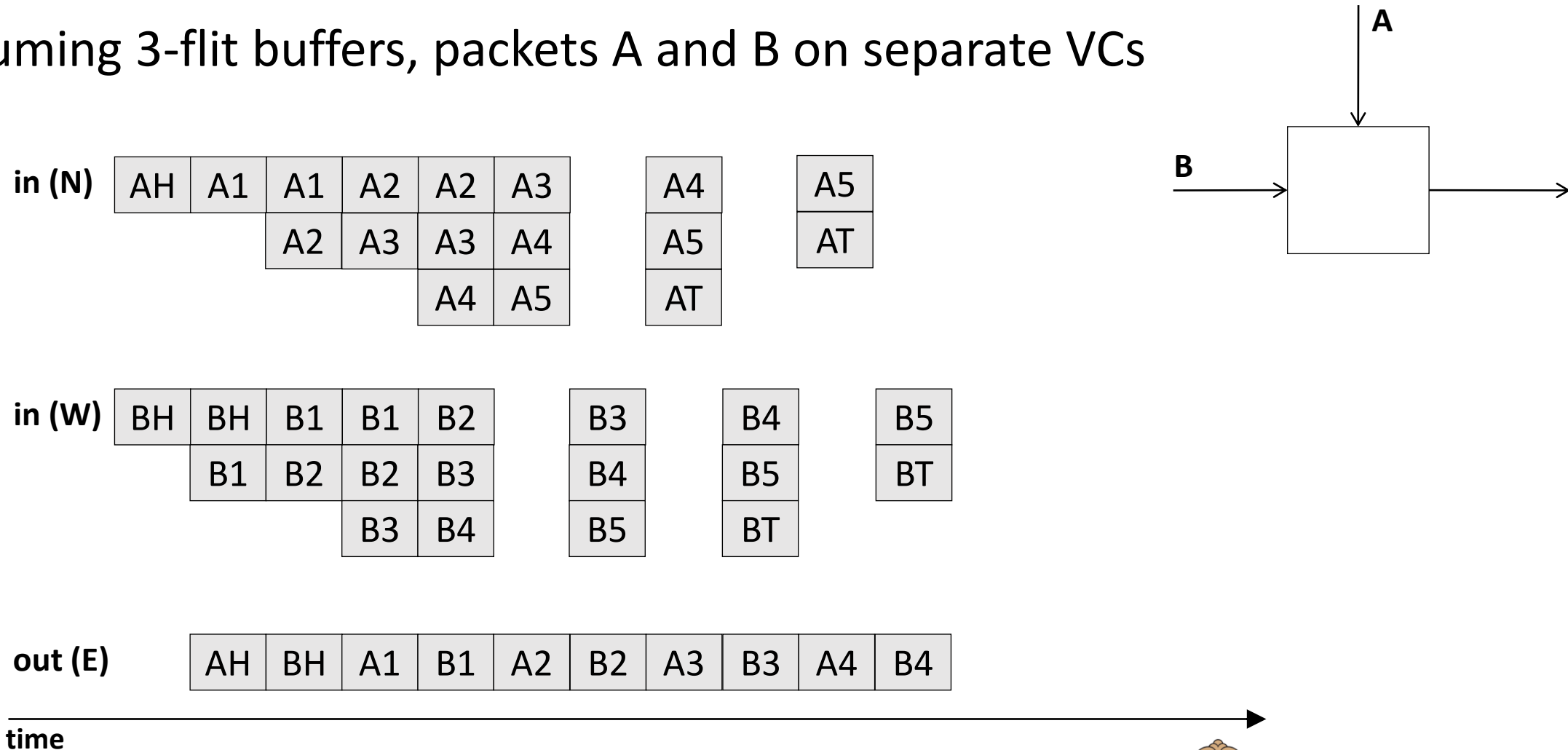


time



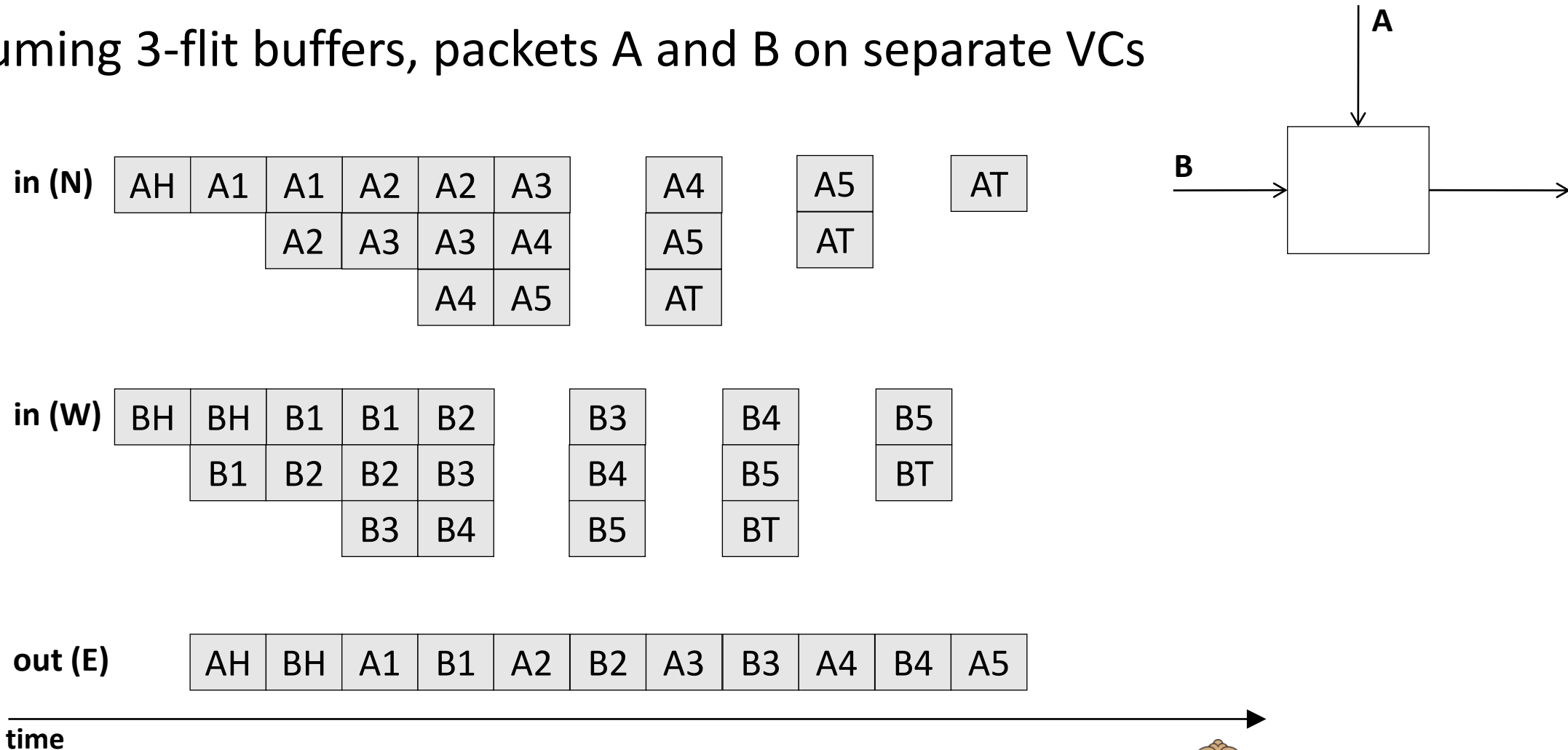
Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs



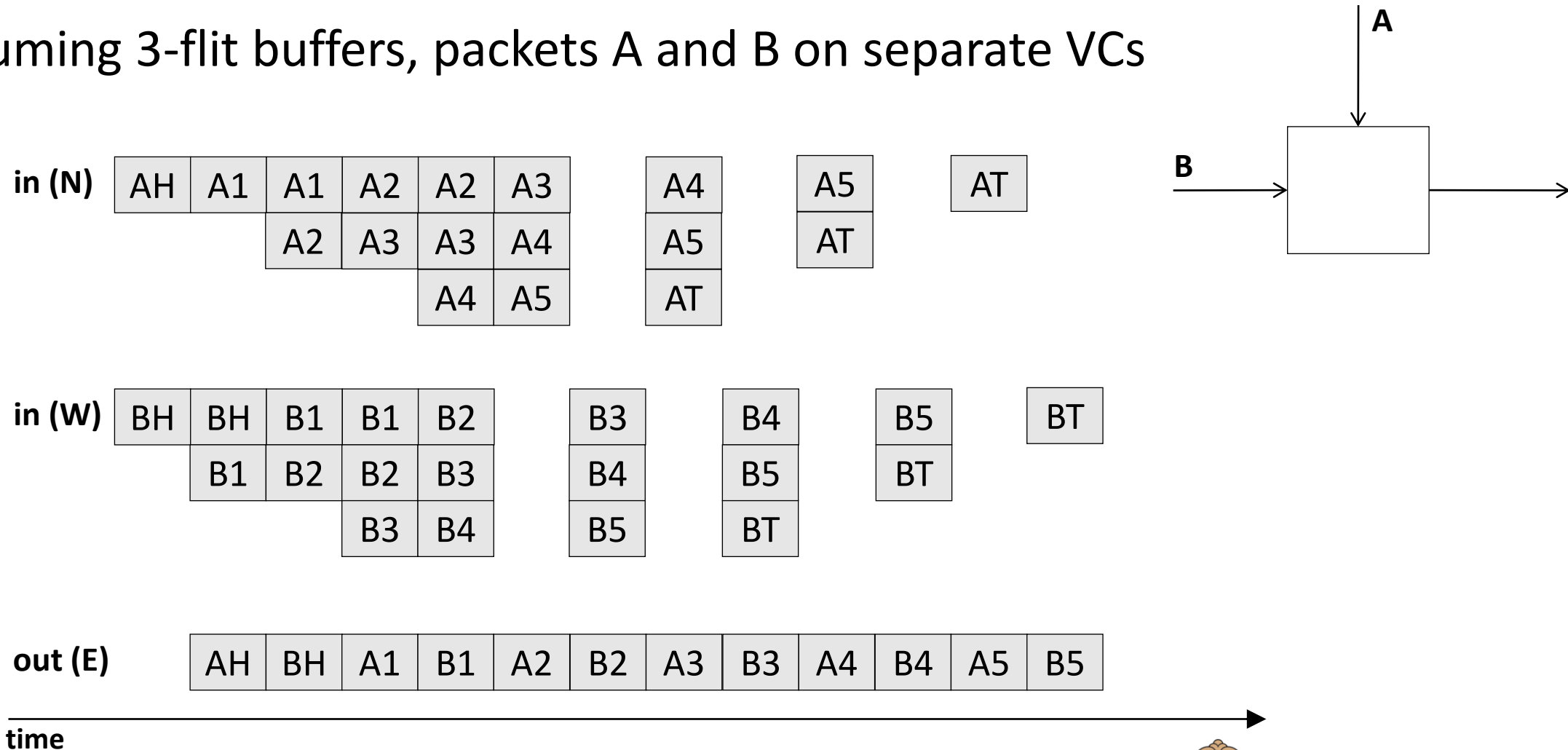
Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs



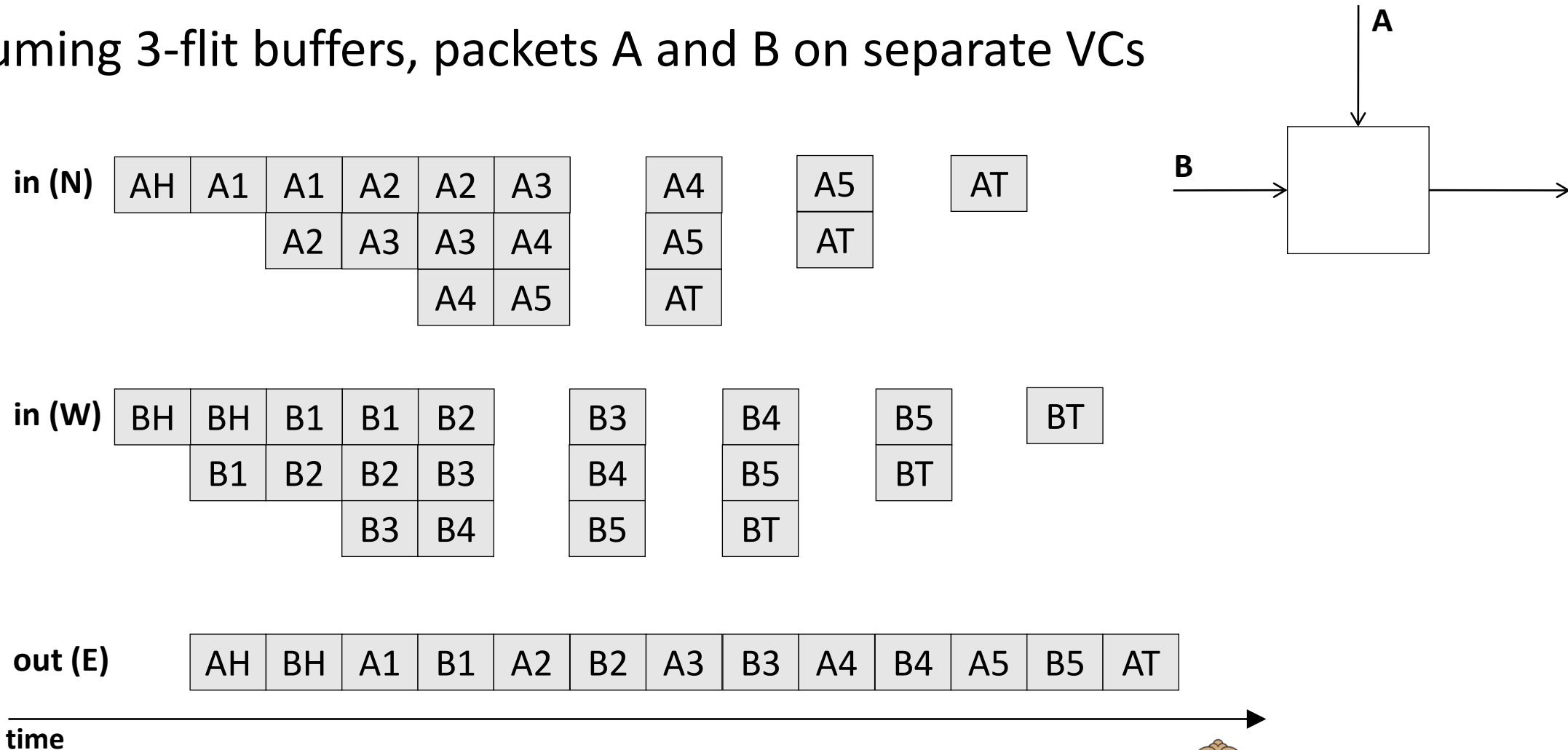
Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs



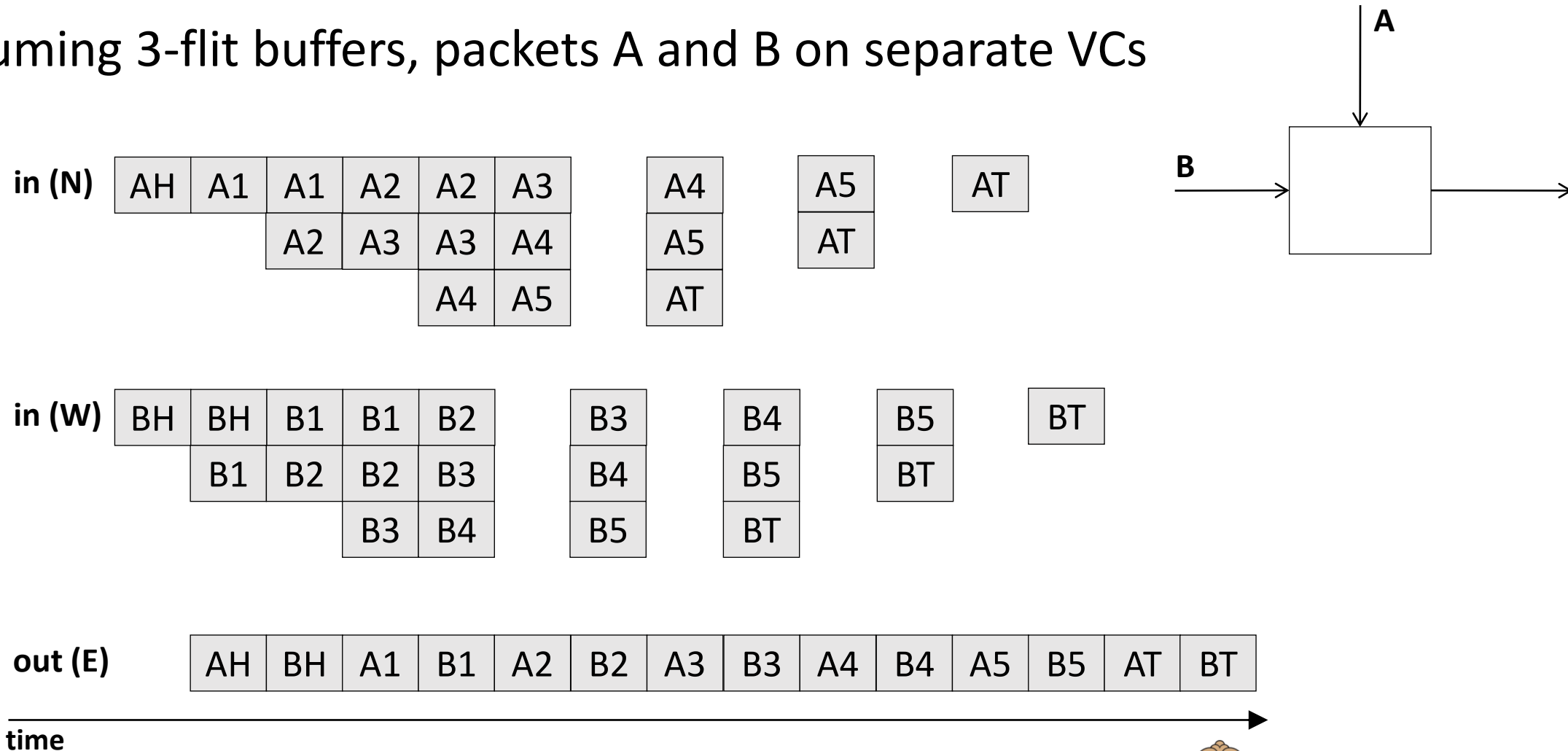
Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs



Virtual Channels

e.g., Assuming 3-flit buffers, packets A and B on separate VCs



Virtual Channels – Deadlocks

Instead of routing restrictions to prevent deadlocks, virtual channels can be used

- VCs allow packets to bypass each other on a physical link
- Can break any resource cycles if routing algorithm is not deadlock-free
- Need to enforce ordering of VCs or use escape VCs

Virtual Channels – Deadlocks

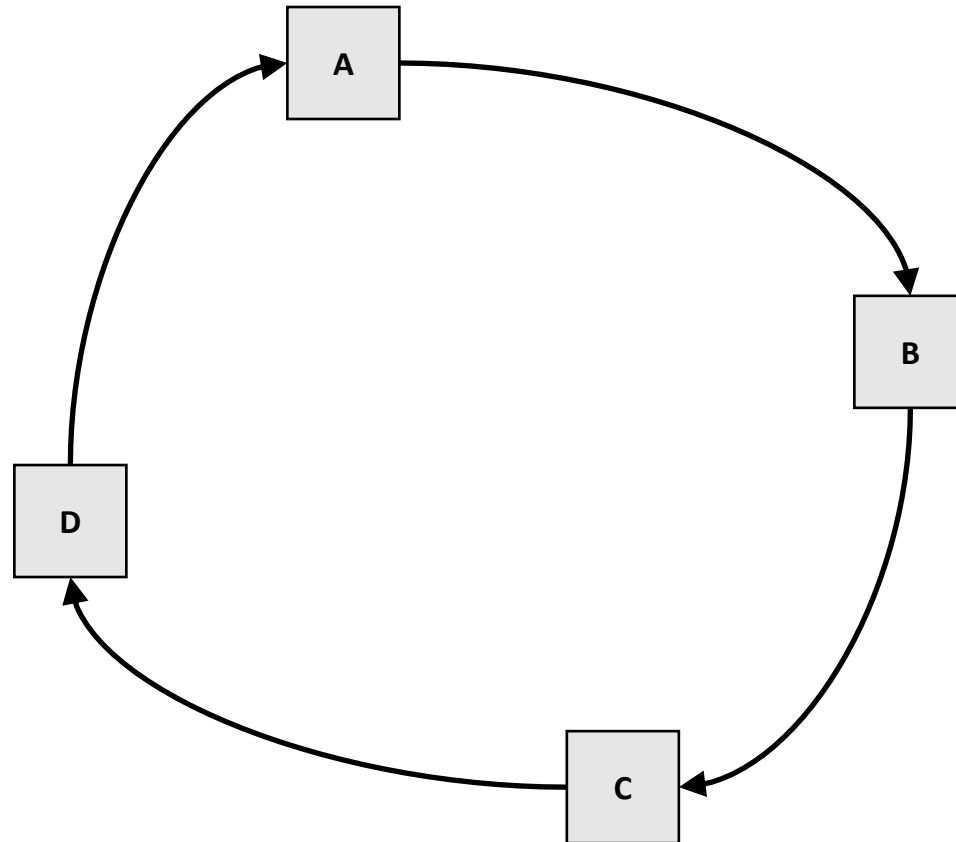
Instead of routing restrictions to prevent deadlocks, virtual channels can be used

- VCs allow packets to bypass each other on a physical link
- Can break any resource cycles if routing algorithm is not deadlock-free
- Need to enforce ordering of VCs or use escape VCs

VCs are used to implement virtual networks for avoiding coherence protocol deadlocks

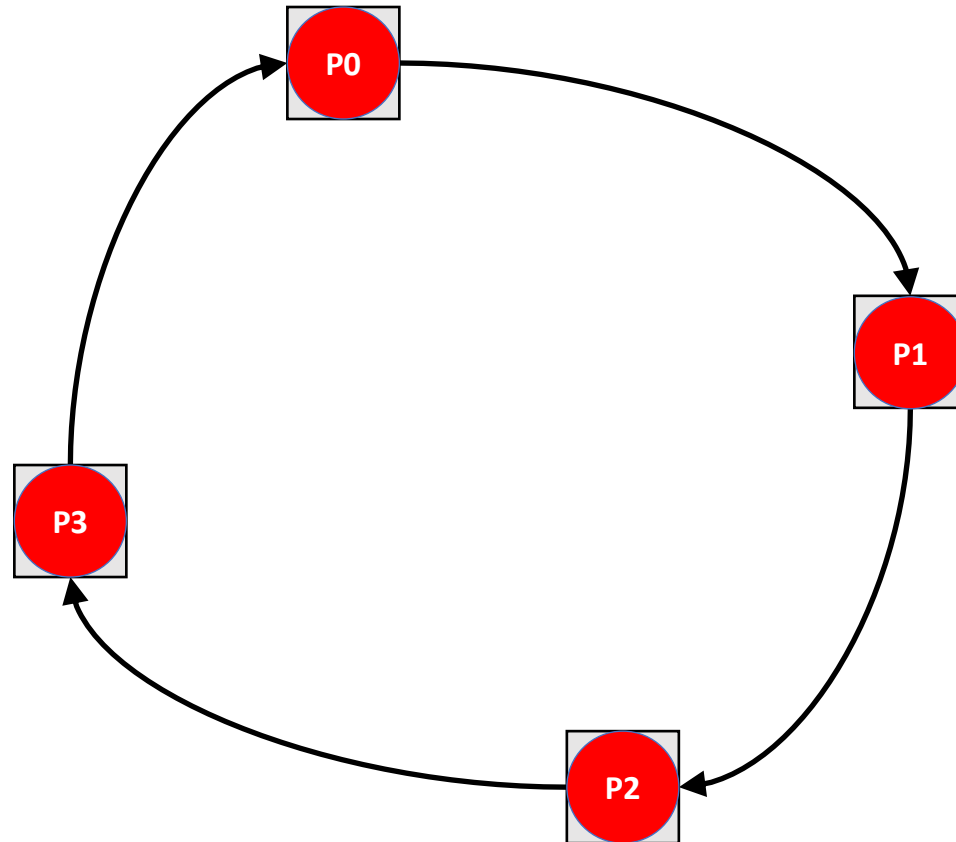
Virtual Channels – Deadlocks

e.g., Deadlock on routers A, B, C, D



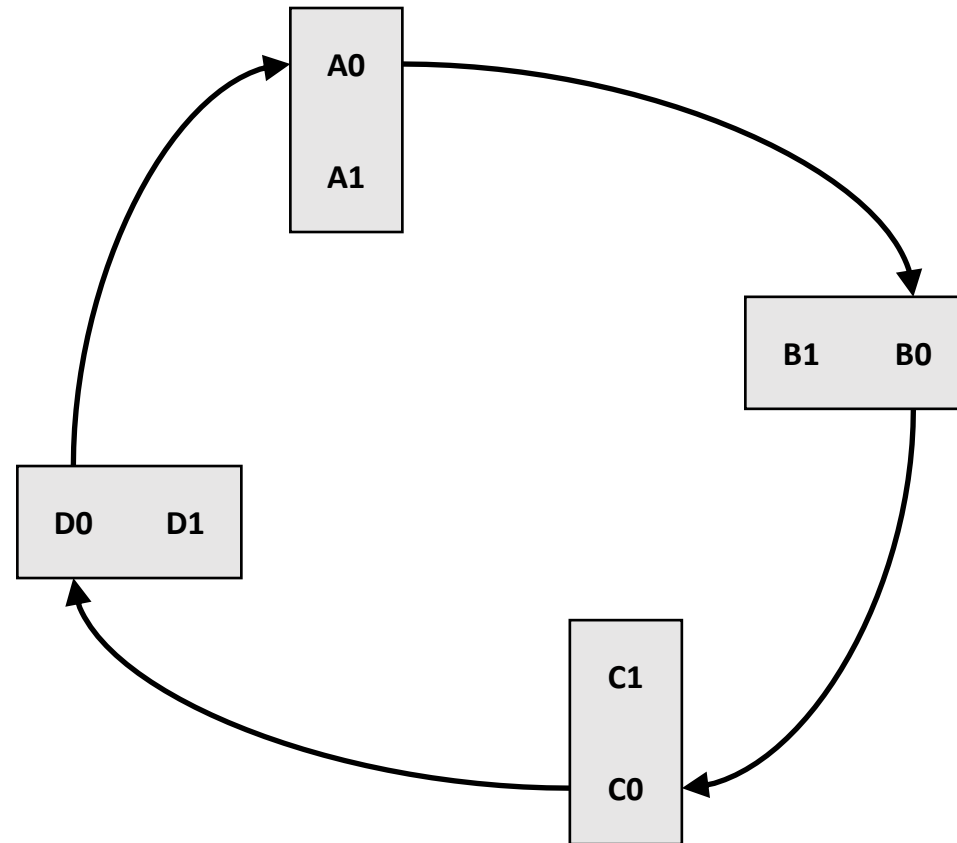
Virtual Channels – Deadlocks

e.g., Deadlock on routers A, B, C, D



Ordered Virtual Channels

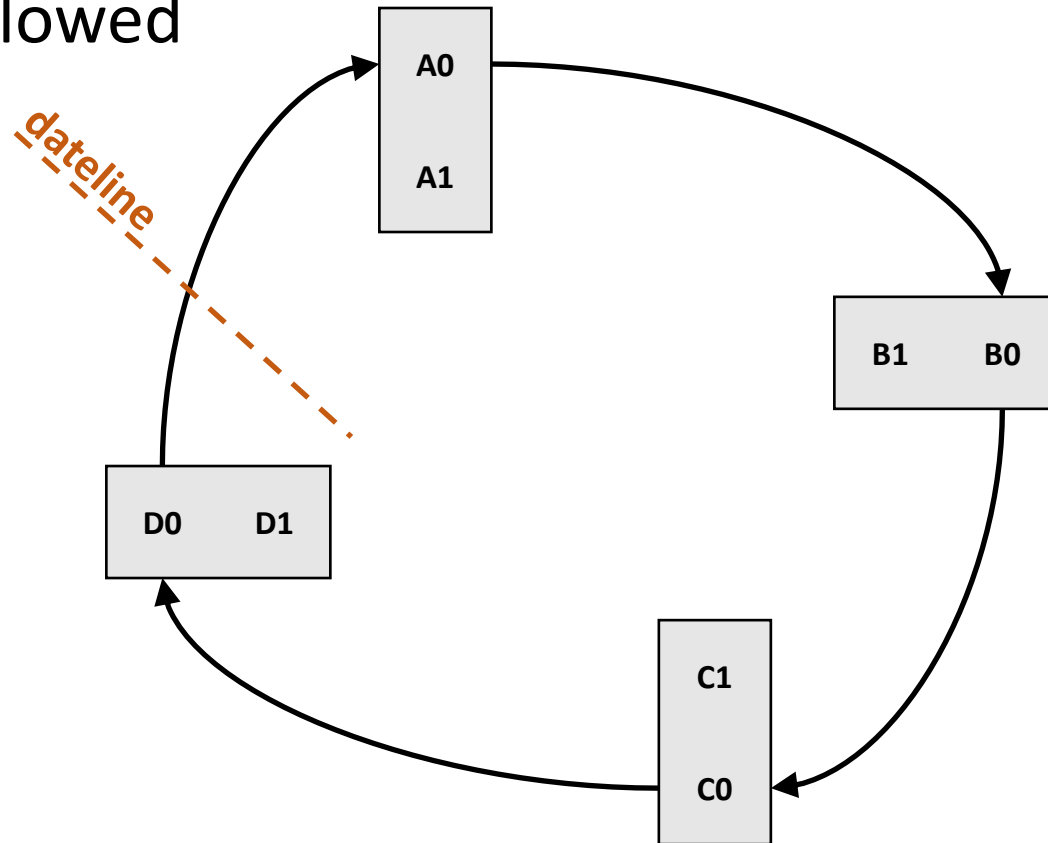
e.g., Deadlock freedom using 2 VCs per router A, B, C, D



Ordered Virtual Channels

e.g., Deadlock freedom using 2 VCs per router A, B, C, D

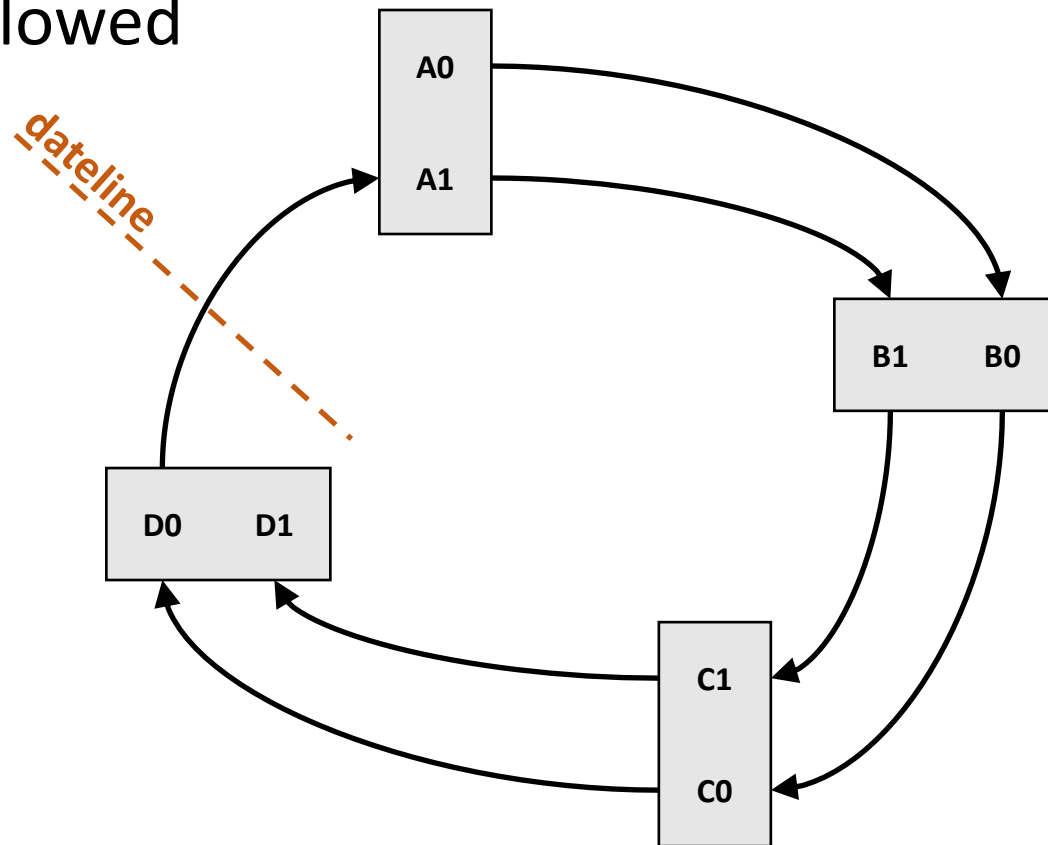
- All packets use VC0 first and must switch to VC1 only upon crossing *dateline*
- VC1 to VC0 not allowed



Ordered Virtual Channels

e.g., Deadlock freedom using 2 VCs per router A, B, C, D

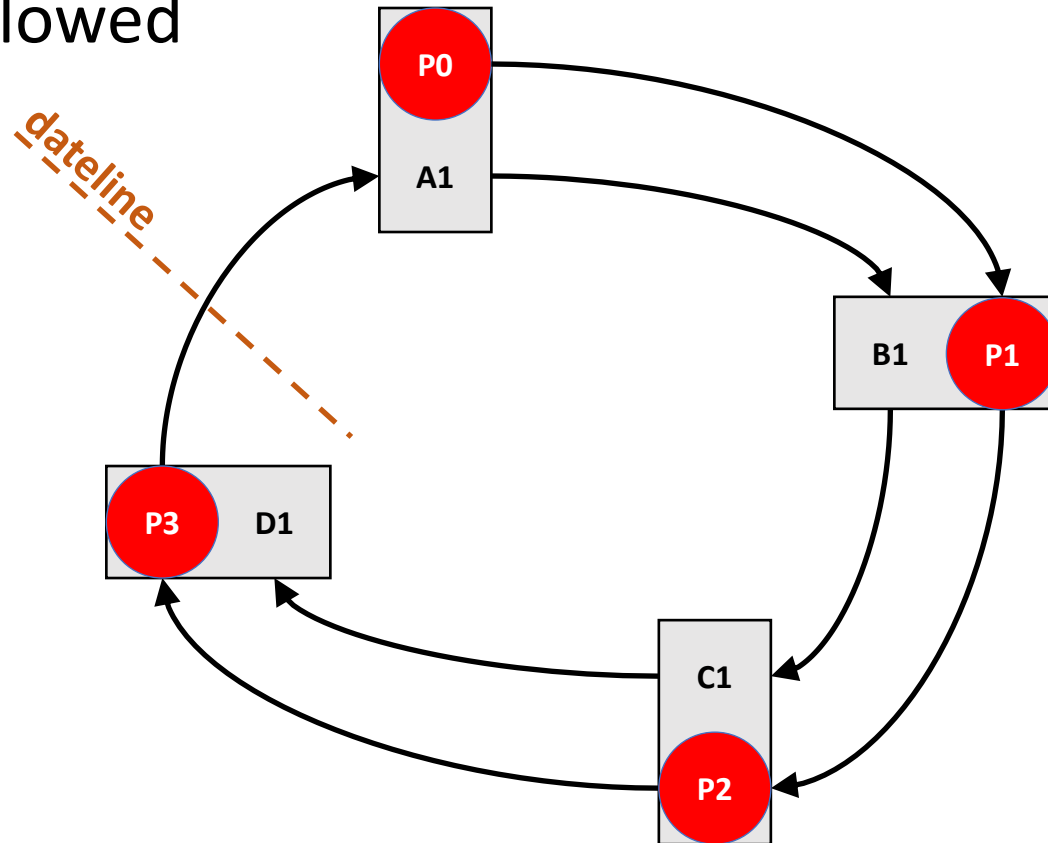
- All packets use VC0 first and must switch to VC1 only upon crossing *dateline*
- VC1 to VC0 not allowed



Ordered Virtual Channels

e.g., Deadlock freedom using 2 VCs per router A, B, C, D

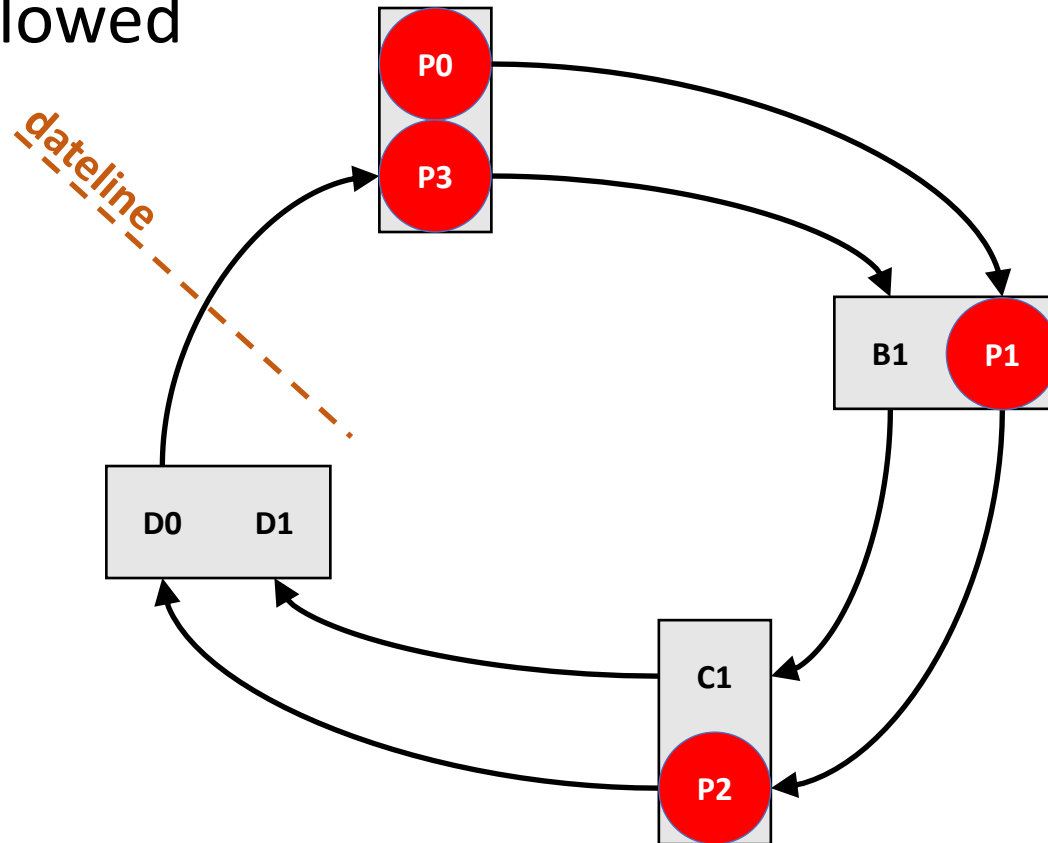
- All packets use VC0 first and must switch to VC1 only upon crossing *dateline*
- VC1 to VC0 not allowed



Ordered Virtual Channels

e.g., Deadlock freedom using 2 VCs per router A, B, C, D

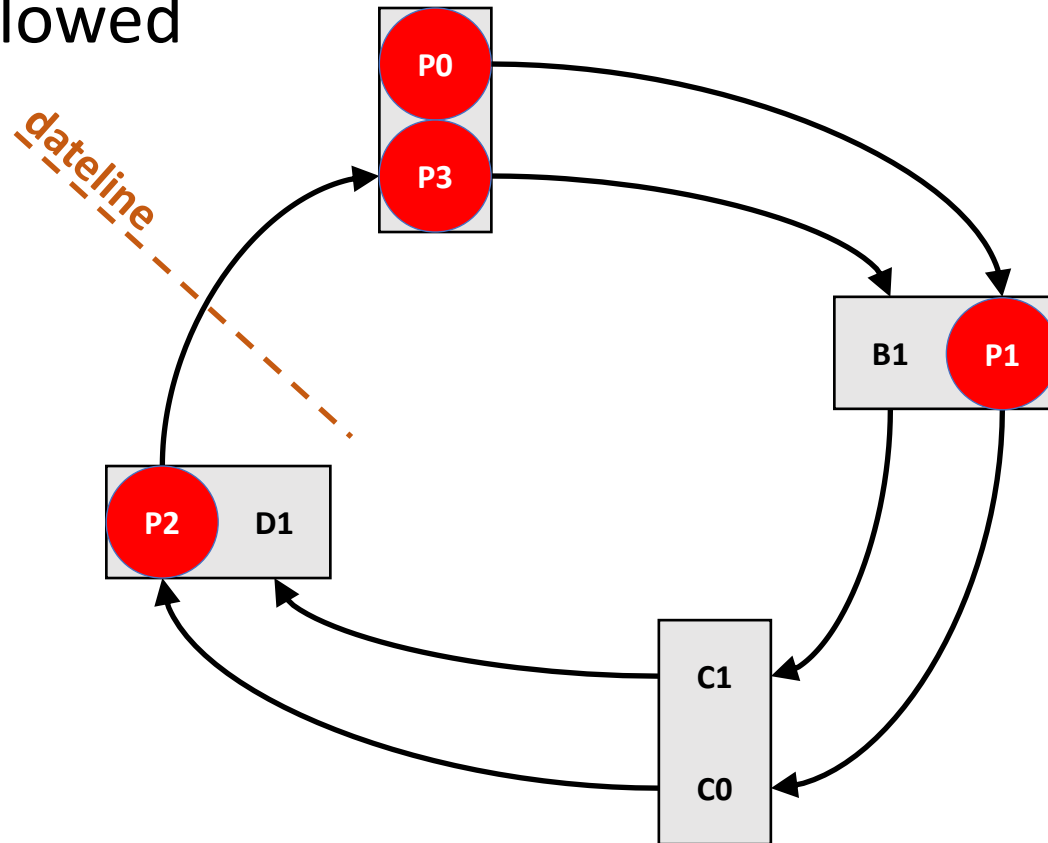
- All packets use VC0 first and must switch to VC1 only upon crossing *dateline*
- VC1 to VC0 not allowed



Ordered Virtual Channels

e.g., Deadlock freedom using 2 VCs per router A, B, C, D

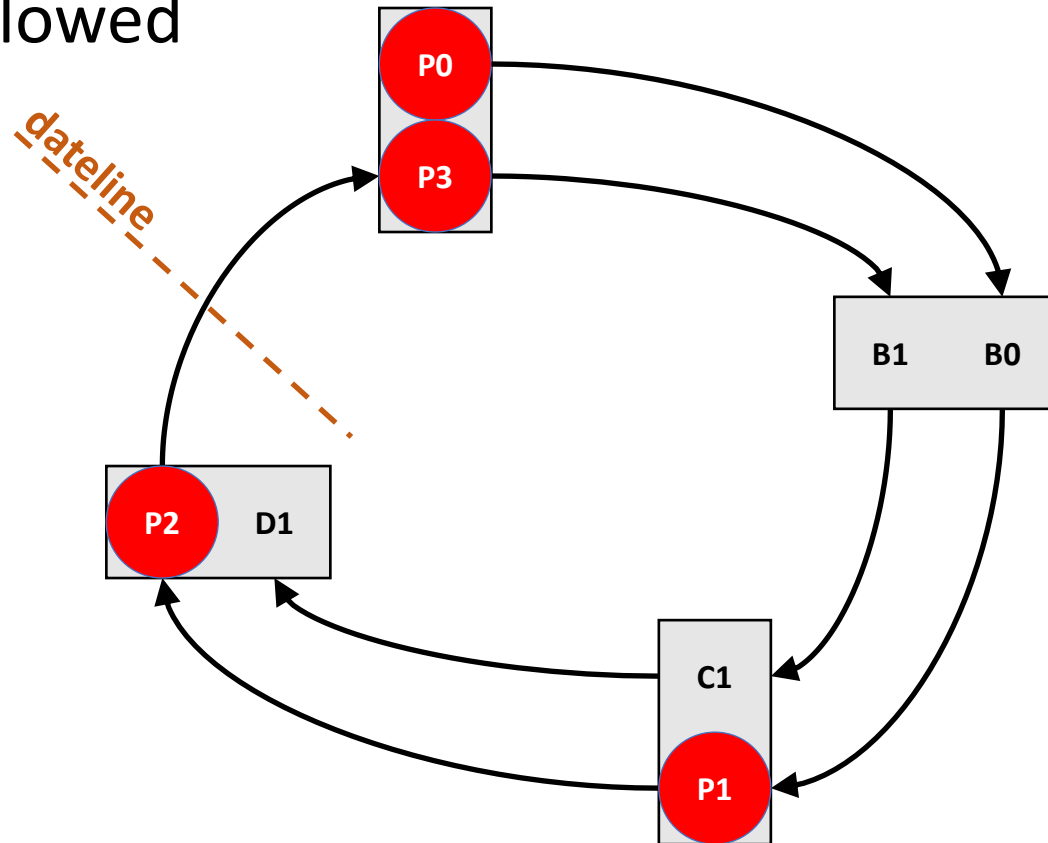
- All packets use VC0 first and must switch to VC1 only upon crossing *dateline*
- VC1 to VC0 not allowed



Ordered Virtual Channels

e.g., Deadlock freedom using 2 VCs per router A, B, C, D

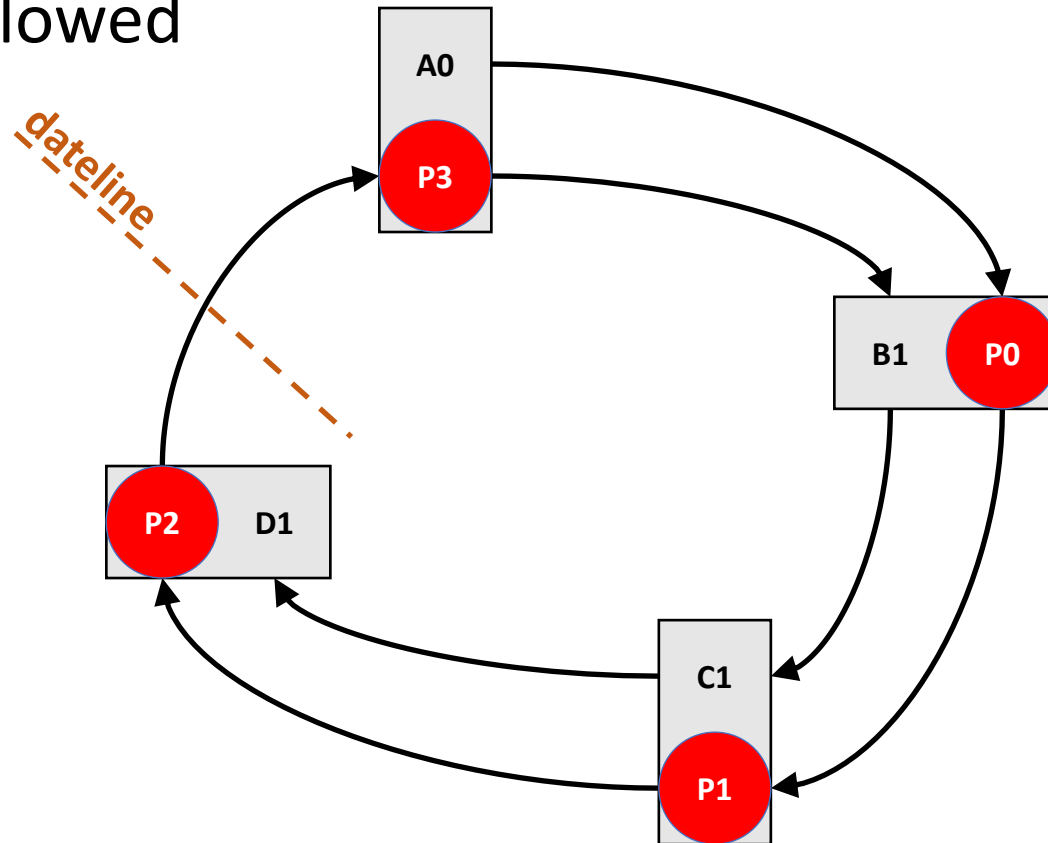
- All packets use VC0 first and must switch to VC1 only upon crossing *dateline*
- VC1 to VC0 not allowed



Ordered Virtual Channels

e.g., Deadlock freedom using 2 VCs per router A, B, C, D

- All packets use VC0 first and must switch to VC1 only upon crossing *dateline*
- VC1 to VC0 not allowed



Escape Virtual Channels

Enforcing VC ordering yields low VC utilization, and can be difficult to determine appropriate datelines for arbitrary topologies and routing algorithms

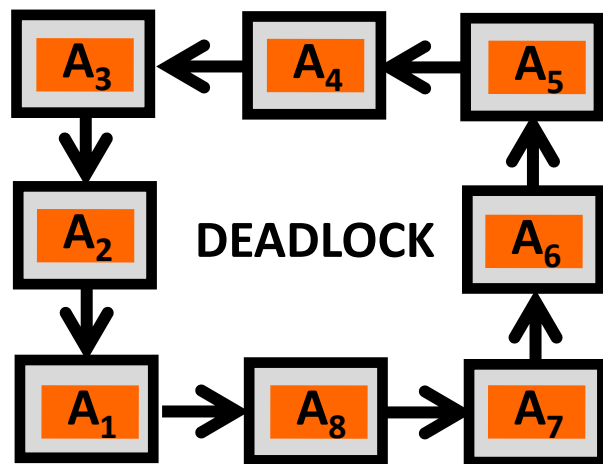
Escape Virtual Channel: A VC that employs deadlock-free routing

- e.g., Escape VC uses dimension order routing while all others use arbitrary routing algorithm
- As long as each packet always has the option to switch to an escape VC, deadlocks are avoided

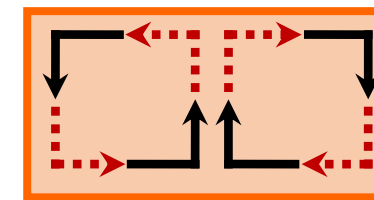
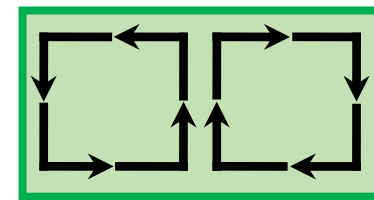
Escape Virtual Channels

Escape Virtual Channel: A VC that employs deadlock-free routing

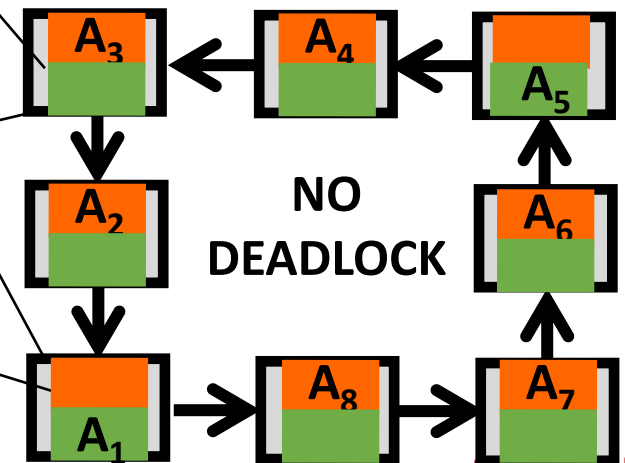
- e.g., Escape VC uses dimension order routing while all others use arbitrary routing algorithm
- As long as each packet always has the option to switch to an escape VC, deadlocks are avoided



All turns allowed in **green** VC



No Y to X turns in **orange** (escape) VC



Deadlock Freedom

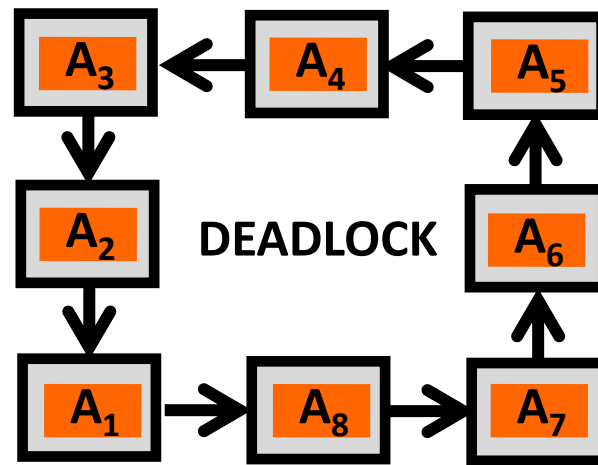
Proactive Solutions:

- Turn model
- Ordered VCs
- Escape VCs
- Critical bubble

Deadlock Freedom – Proactive

Add or restrict resources to prevent deadlocks from ever occurring

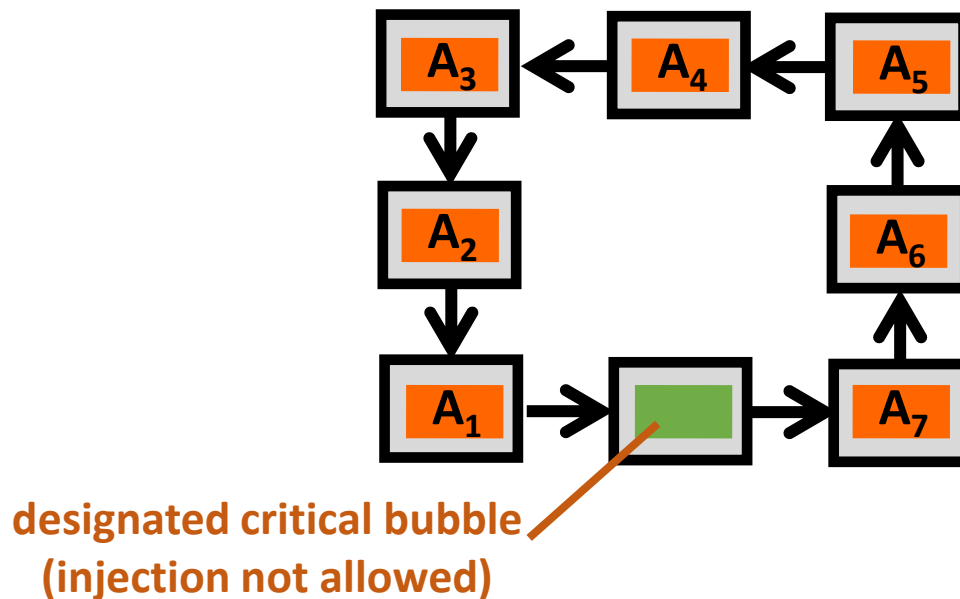
- e.g., Turn model, ordered VCs, escape VCs
- e.g., **Critical Bubble [IPDPS 2011]**: Maintain a *bubble* packet/flit in each ring of the topology that can only be displaced by flits already in network



Deadlock Freedom – Proactive

Add or restrict resources to prevent deadlocks from ever occurring

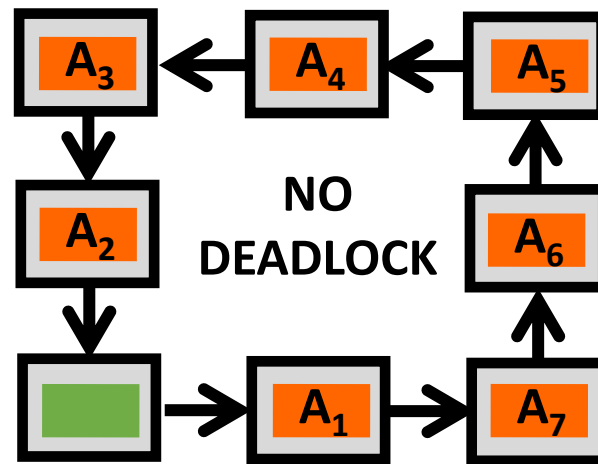
- e.g., Turn model, ordered VCs, escape VCs
- e.g., **Critical Bubble [IPDPS 2011]**: Maintain a *bubble* packet/flit in each ring of the topology that can only be displaced by flits already in network



Deadlock Freedom – Proactive

Add or restrict resources to prevent deadlocks from ever occurring

- e.g., Turn model, ordered VCs, escape VCs
- e.g., **Critical Bubble [IPDPS 2011]**: Maintain a *bubble* packet/flit in each ring of the topology that can only be displaced by flits already in network



Deadlock Freedom

Proactive Solutions:

- Turn model
- Ordered VCs
- Escape VCs
- Critical bubble

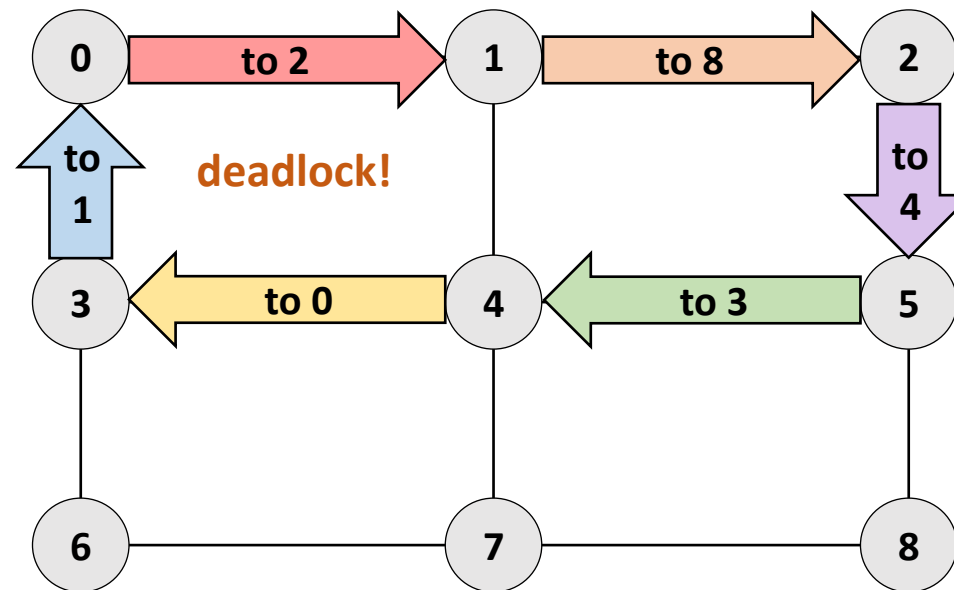
Reactive Solutions:

- SPIN

Deadlock Freedom – Reactive

Dynamically detect and resolve deadlocks on-the-fly

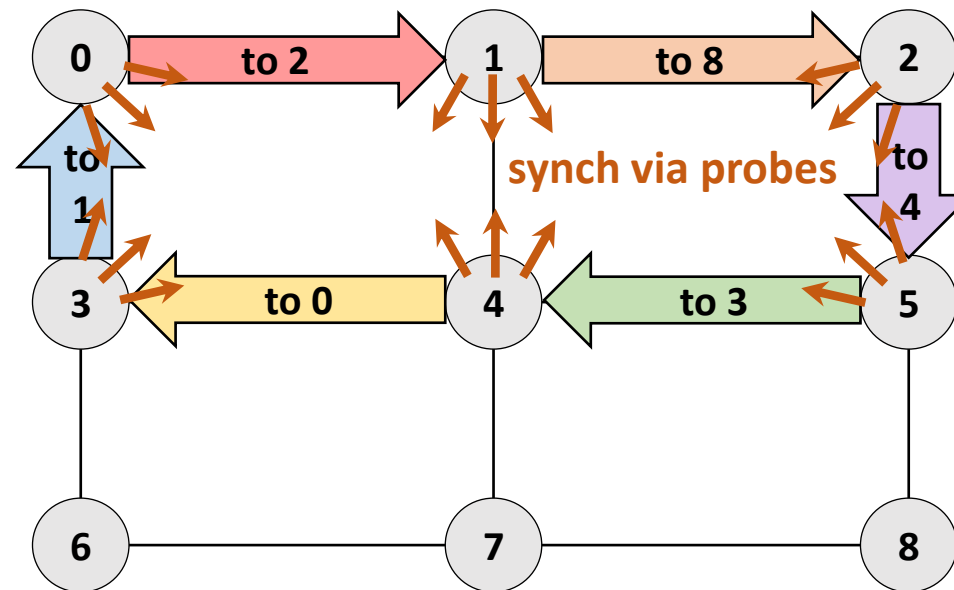
- e.g., **SPIN [ISCA 2018]**: Use probes to synchronize a *spin* (i.e., move forward in a pipelined manner) of all packets caught in the deadlock cycle
 - Can support arbitrary routing algorithm and VC organization



Deadlock Freedom – Reactive

Dynamically detect and resolve deadlocks on-the-fly

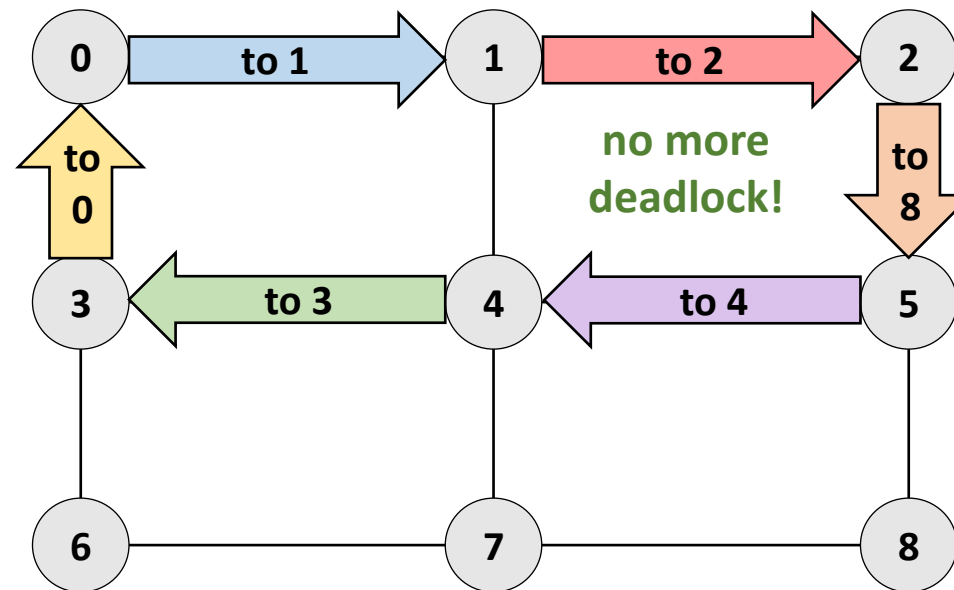
- e.g., **SPIN [ISCA 2018]**: Use probes to synchronize a *spin* (i.e., move forward in a pipelined manner) of all packets caught in the deadlock cycle
 - Can support arbitrary routing algorithm and VC organization



Deadlock Freedom – Reactive

Dynamically detect and resolve deadlocks on-the-fly

- e.g., **SPIN [ISCA 2018]**: Use probes to synchronize a *spin* (i.e., move forward in a pipelined manner) of all packets caught in the deadlock cycle
 - Can support arbitrary routing algorithm and VC organization



Deadlock Freedom

Proactive Solutions:

- Turn model
- Ordered VCs
- Escape VCs
- Critical bubble

Reactive Solutions:

- SPIN

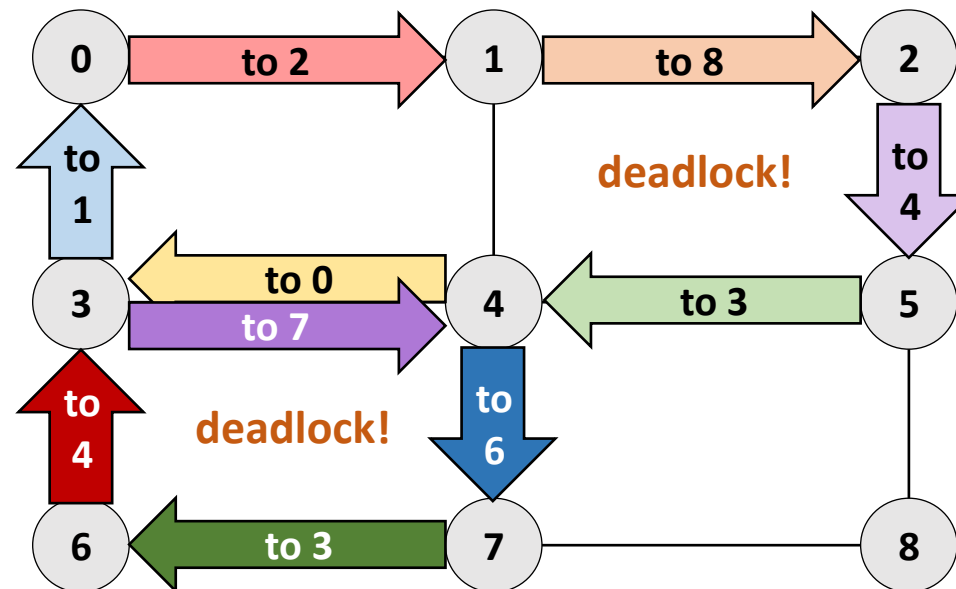
Subactive Solutions:

- DRAIN

Deadlock Freedom – Subactive

Let deadlocks happen and obviously take maintenance actions to clear deadlocks from the network

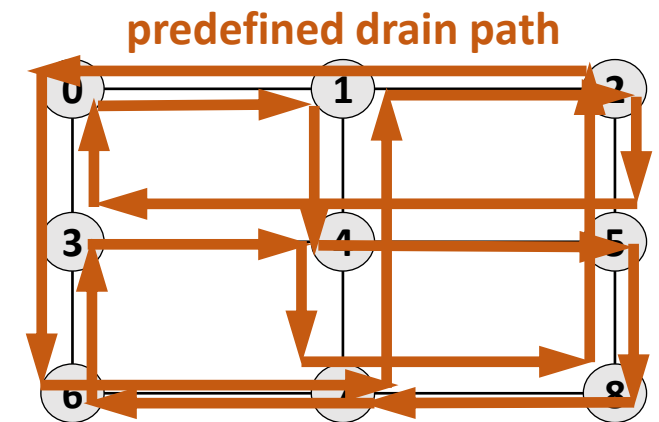
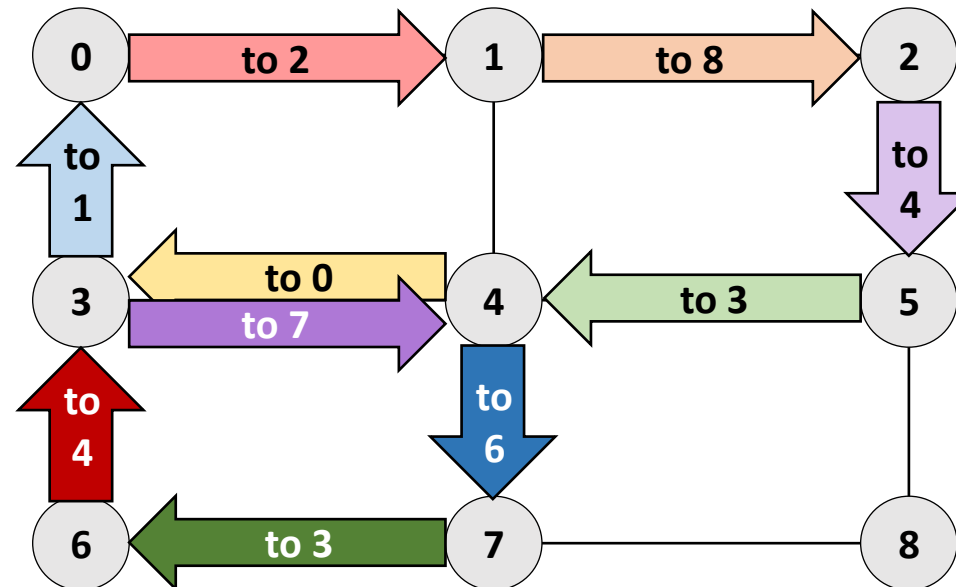
- e.g., **DRAIN [HPCA 2020]**: Periodically *drain* (i.e., force the movement of) all packets based on a predefined cyclic path across all links in the network
 - Can resolve both routing- and protocol-level deadlocks without virtual networks



Deadlock Freedom – Subactive

Let deadlocks happen and obviously take maintenance actions to clear deadlocks from the network

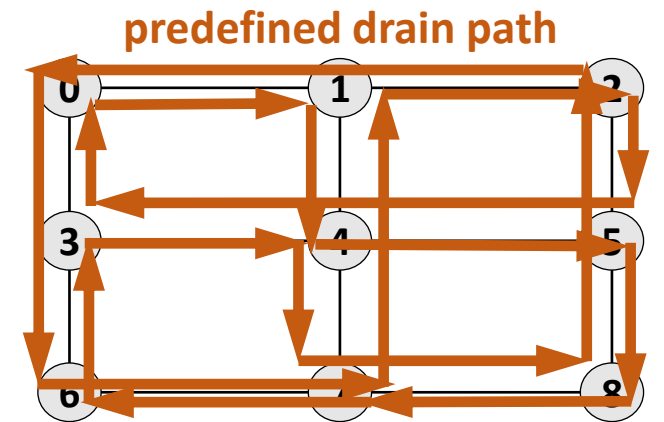
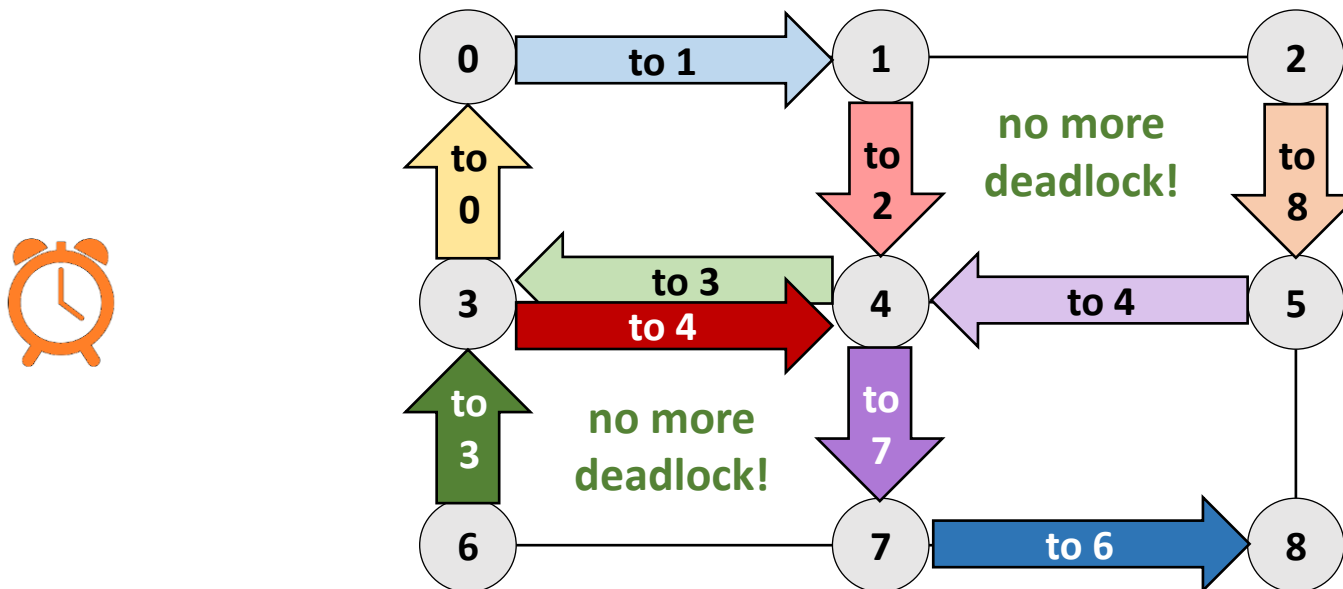
- e.g., **DRAIN [HPCA 2020]**: Periodically *drain* (i.e., force the movement of) all packets based on a predefined cyclic path across all links in the network
 - Can resolve both routing- and protocol-level deadlocks without virtual networks



Deadlock Freedom – Subactive

Let deadlocks happen and obviously take maintenance actions to clear deadlocks from the network

- e.g., **DRAIN [HPCA 2020]**: Periodically *drain* (i.e., force the movement of) all packets based on a predefined cyclic path across all links in the network
 - Can resolve both routing- and protocol-level deadlocks without virtual networks



Buffer Backpressure

Need mechanism to prevent overflow of router buffers

- Cannot drop packets; NoCs are lossless
- Upstream routers need to know buffer availability at downstream routers
- Must be conservative; can't overestimate available buffer space
- Significantly impacts network throughput

Buffer Backpressure – Credit-Based Flow Control

Upstream router keeps track of credit counts from downstream router, which indicates available buffer space

Upstream router:

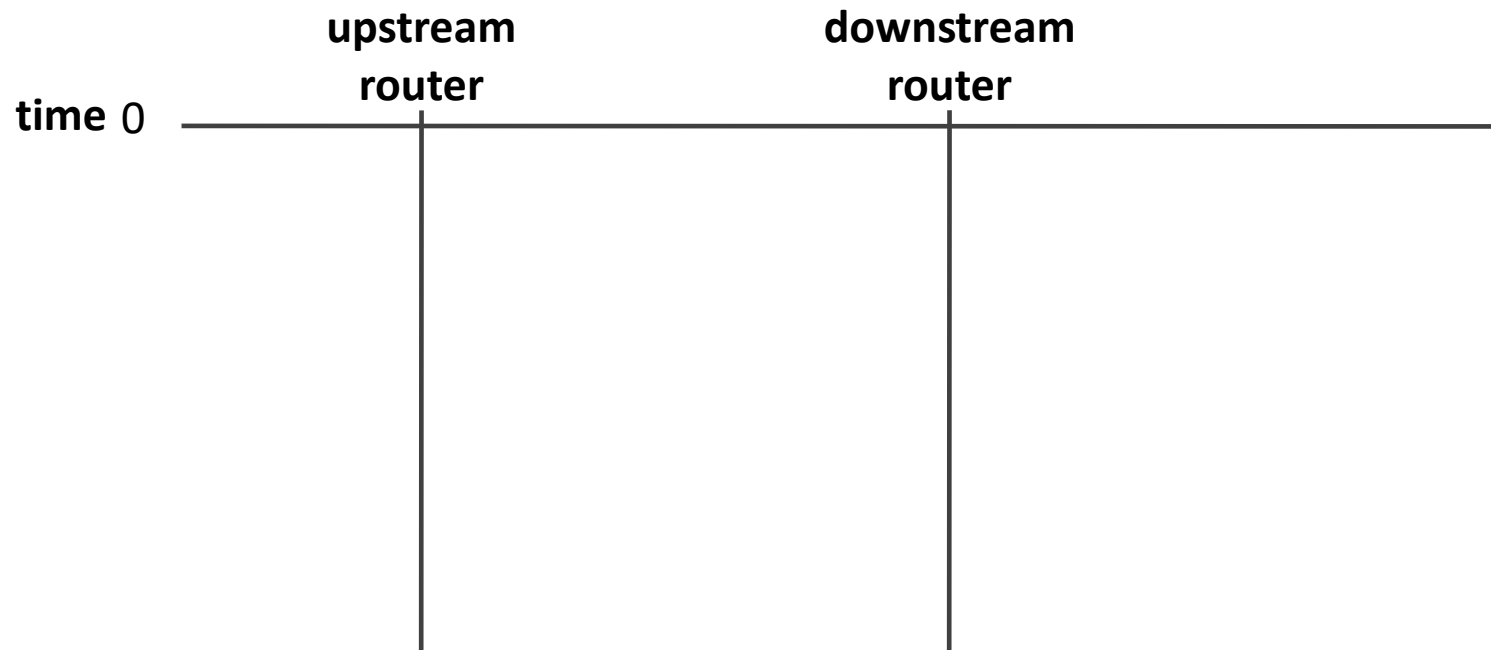
- When flit forwarded, decrement credit count
- If credit count 0, downstream buffer is full; stop sending flits

Downstream router:

- When flit forwarded and buffer freed, send credit to upstream router

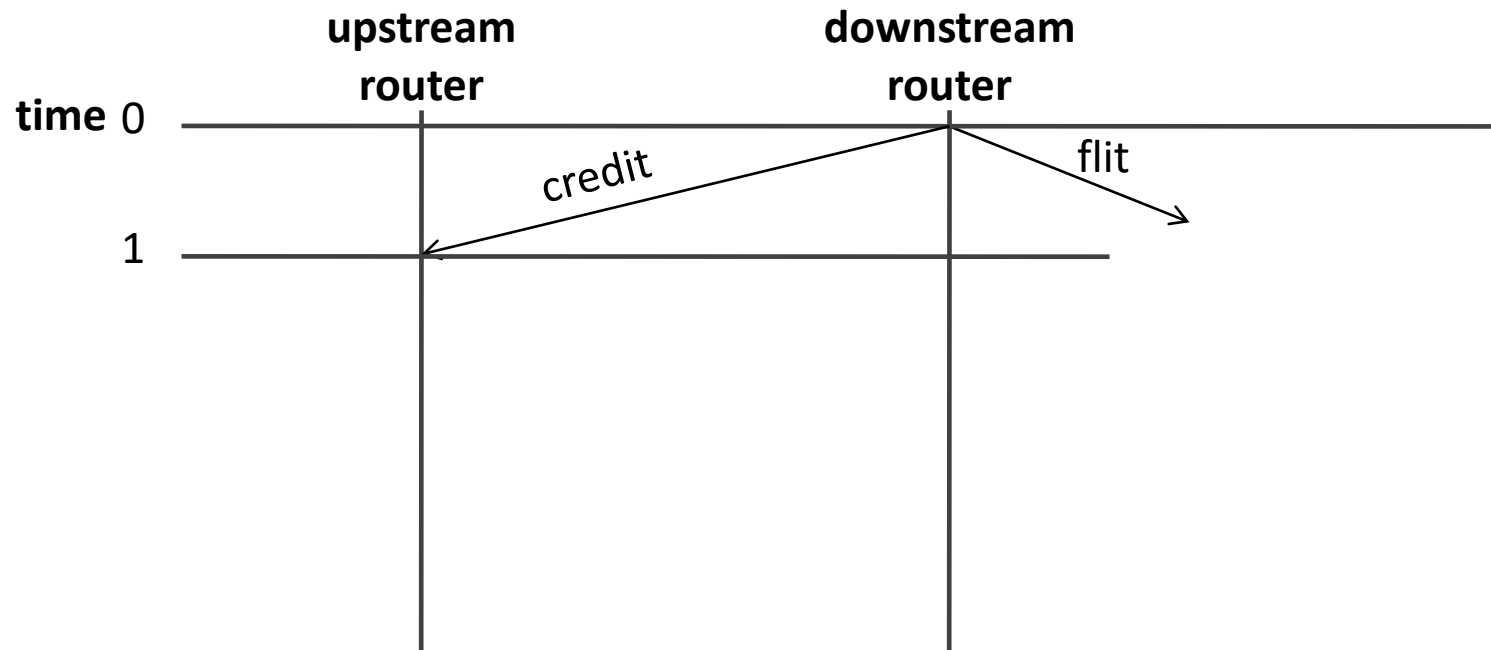
Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



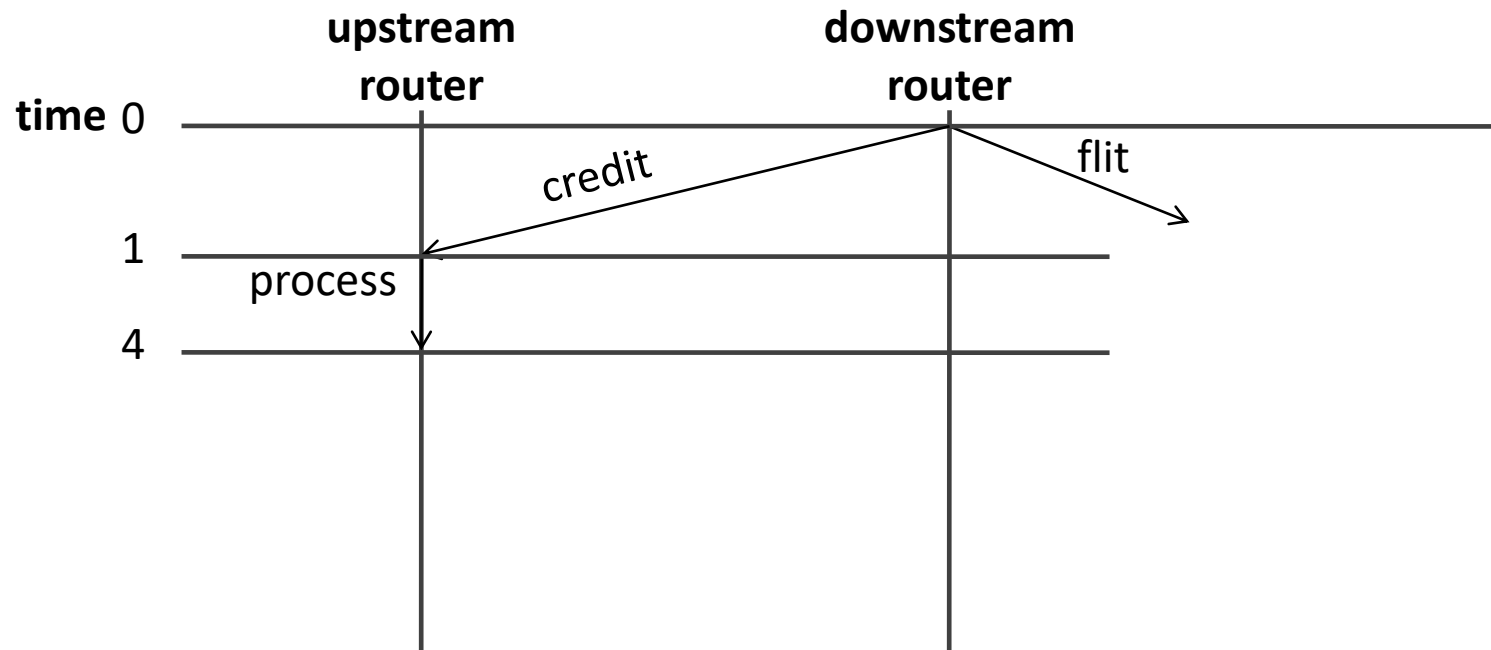
Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



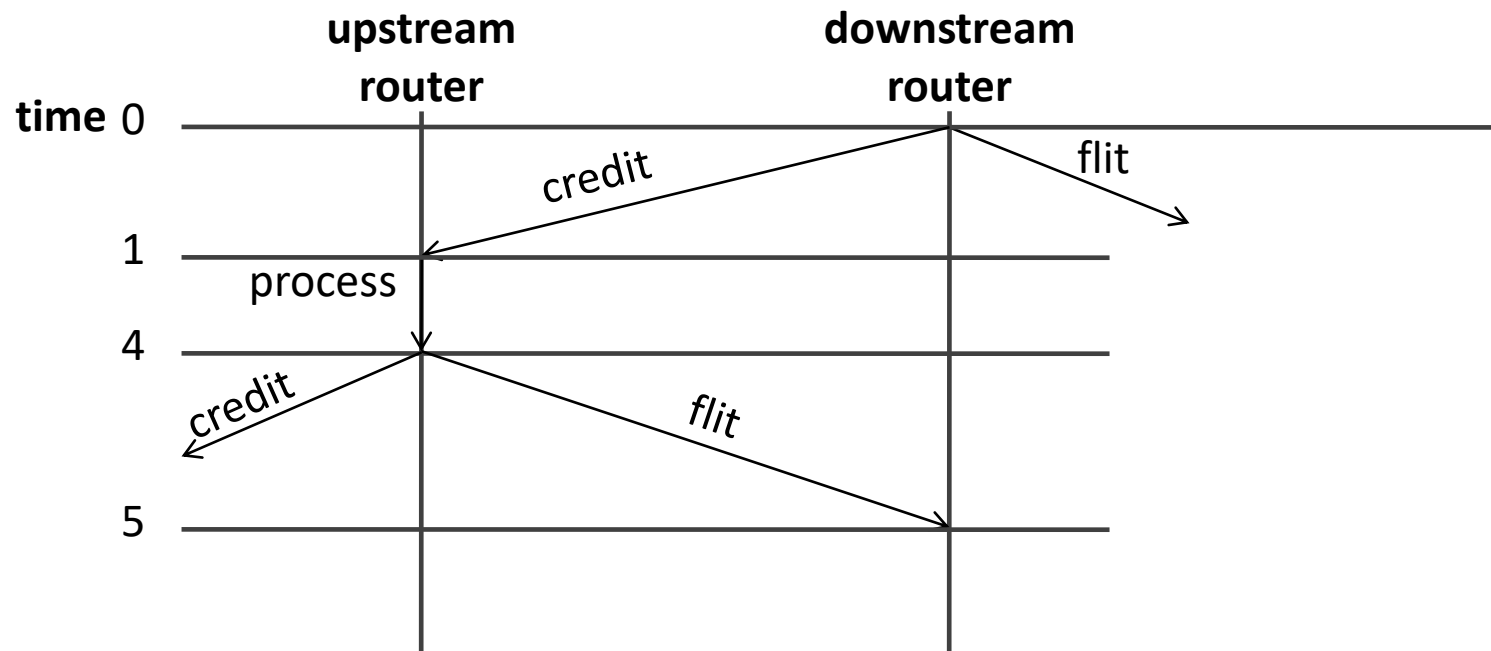
Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



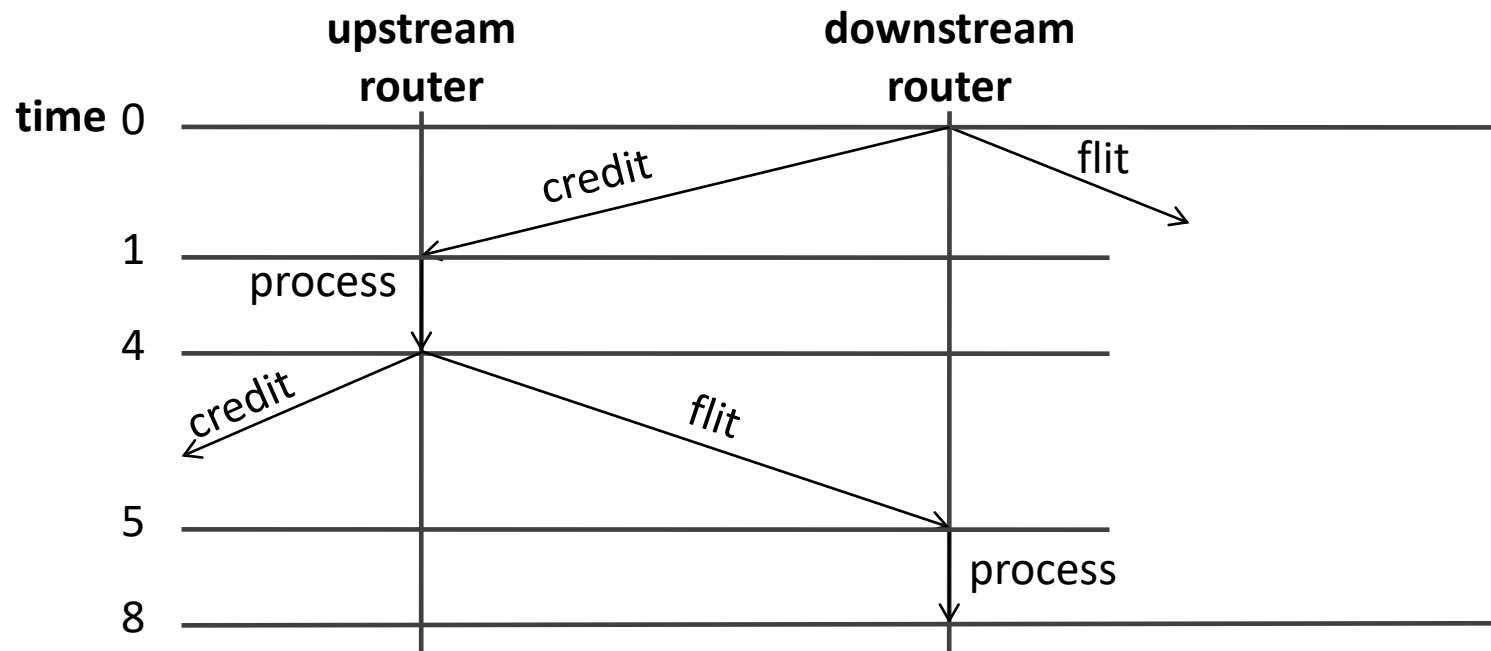
Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



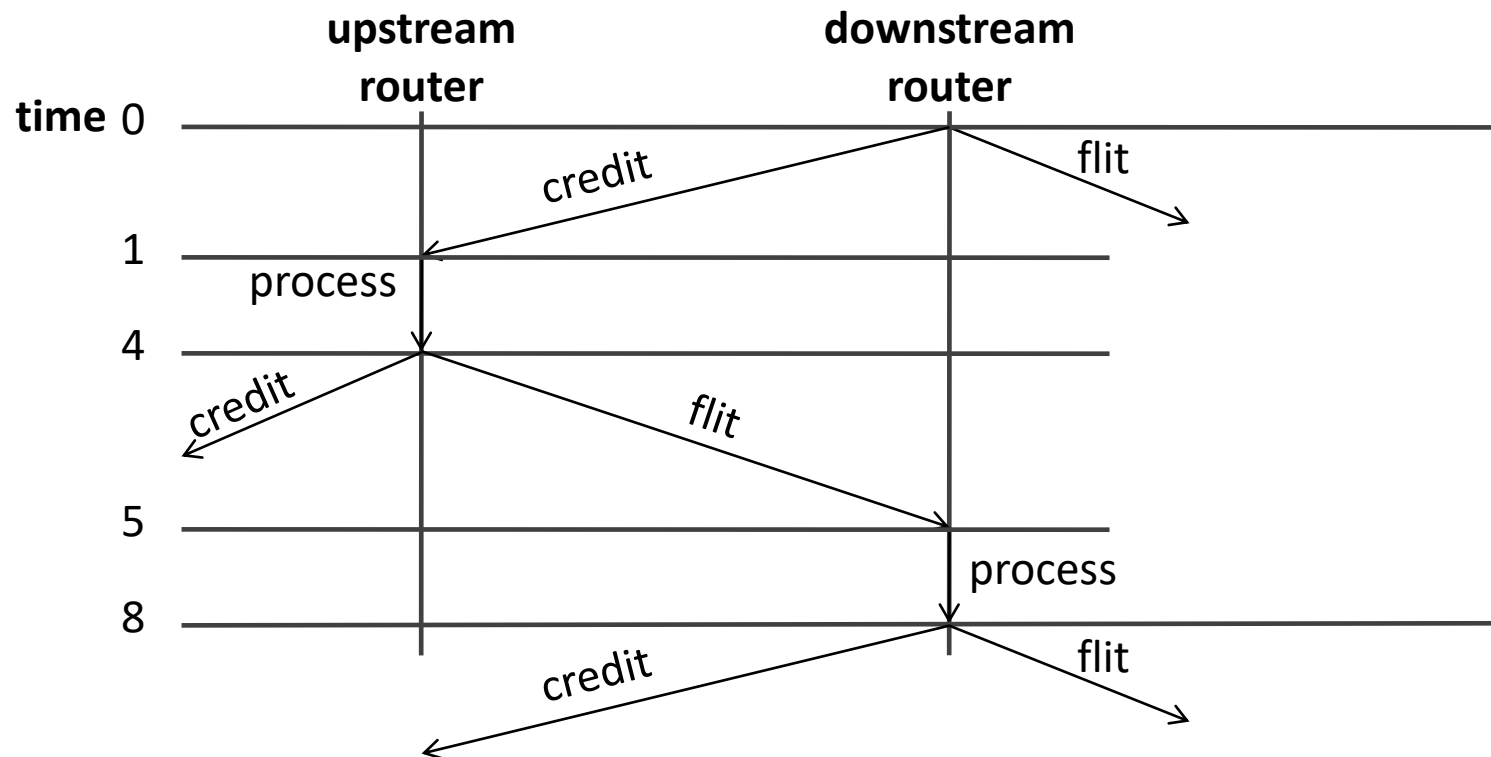
Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



Buffer Backpressure – Credit-Based Flow Control

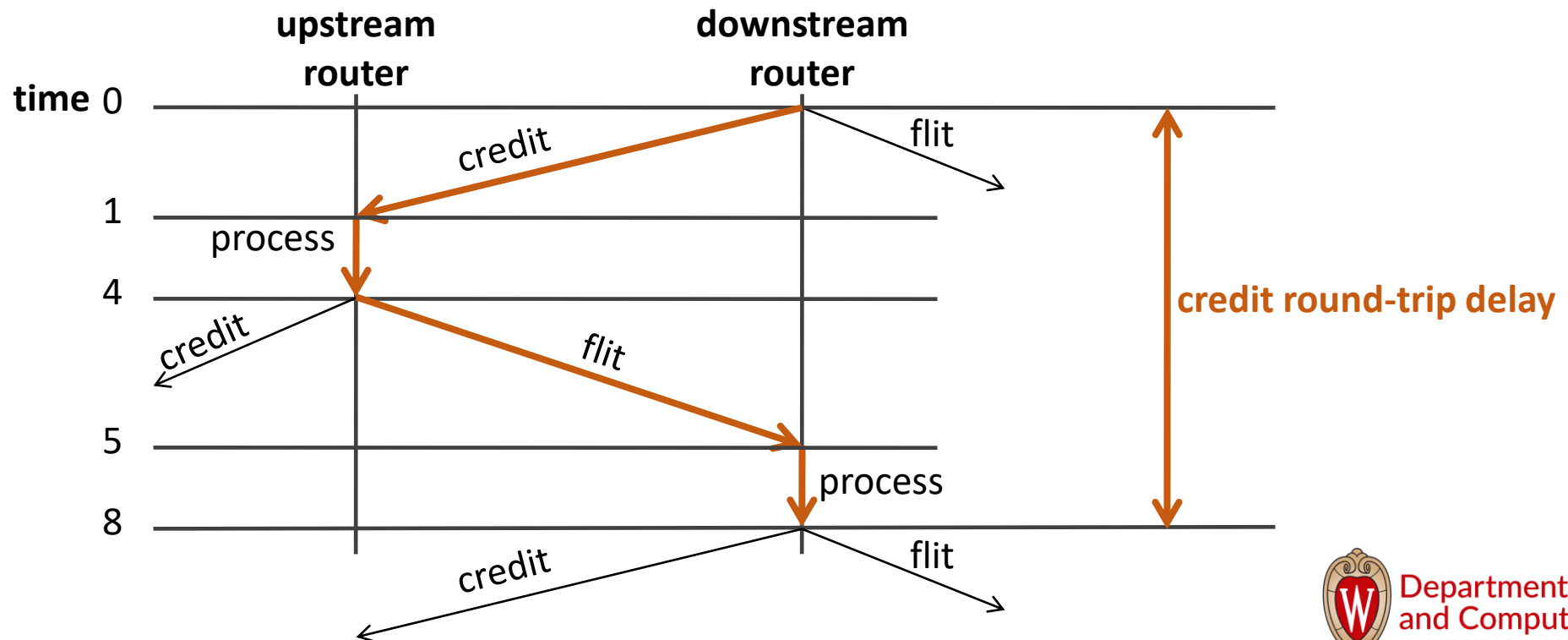
e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)



Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)

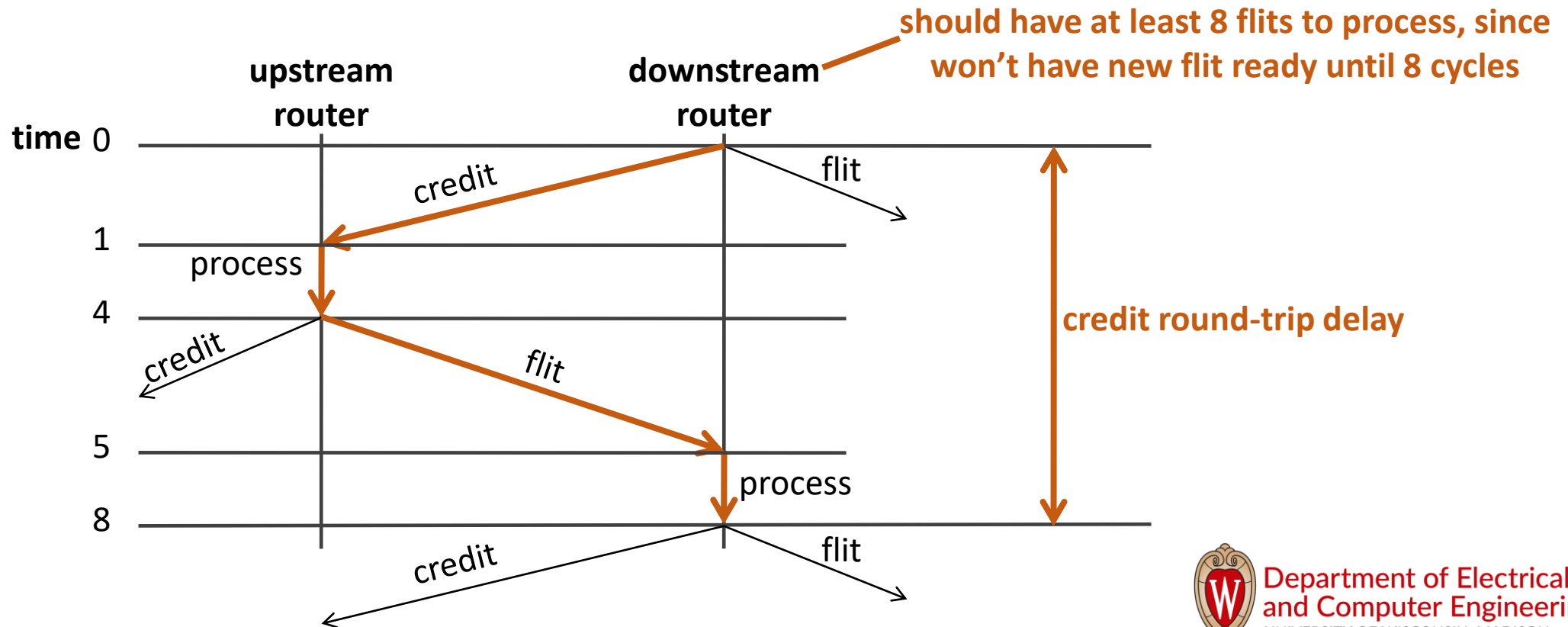
- **Credit round-trip delay:** time between credits being sent upstream for a given buffer slot
- Avoid idle cycles; buffer size should match credit round-trip delay



Buffer Backpressure – Credit-Based Flow Control

e.g., Assume 3-cycle flit processing pipeline and 1-cycle link delay (data/credits)

- **Credit round-trip delay:** time between credits being sent upstream for a given buffer slot
- Avoid idle cycles; buffer size should match credit round-trip delay (e.g., 8 flits per buffer)



Buffer Backpressure – On-Off Flow Control

Reduce upstream signaling; unlike credit-based flow control which requires a credit sent upstream for every flit

Off signal:

- Sent when number of free buffer slots falls below some threshold F_{off}

On signal:

- Sent when number of free buffer slots rises above some threshold F_{on}

Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

- Set thresholds to trade off between amount of signaling and buffer utilization

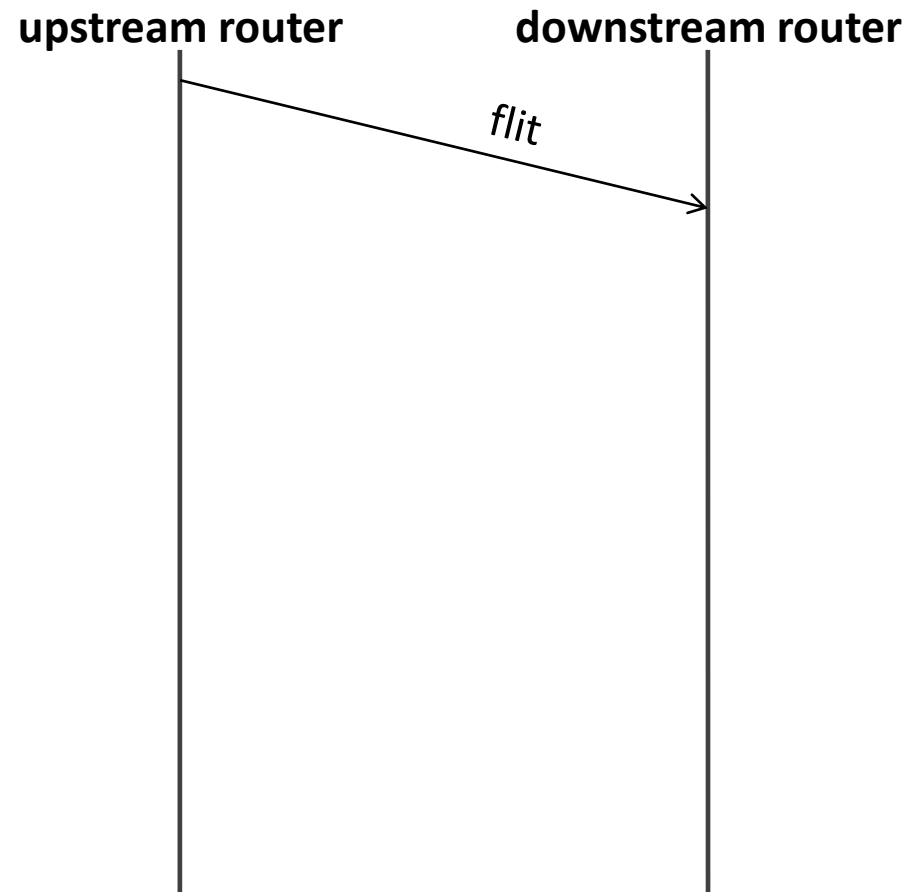
upstream router

downstream router

Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

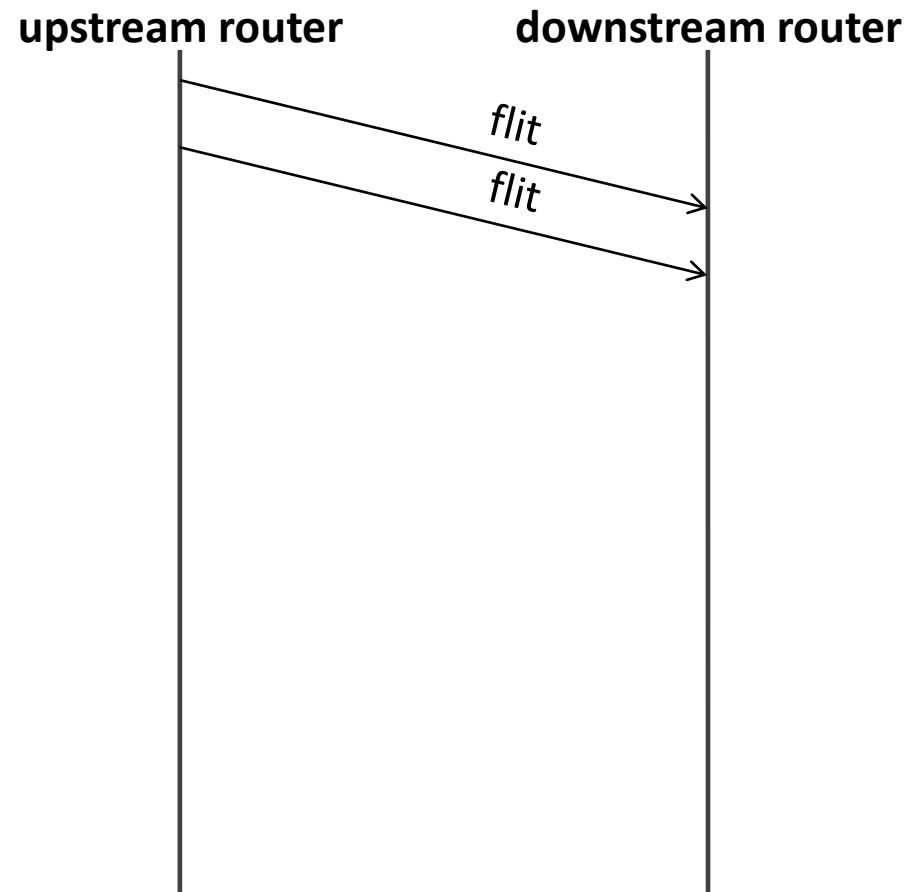
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

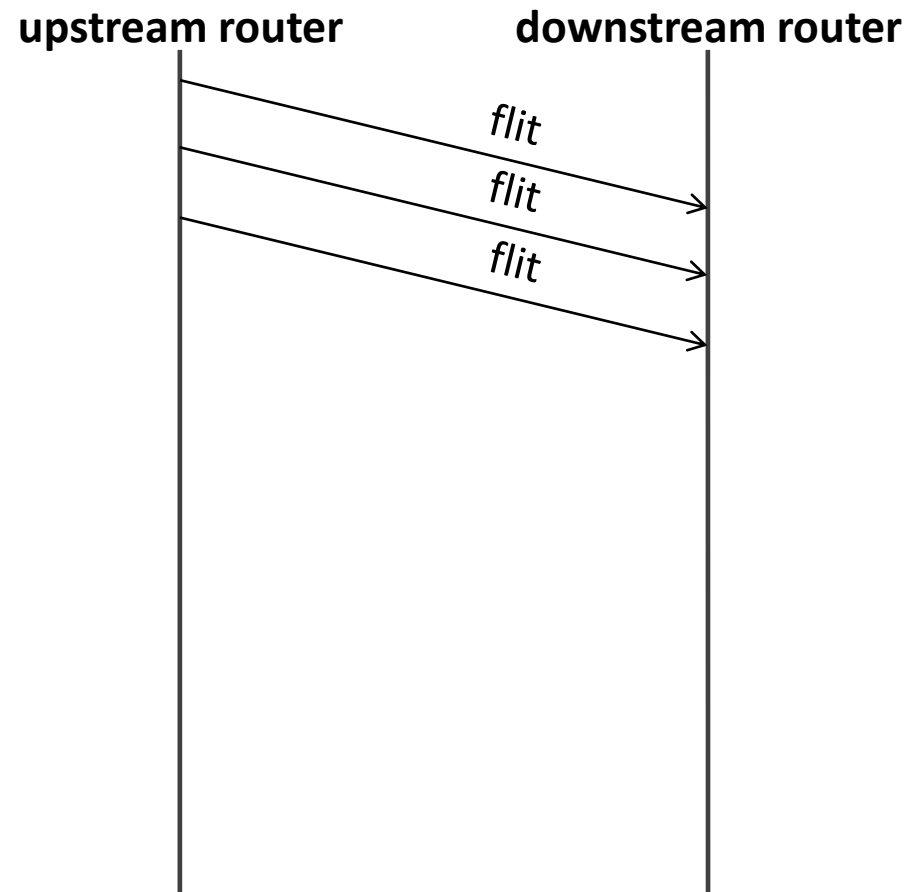
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

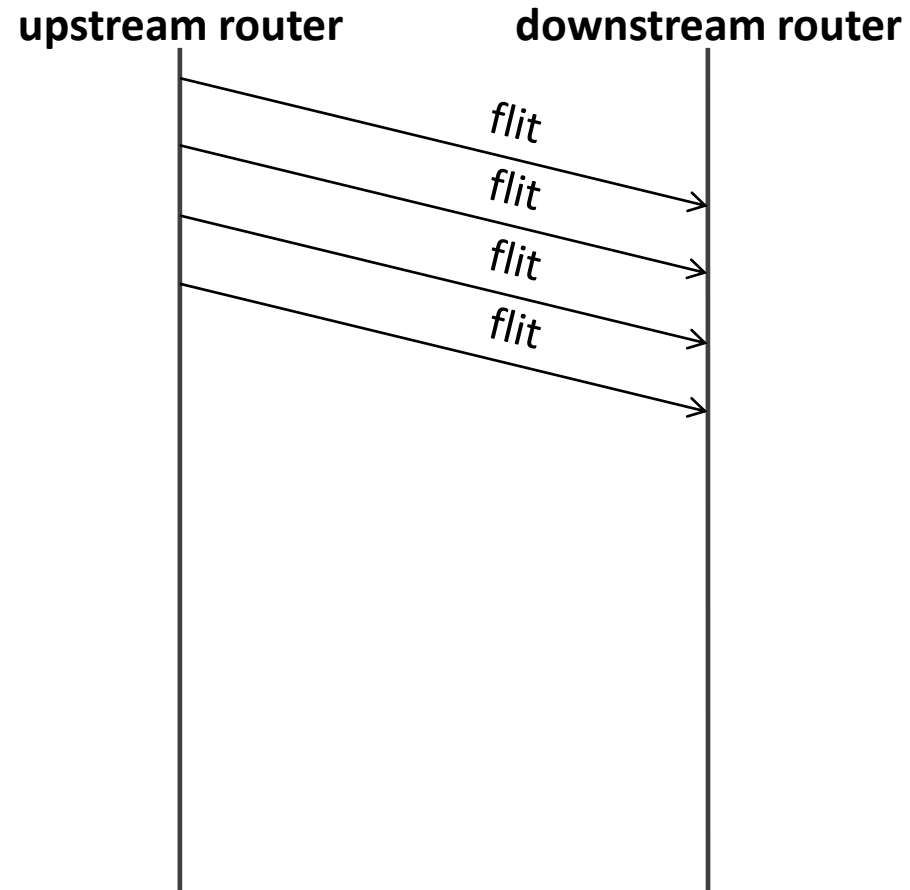
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

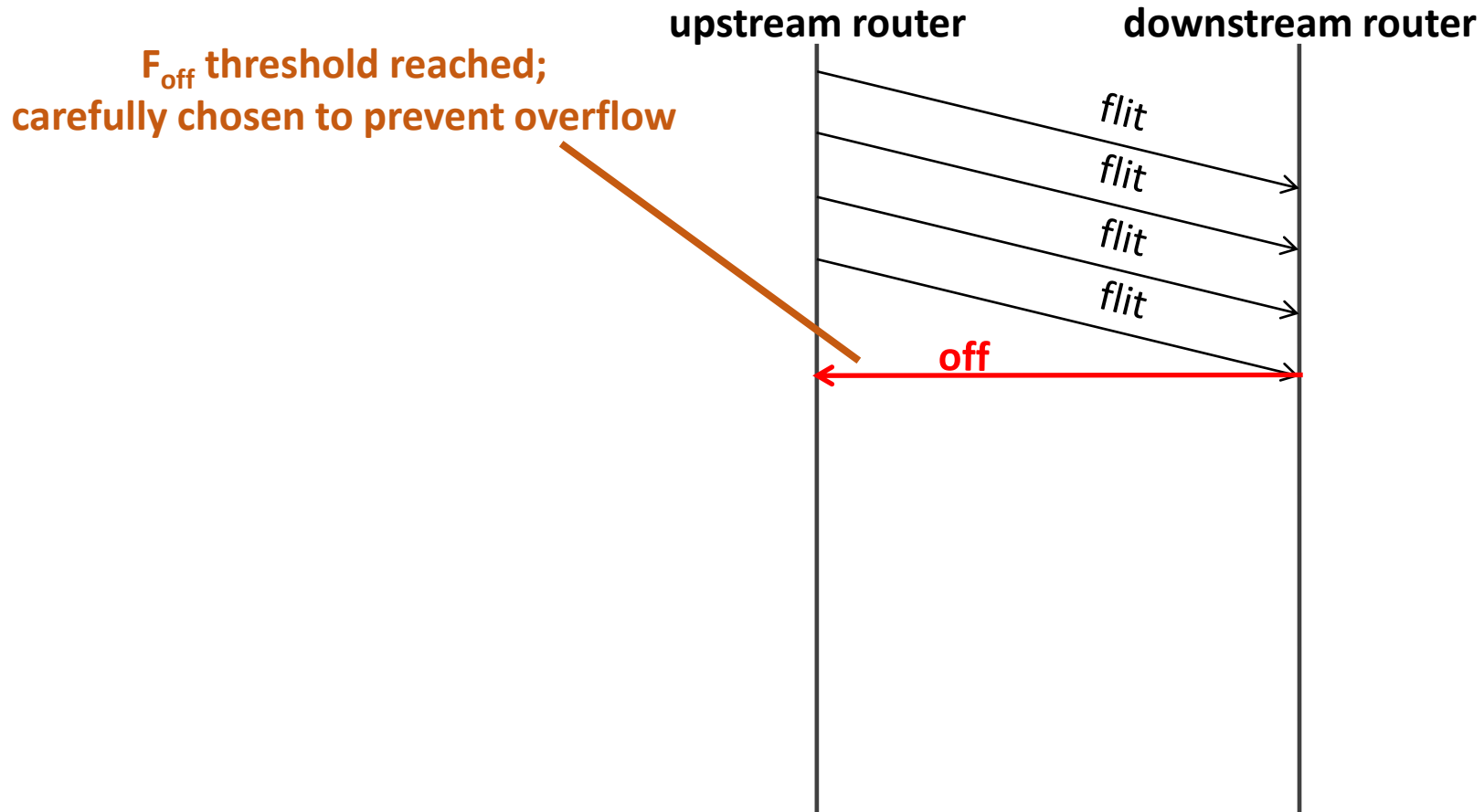
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

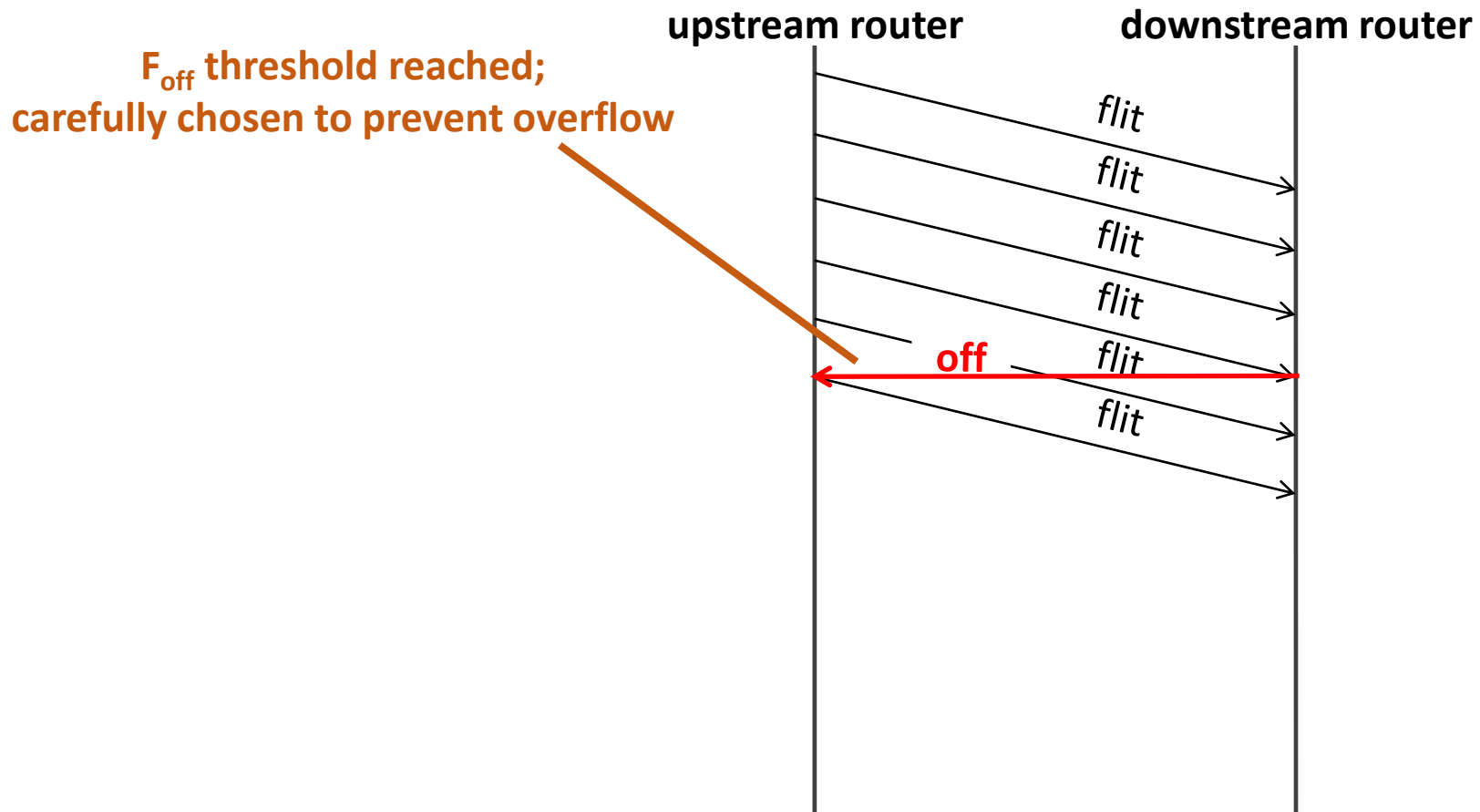
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

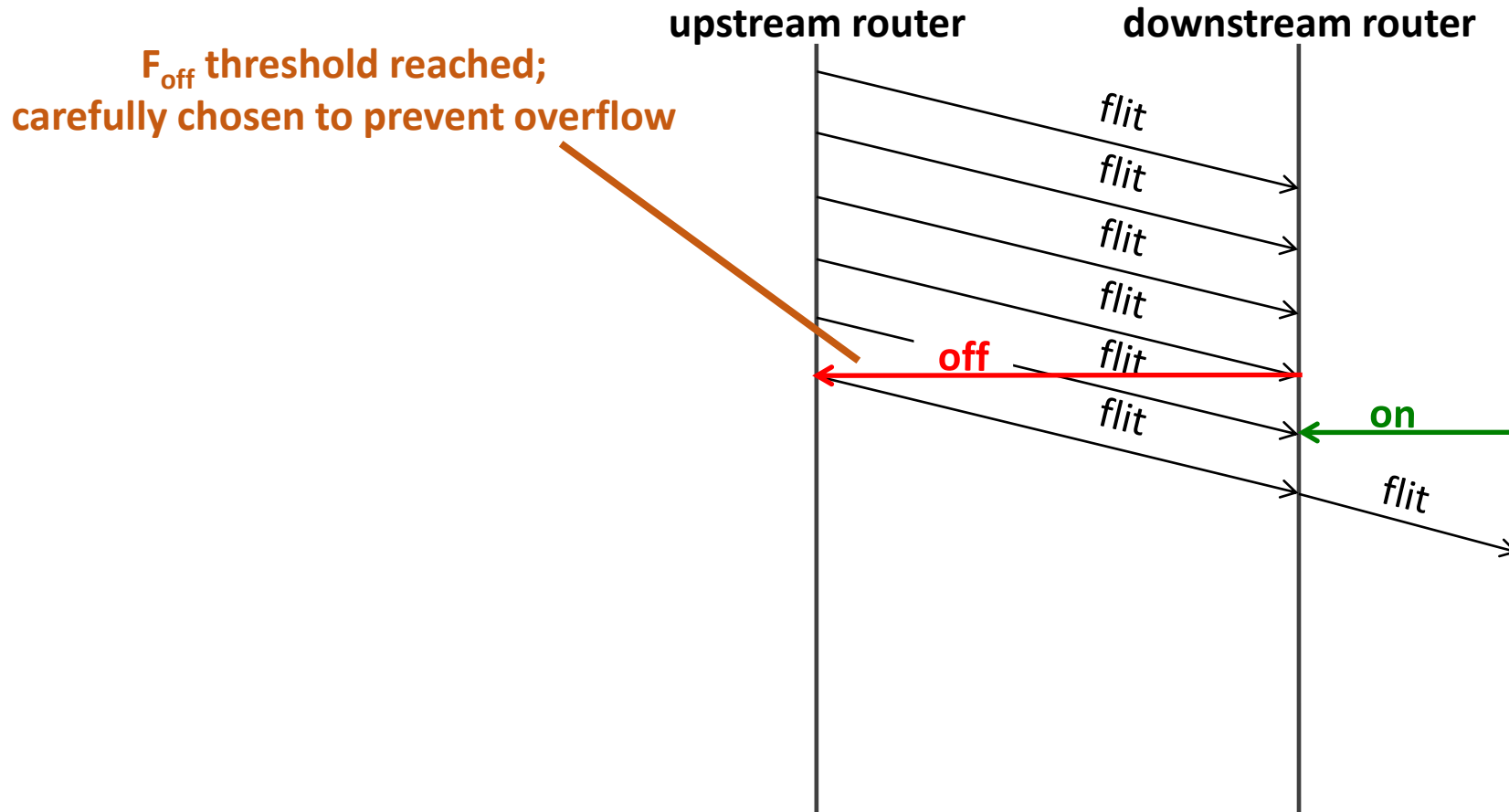
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

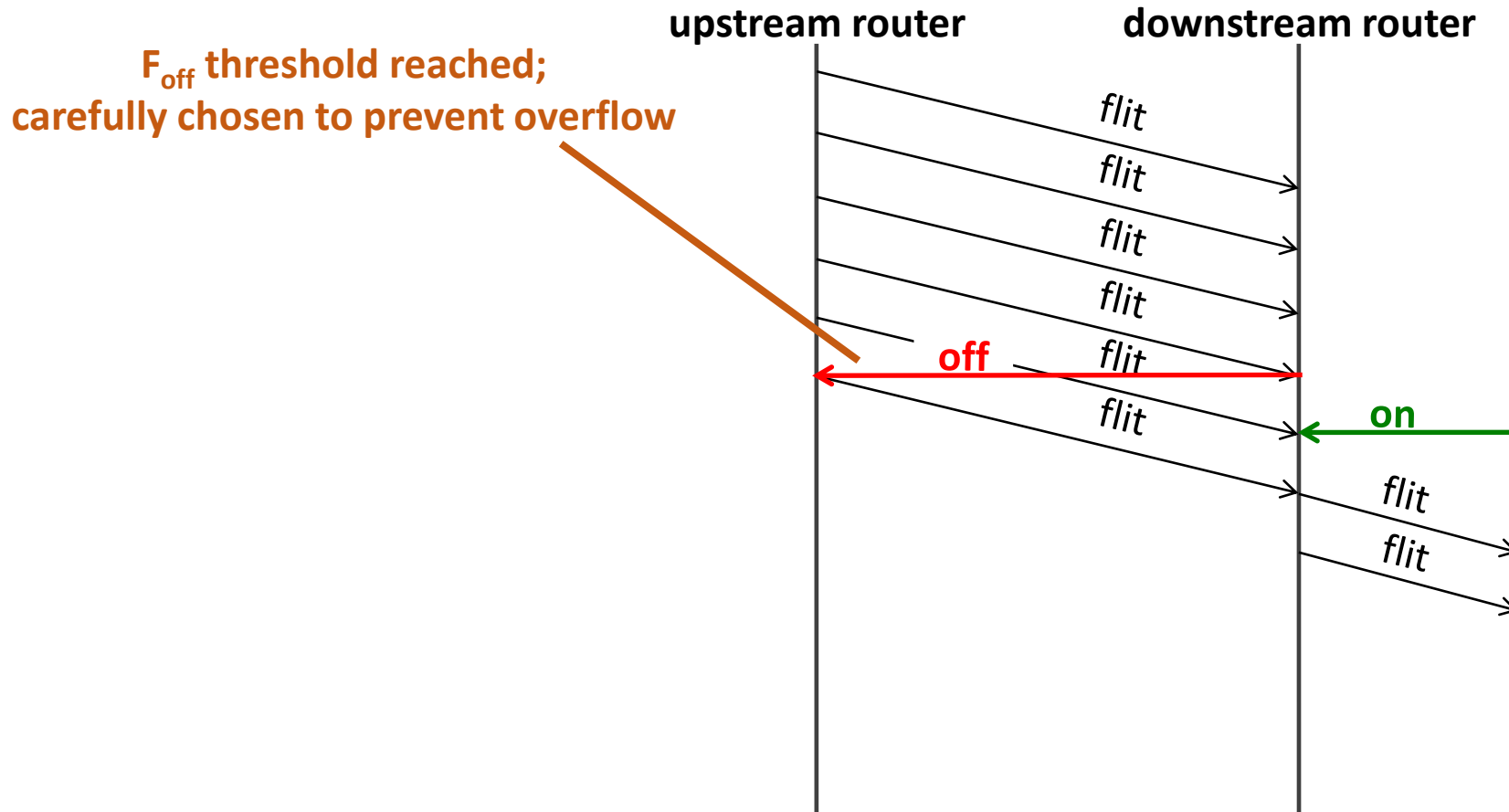
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

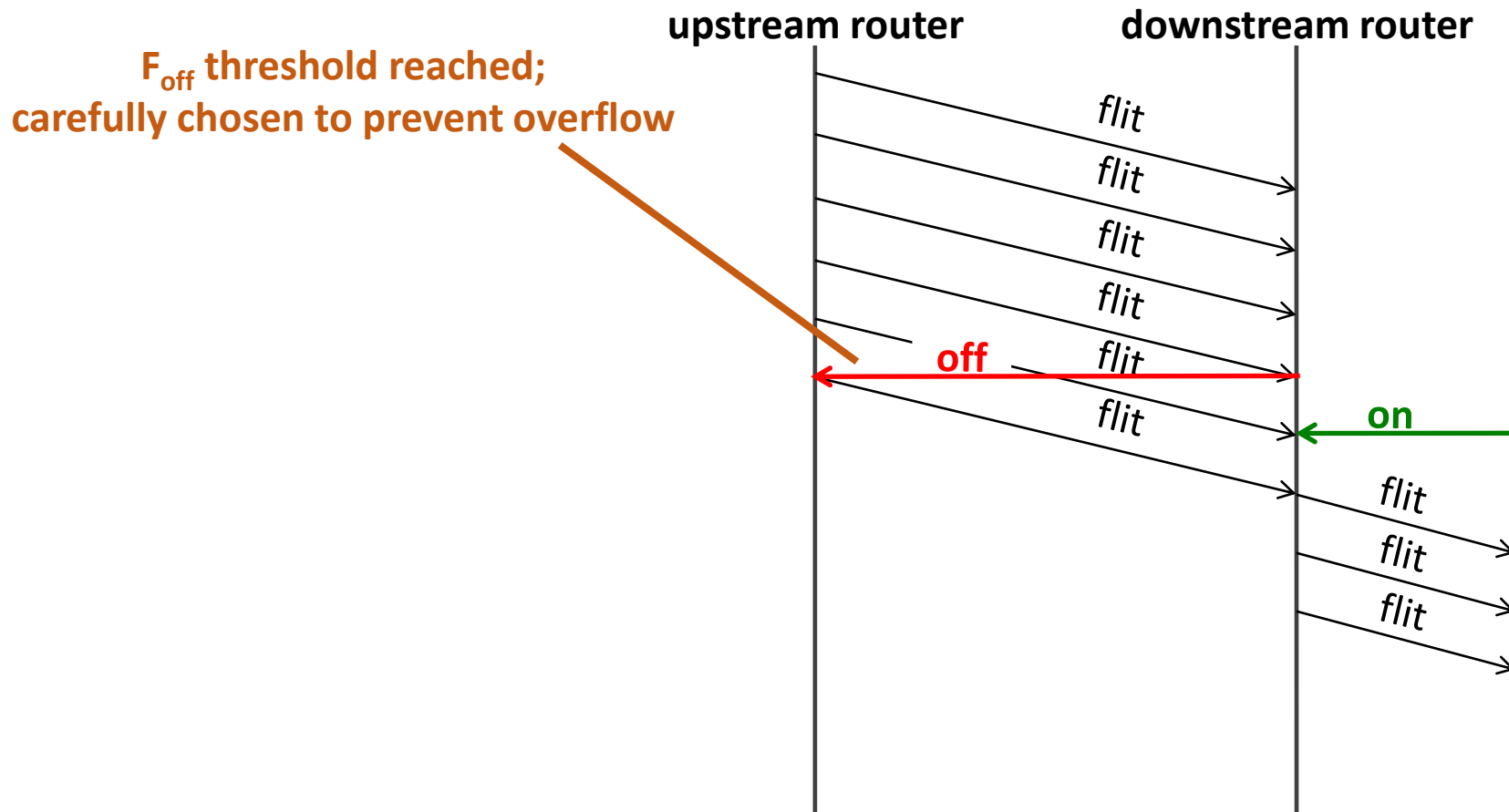
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

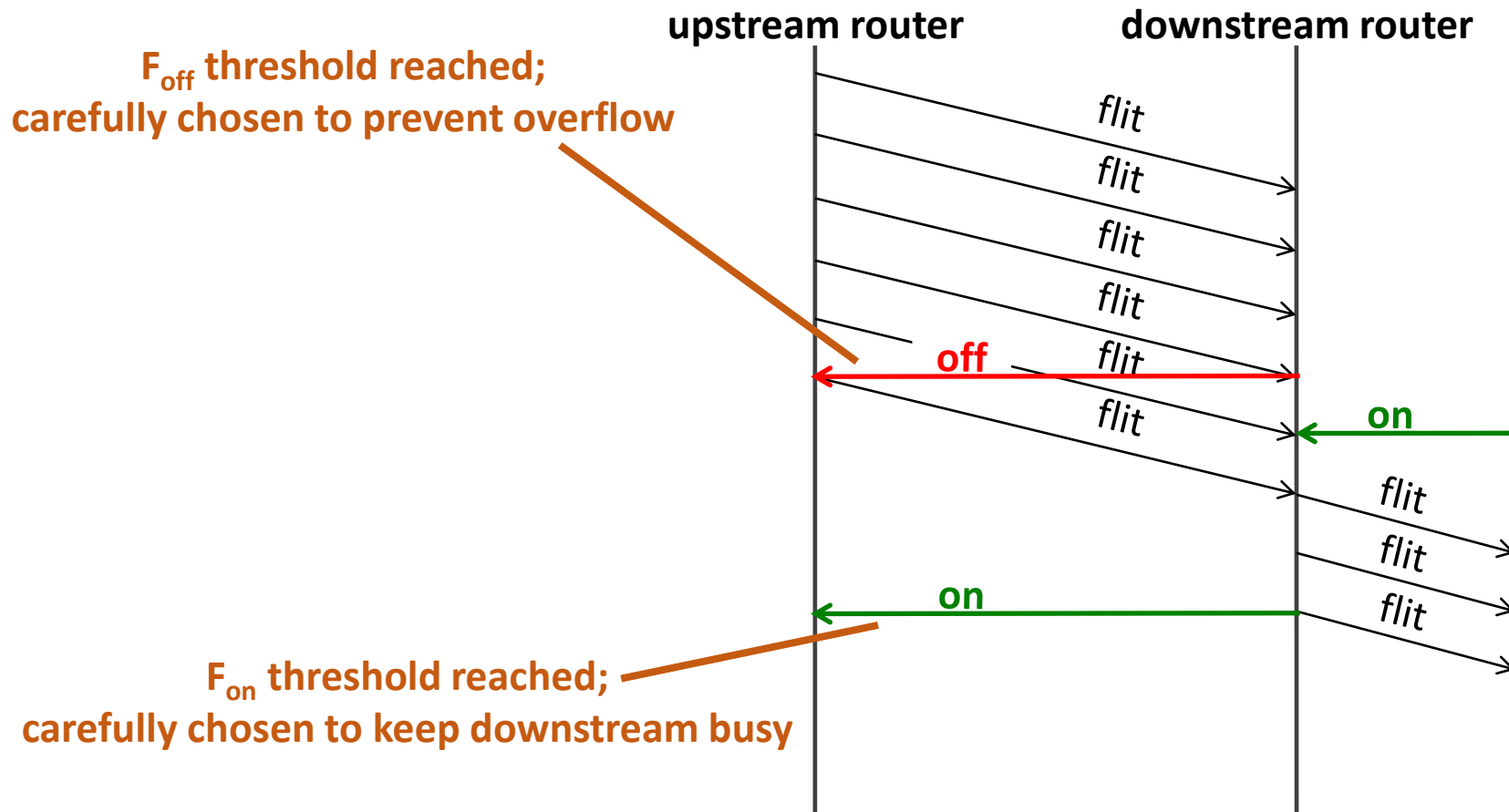
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

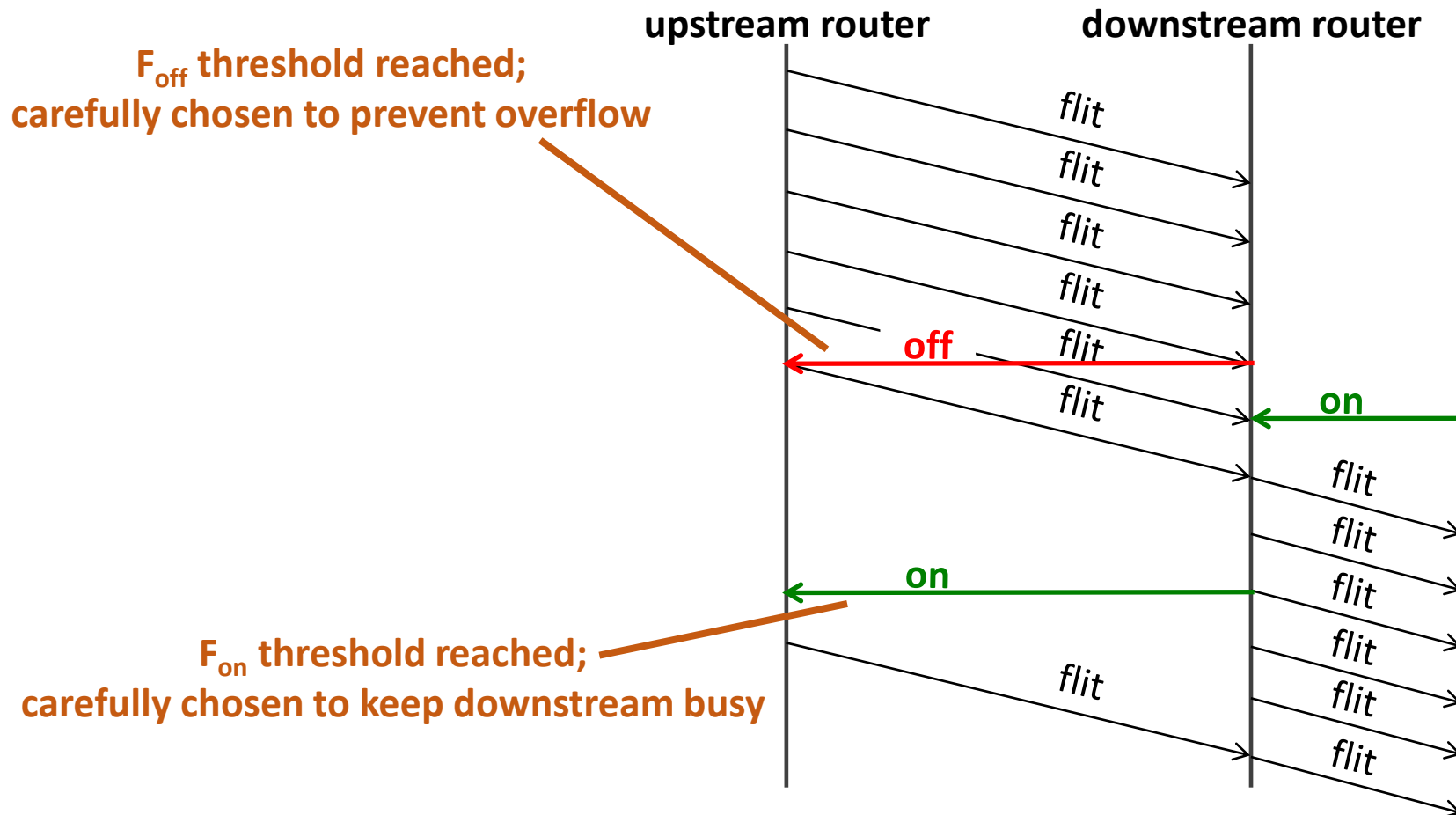
- Set thresholds to trade off between amount of signaling and buffer utilization



Buffer Backpressure – On-Off Flow Control

e.g., Assume 6-flit downstream buffers, 1-cycle flit processing and 1-cycle link

- Set thresholds to trade off between amount of signaling and buffer utilization



Network-On-Chip

Key design decisions:

Topology

- Where to place routers/nodes and how to link them together

Routing

- Which set of links to traverse between source-destination pairs

Flow Control

- What granularity to move messages through links and routers

Router Microarchitecture

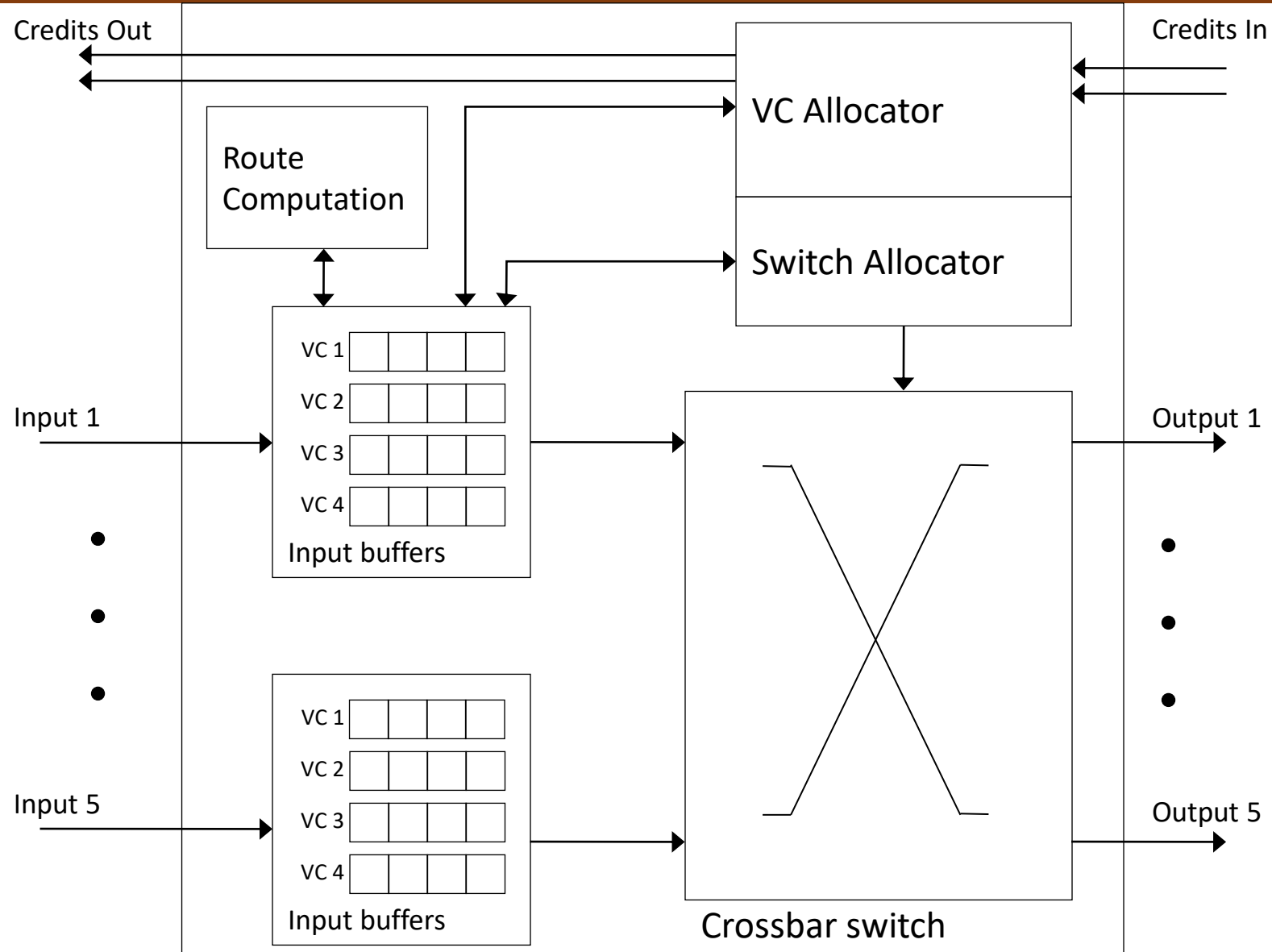
- How to pipeline and arbitrate incoming messages in routers

Router Microarchitecture

Implementation of routing algorithm, flow control and router pipeline

- Impacts per-hop delay and energy
- Router complexity increases with bandwidth demands
- Focus on router microarchitecture with virtual channels

Router Microarchitecture



Router Components

Input buffers, route computation logic, virtual channel allocator, switch allocator, crossbar switch

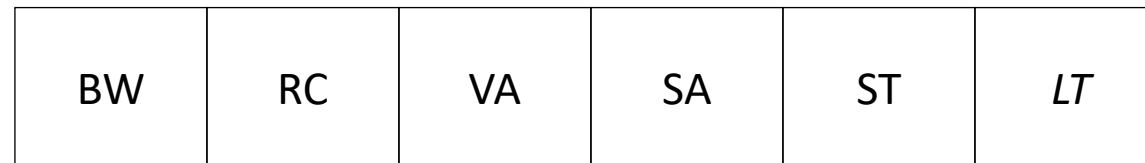
Most NoC routers are input-buffered

- Use single-ported memories
- Input buffer stores flits for duration in router

Router Pipeline

Canonical 5-stage pipeline

- BW: buffer write
- RC: routing computation
- VA: virtual channel allocation
- SA: switch allocation
- ST: switch traversal
- *LT: link traversal*



Router Pipeline

Routing computation and virtual channel allocation are performed once per packet

- Body and tail flits inherit this info from head flit

Router Pipeline

Routing computation and virtual channel allocation are performed once per packet

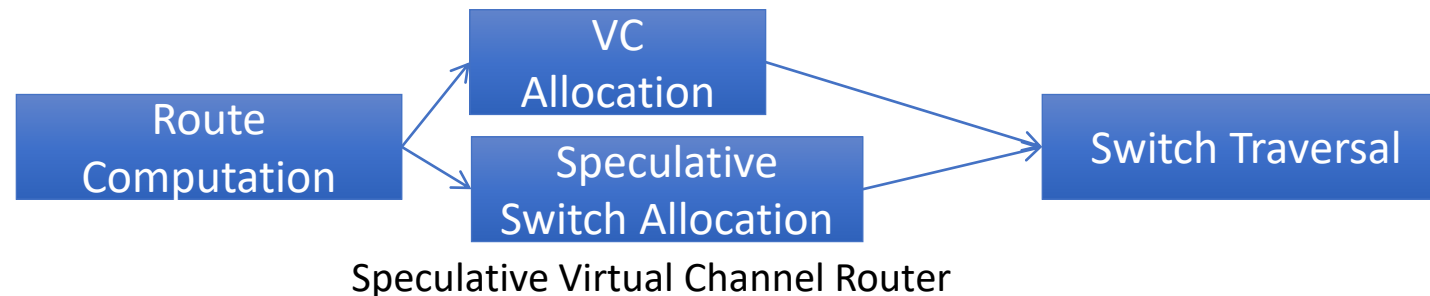
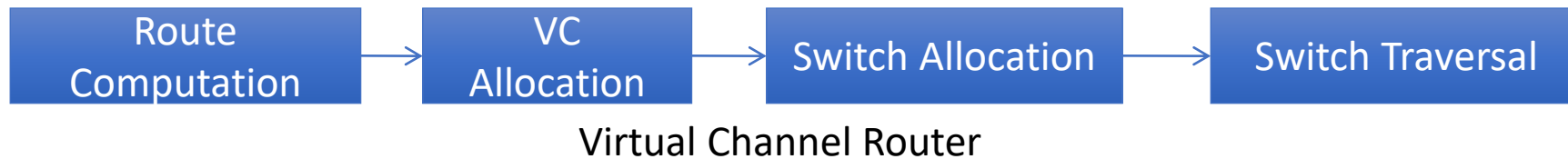
- Body and tail flits inherit this info from head flit

Cycle	1	2	3	4	5	6	7	8	9
Head	BW	RC	VA	SA	ST	LT			
Body 1		BW			SA	ST	LT		
Body 2			BW			SA	ST	LT	
Tail				BW			SA	ST	LT

Router Pipeline

Dependence between router modules

- Determines critical path through router
- Cannot bid for switch port (SA) until routing computation performed



Router Pipeline

Zero-load packet latency with baseline wormhole VC router

- Ideally, only pay link delay

$$= (5 \text{ cycles} + \text{link delay}) \times \text{hops} + t_{\text{serialization}}$$

Pipeline Optimizations – Lookahead Routing

At current router, perform routing computation for next router

- Overlap RC with BW
- Pre-computing route allows flits to compete for VCs immediately after BW

BW RC	VA	SA	ST	LT
----------	----	----	----	----

Pipeline Optimizations – Speculation

Assume virtual channel allocation will be successful

- Typically true under low to moderate network loads
- Perform VA and SA in parallel
- If VA unsuccessful (i.e., no VC returned), try again in next cycle
- Prioritize non-speculative requests

BW RC	VA SA	ST	LT
----------	----------	----	----

Pipeline Optimizations – Bypassing

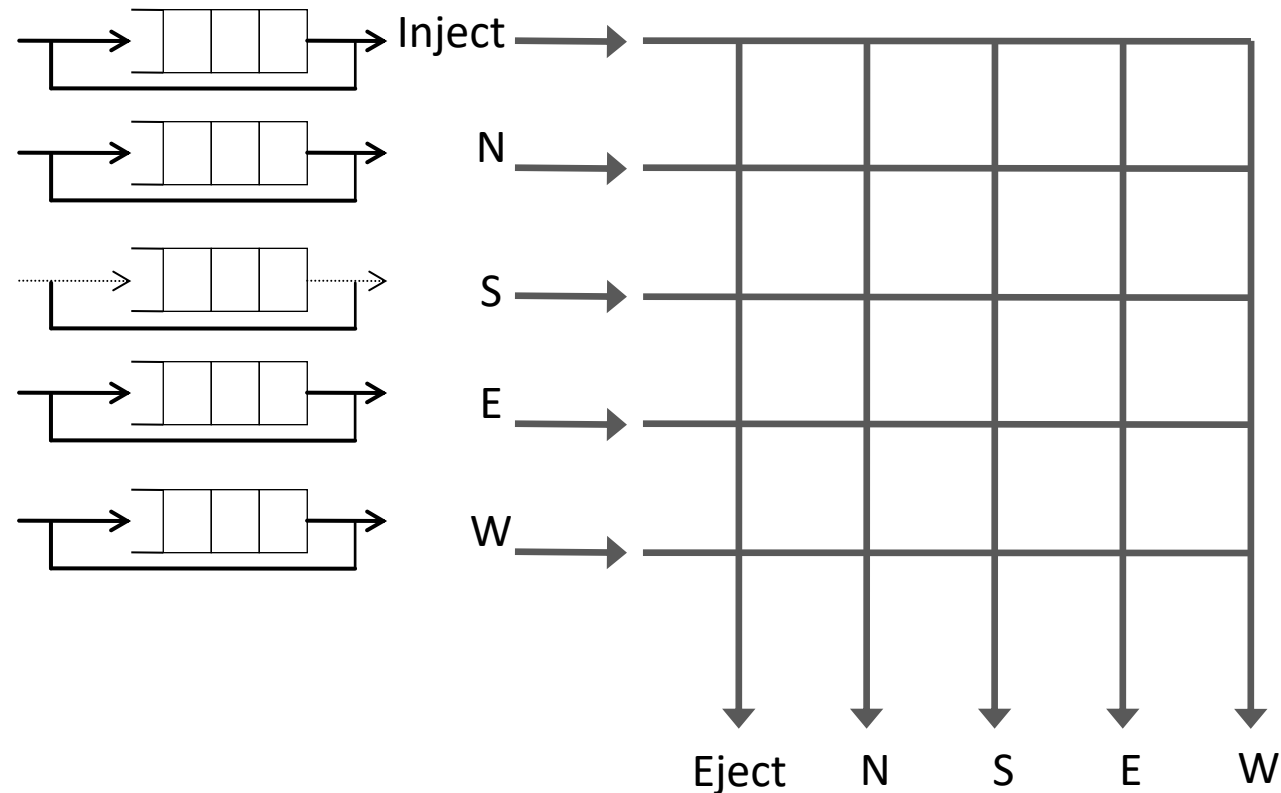
When no flits in input buffer, speculatively enter ST

- In the first stage, a free VC is allocated, then routing is performed and the crossbar is set up
- On port conflict, speculation canceled

VA RC Setup	ST	LT
-------------------	----	----

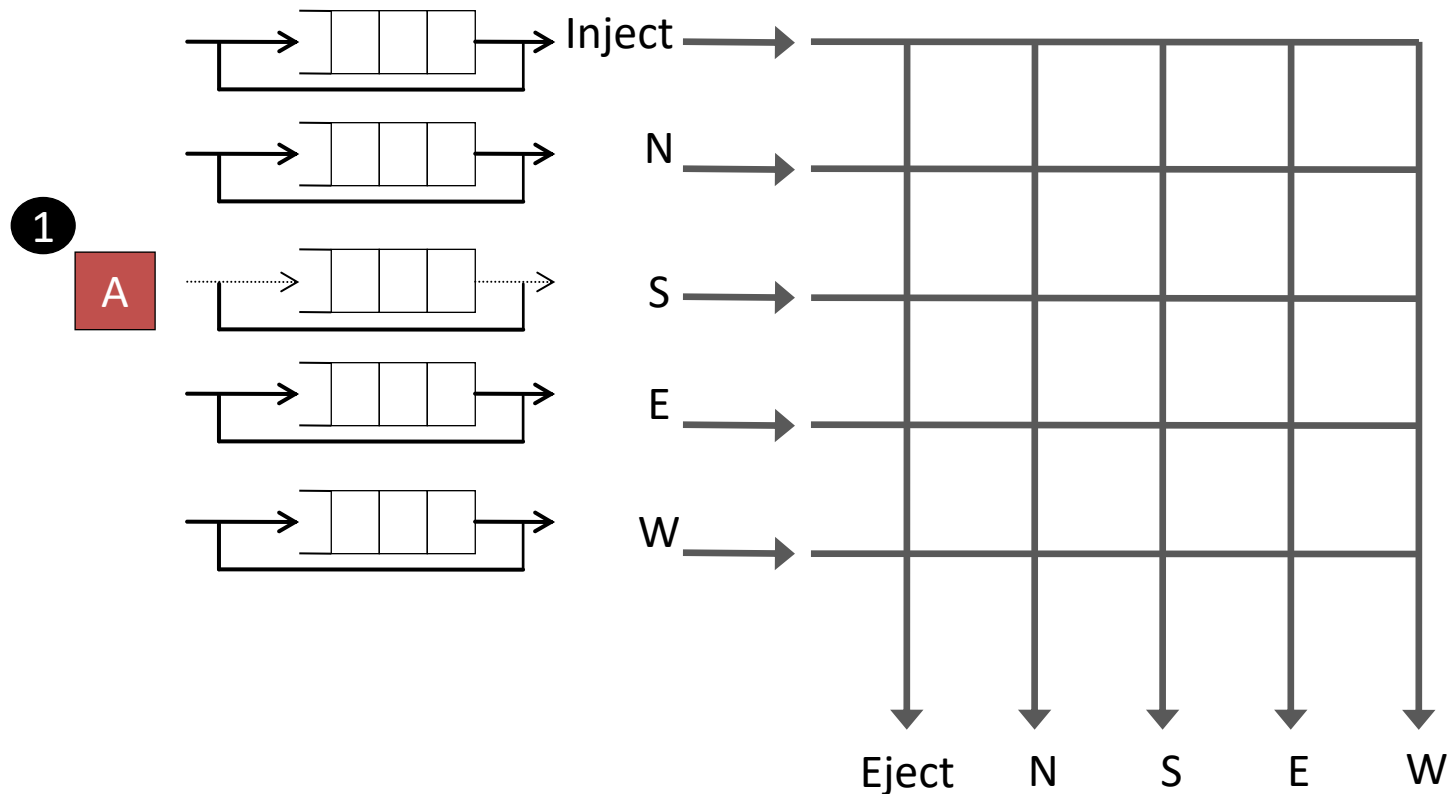
Pipeline Optimizations – Bypassing

e.g., No buffered flits when packet A arrives



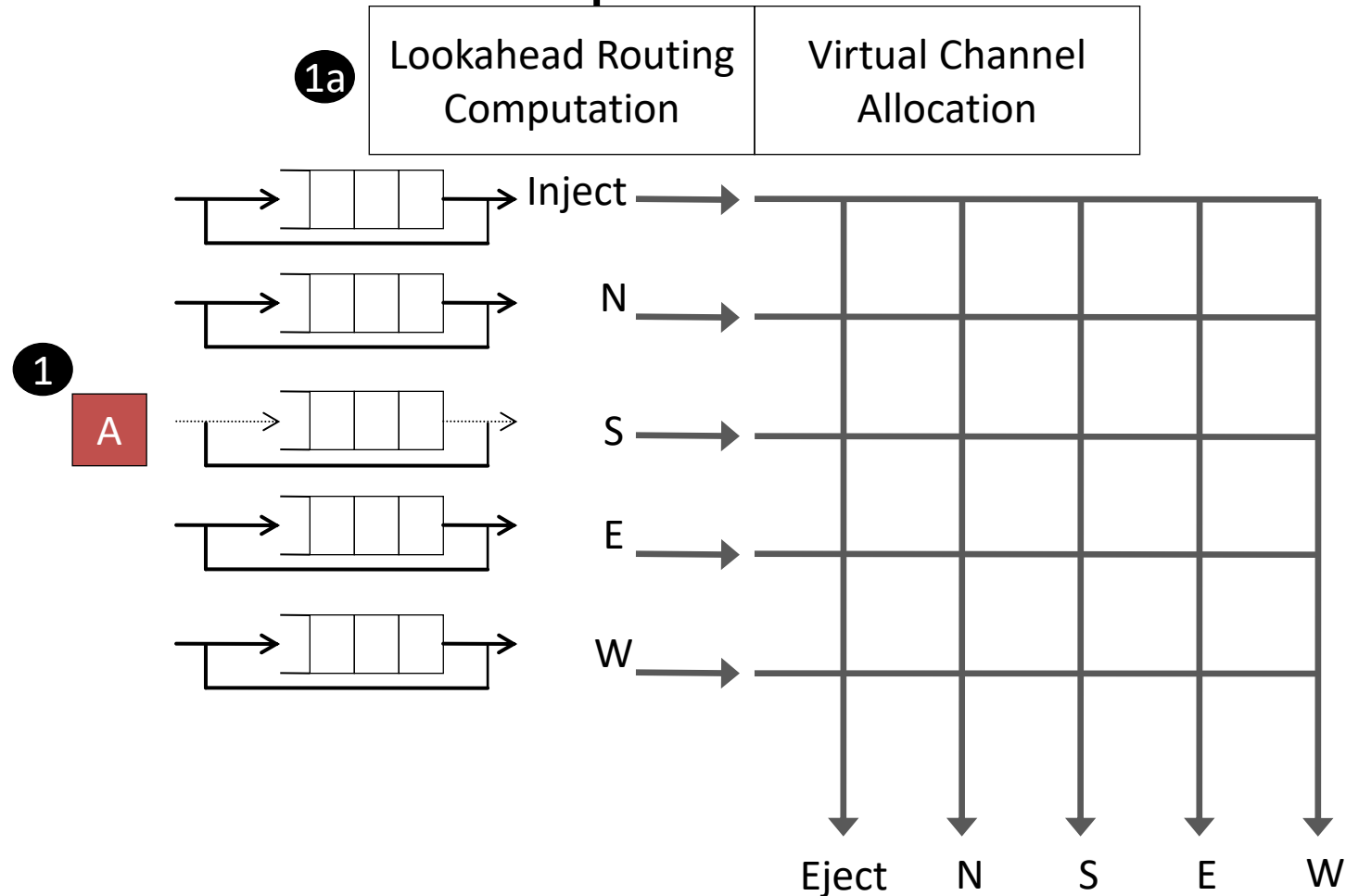
Pipeline Optimizations – Bypassing

e.g., No buffered flits when packet A arrives



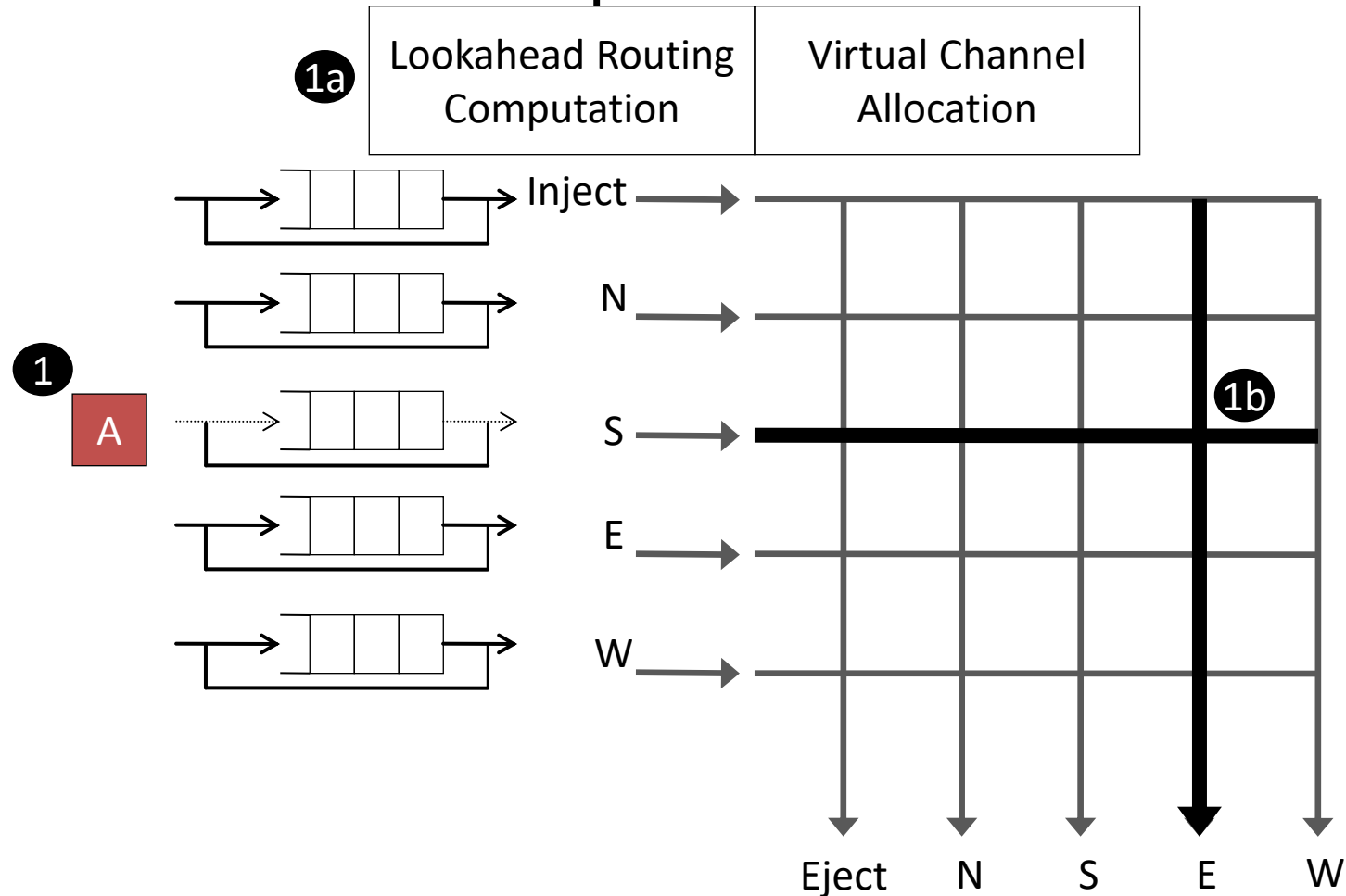
Pipeline Optimizations – Bypassing

e.g., No buffered flits when packet A arrives



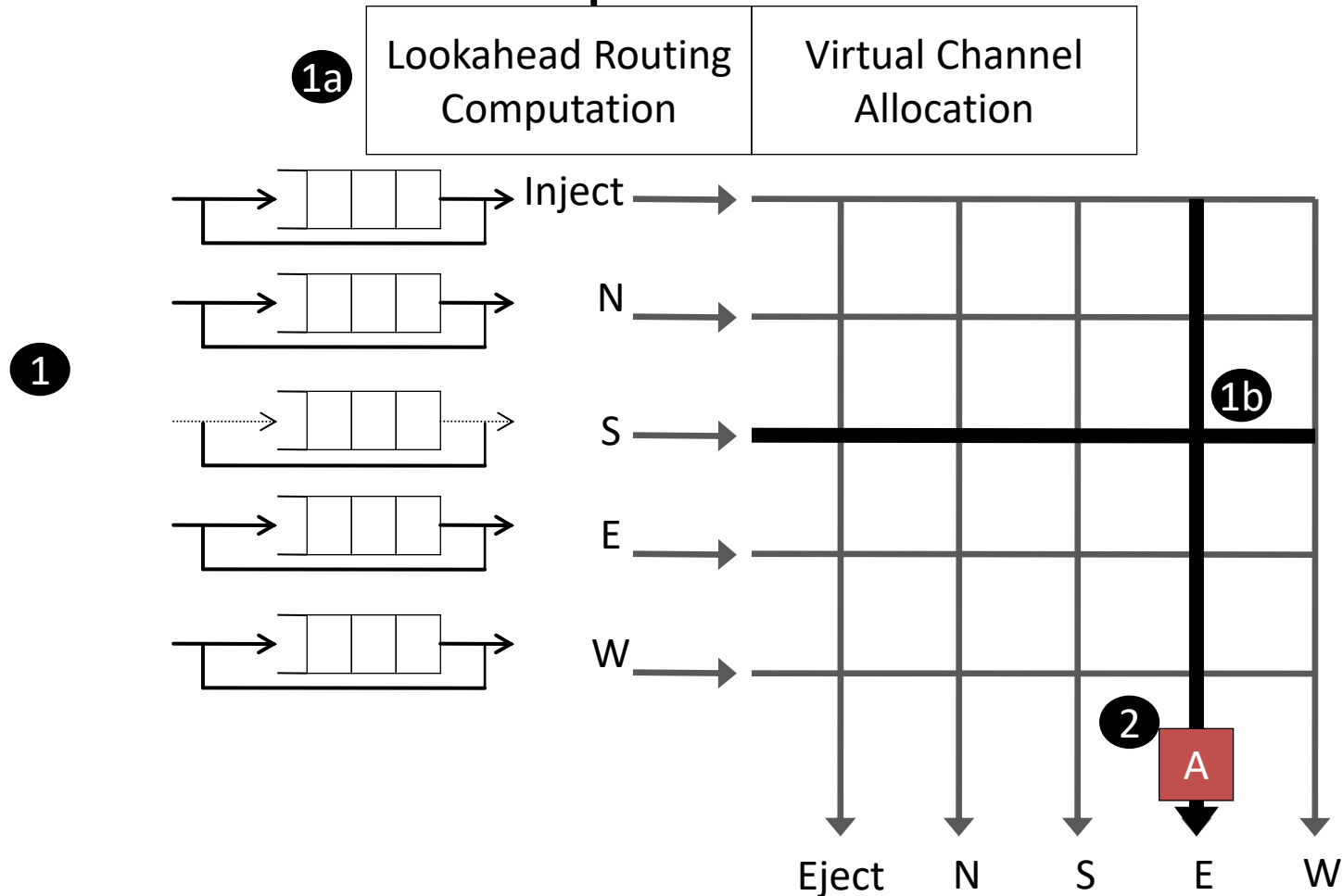
Pipeline Optimizations – Bypassing

e.g., No buffered flits when packet A arrives



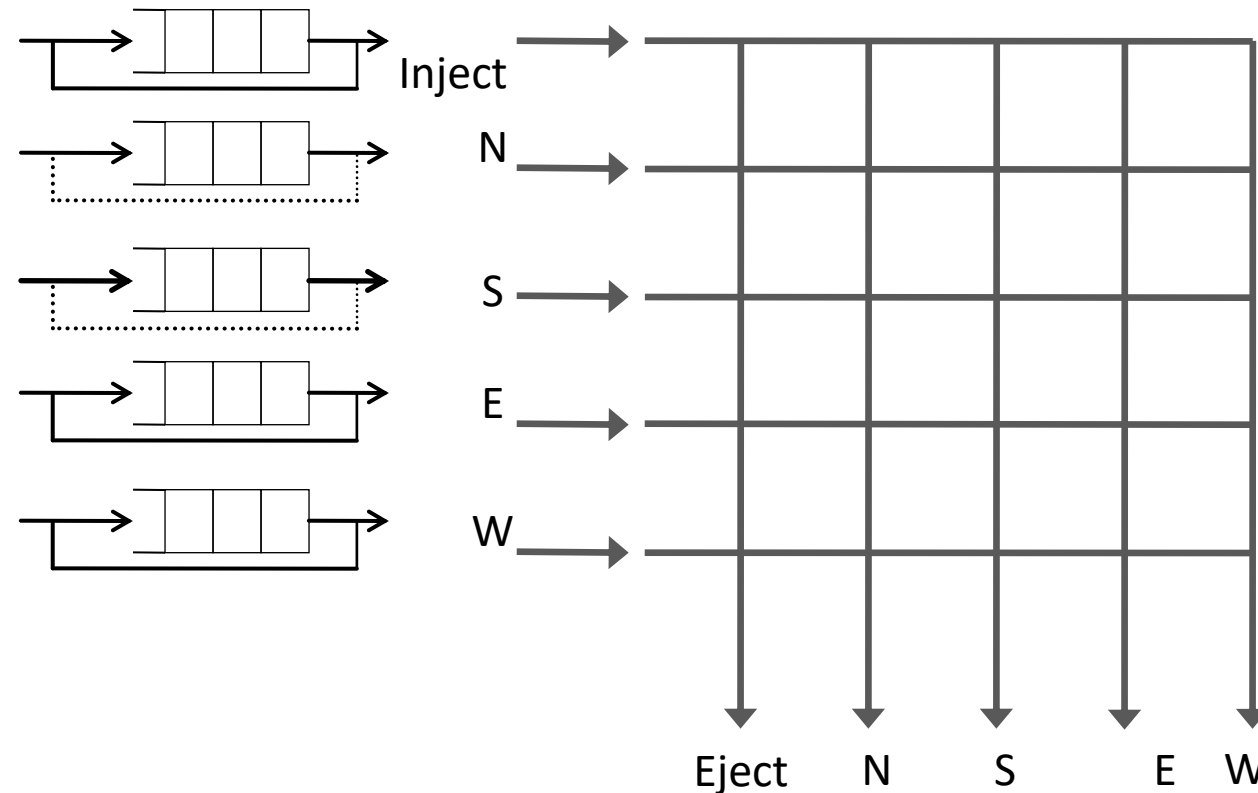
Pipeline Optimizations – Bypassing

e.g., No buffered flits when packet A arrives



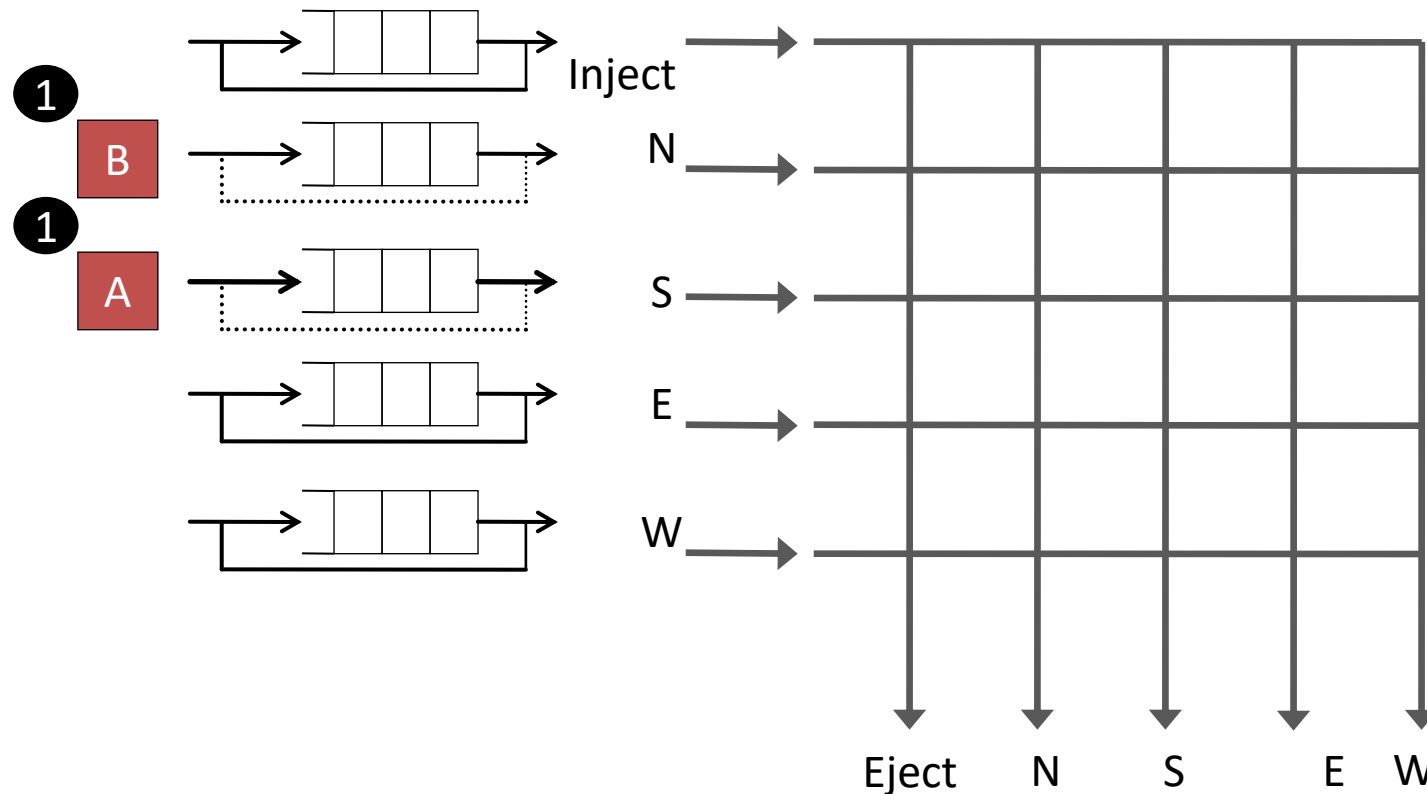
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



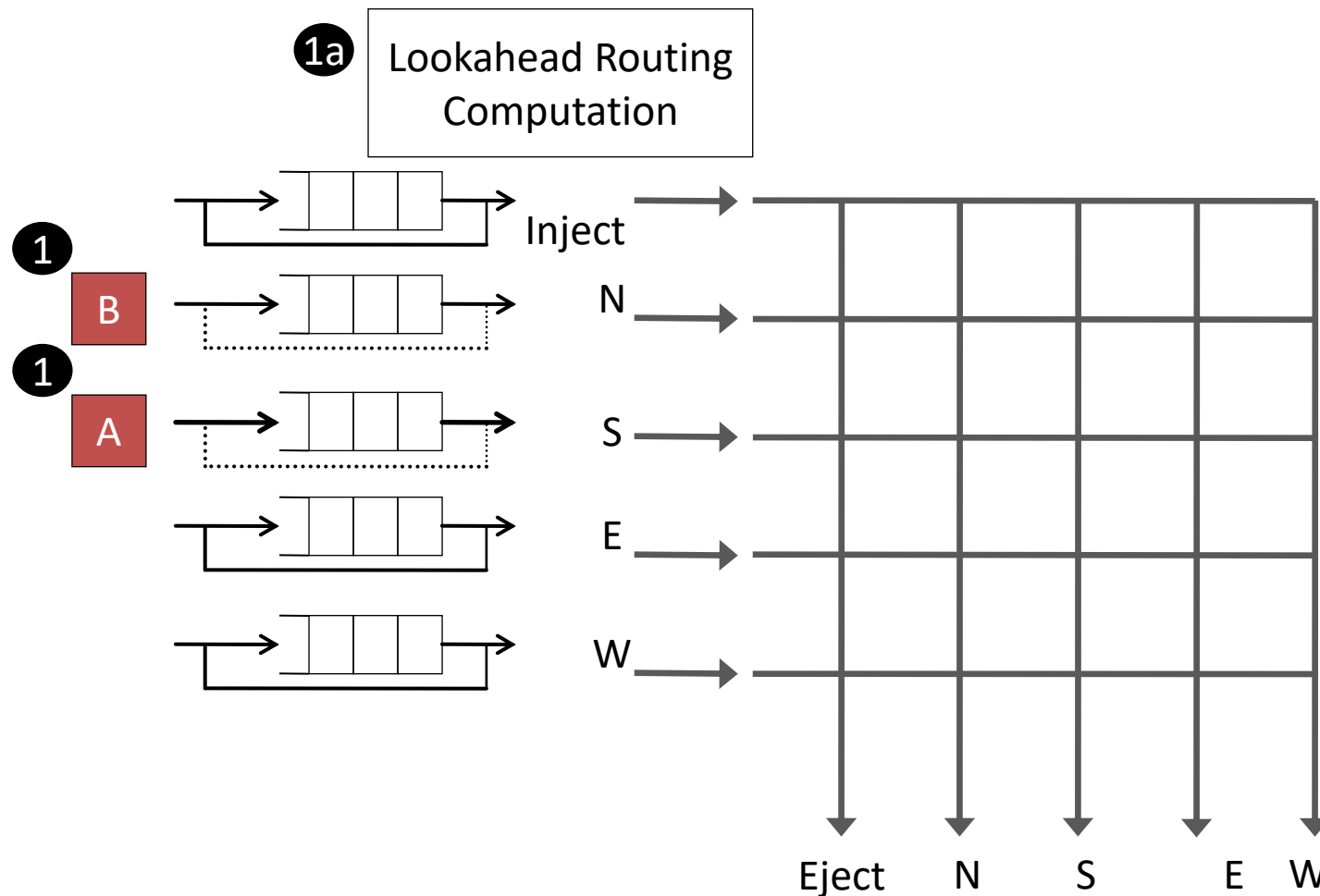
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



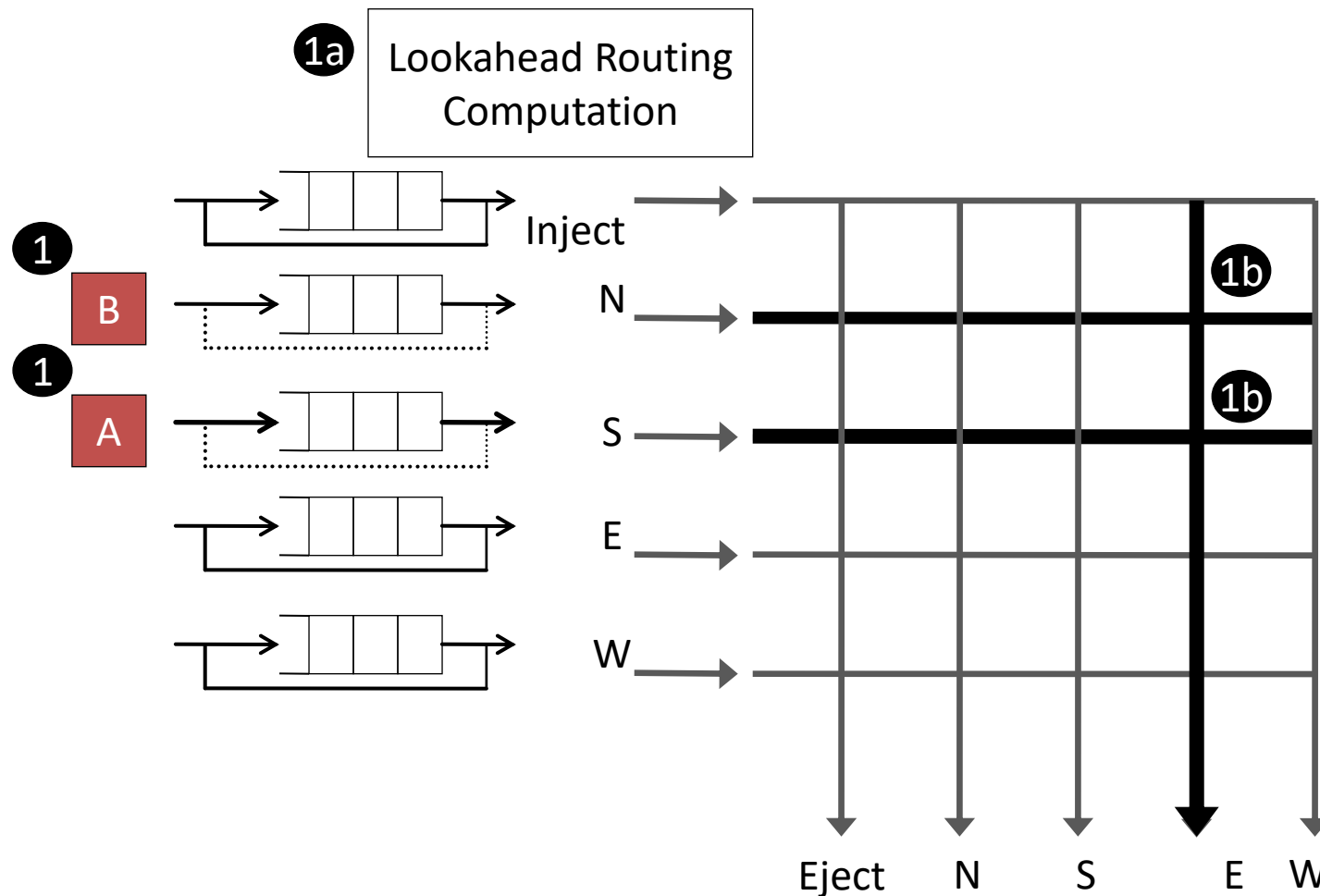
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



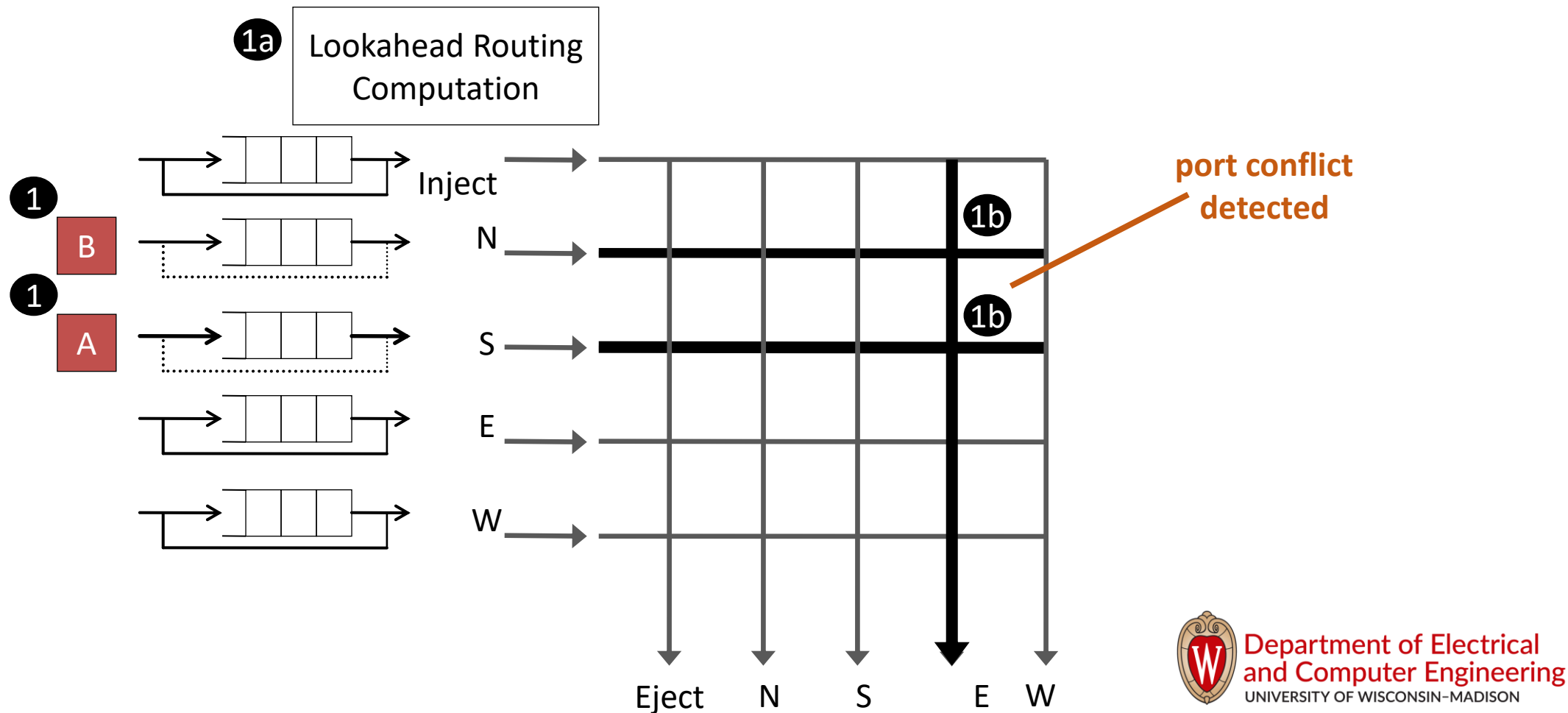
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



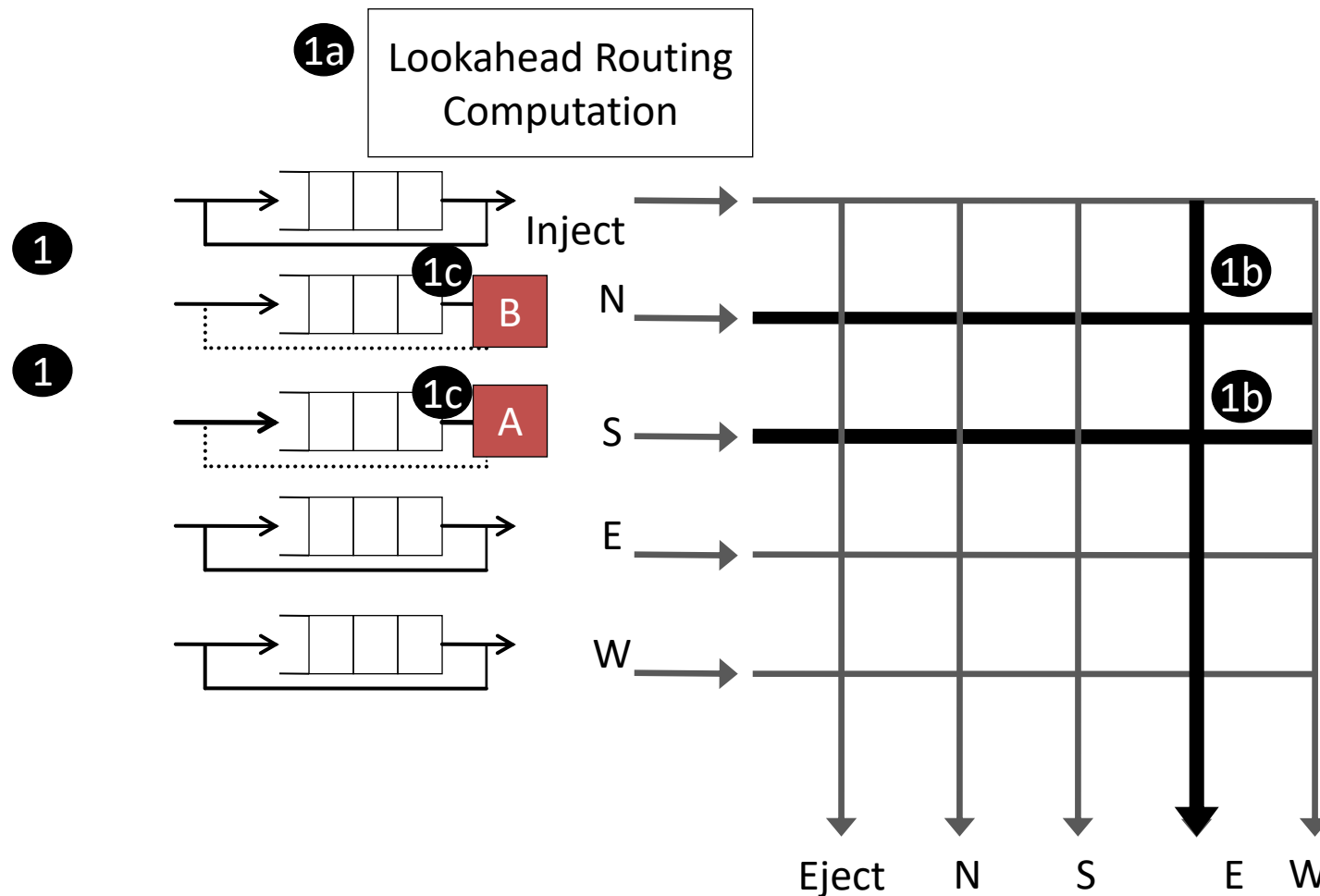
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



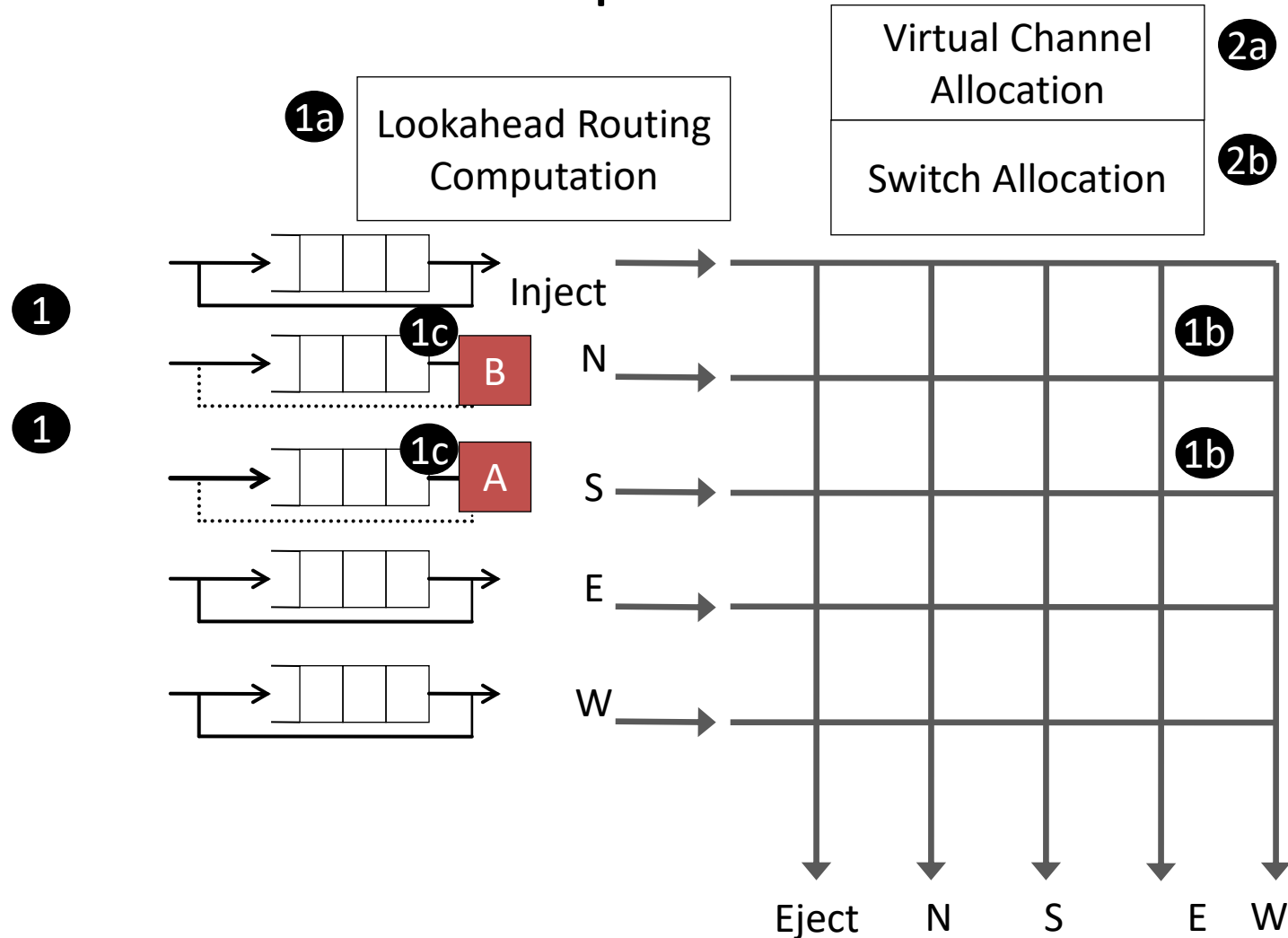
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



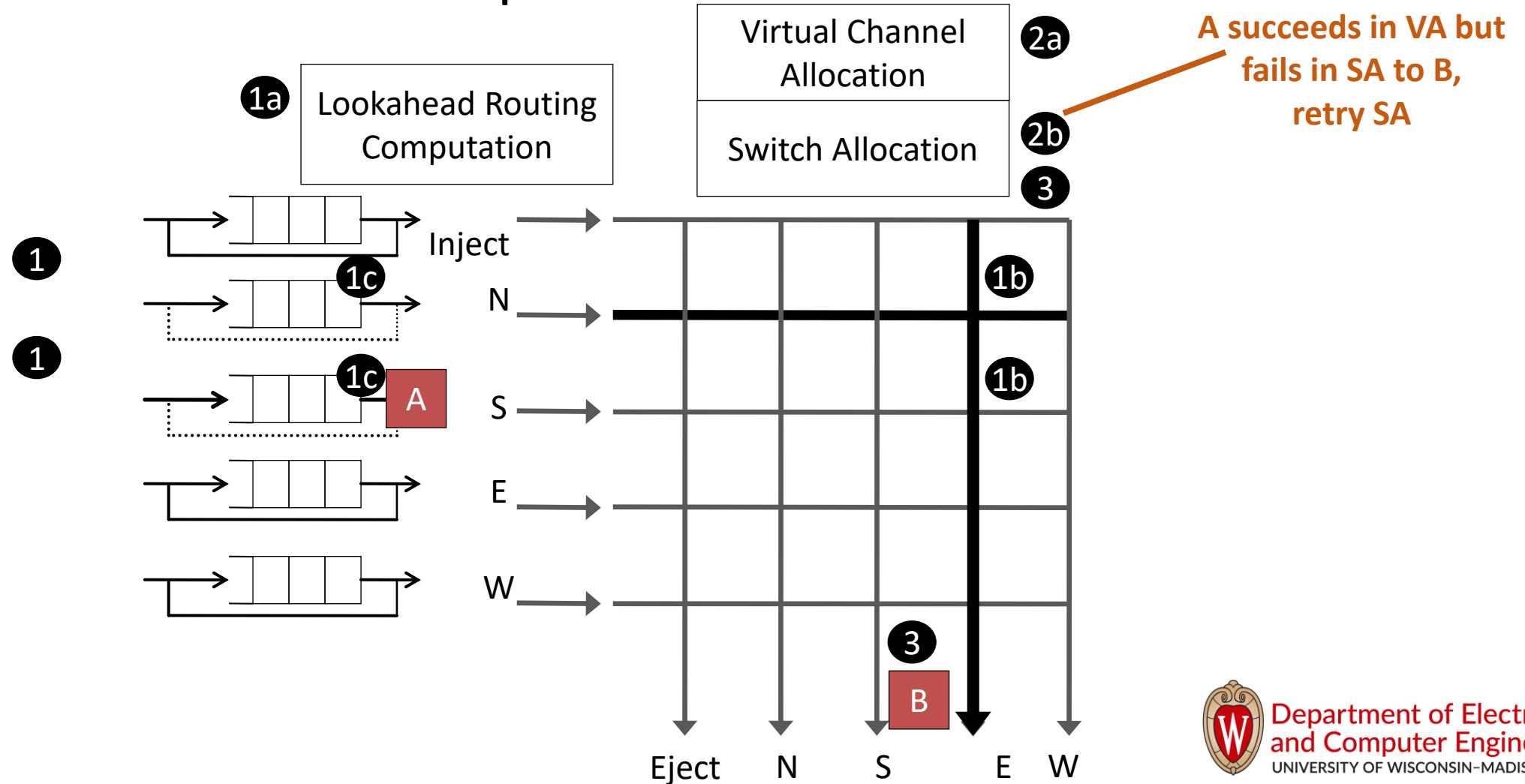
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



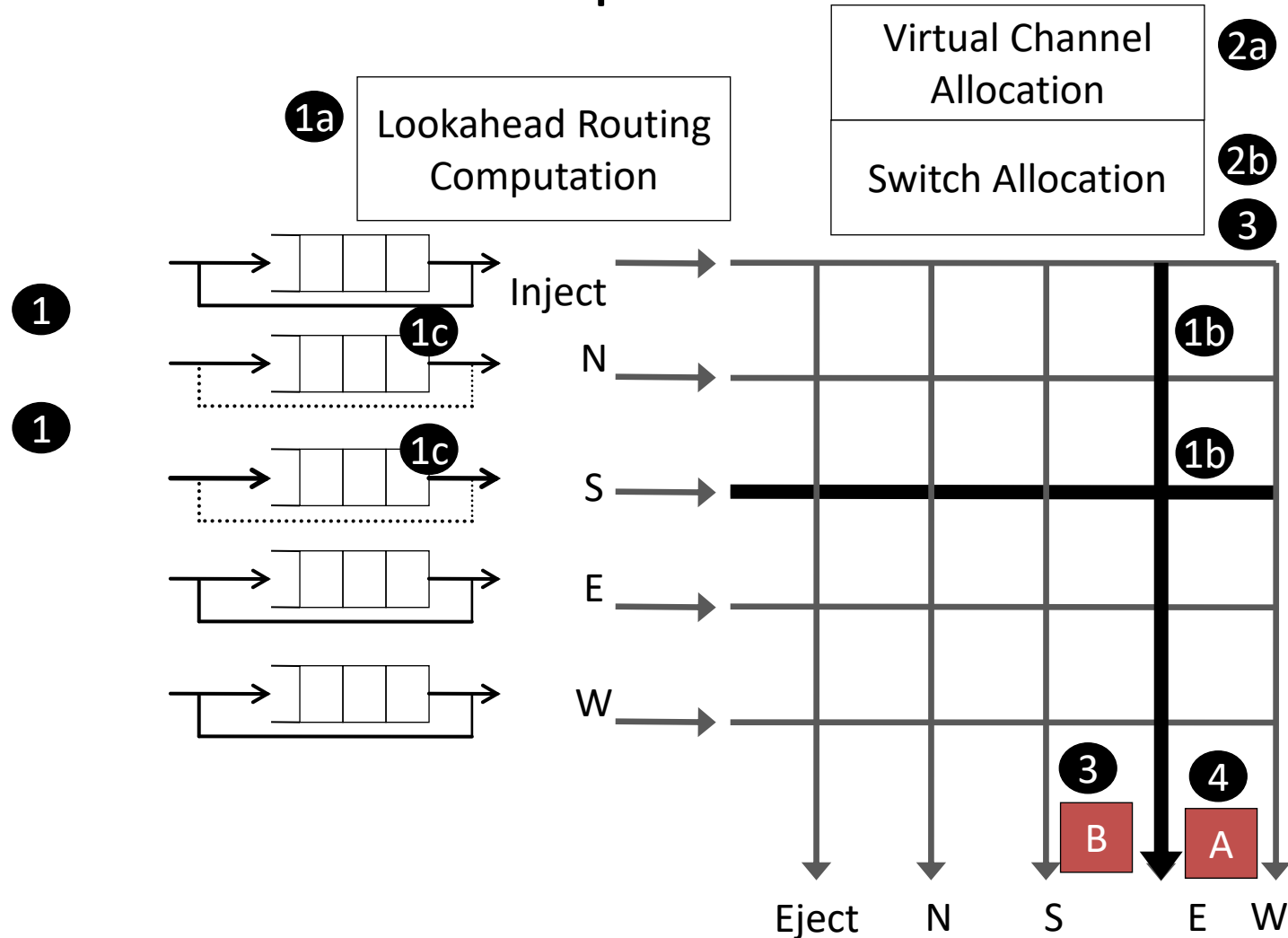
Pipeline Optimizations – Bypassing + Speculation

e.g., No buffered flits when packets A and B arrive



Pipeline Optimizations – Bypassing + Speculation

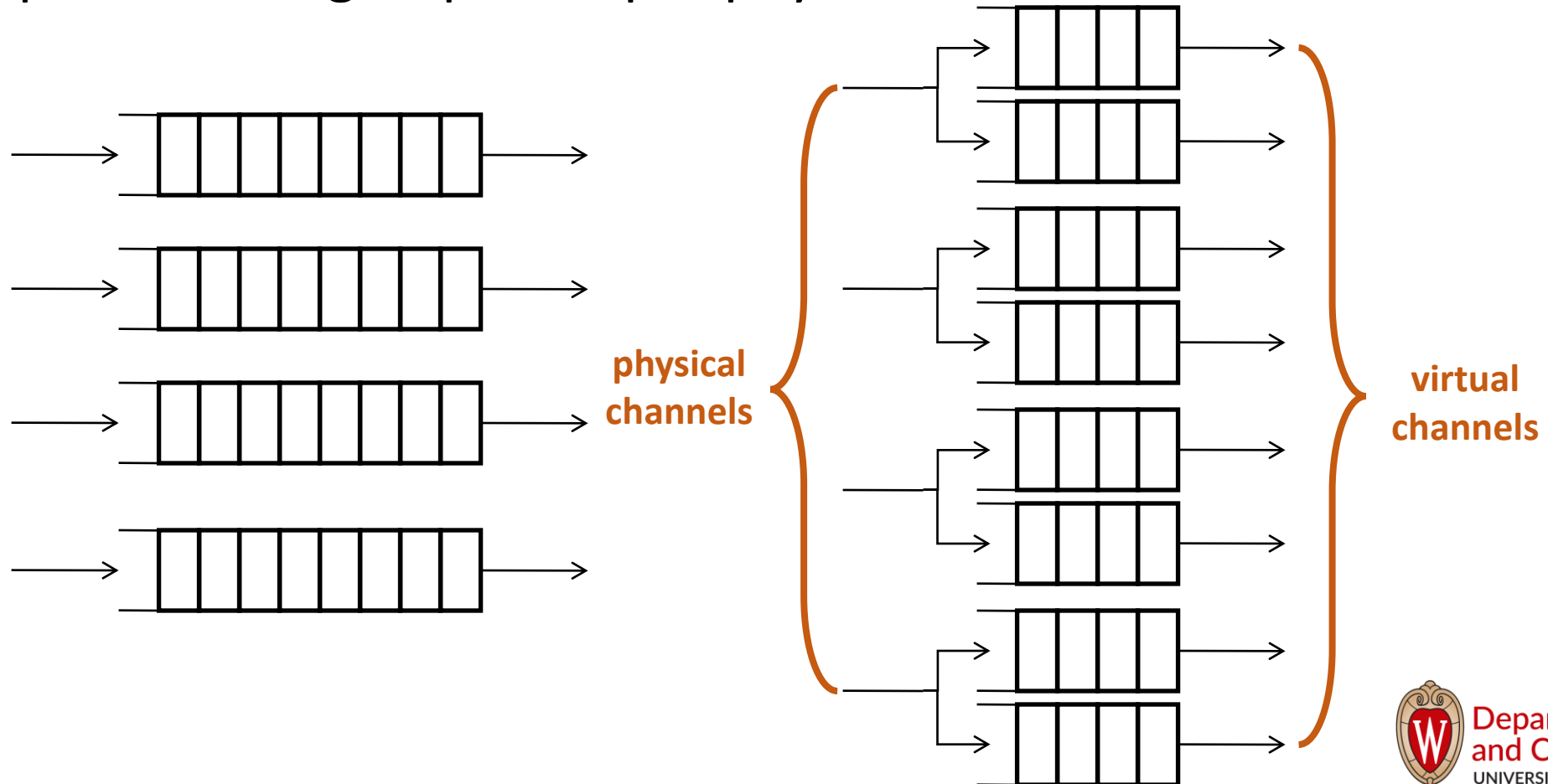
e.g., No buffered flits when packets A and B arrive



Buffer Organization

Single buffer per input

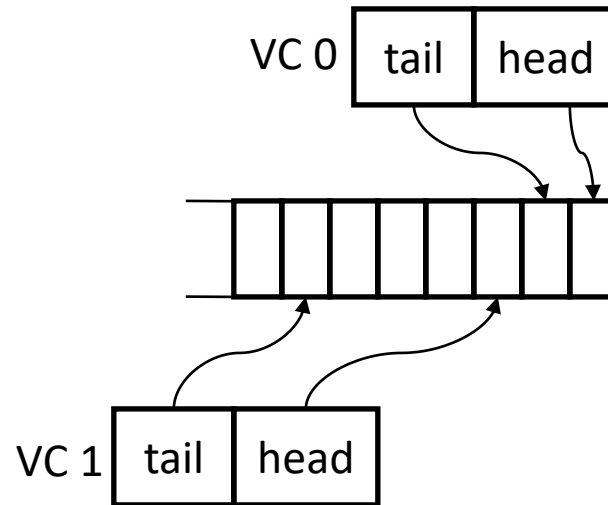
- Multiple fixed length queues per physical channel



Buffer Organization

Multiple variable length queues

- Multiple VCs share a large buffer
- Each VC must have minimum 1 flit buffer to prevent deadlock
- More complex circuitry



Buffer Organization

Many shallow VCs

- + Better addresses head-of-line blocking
- + Good for heavy traffic

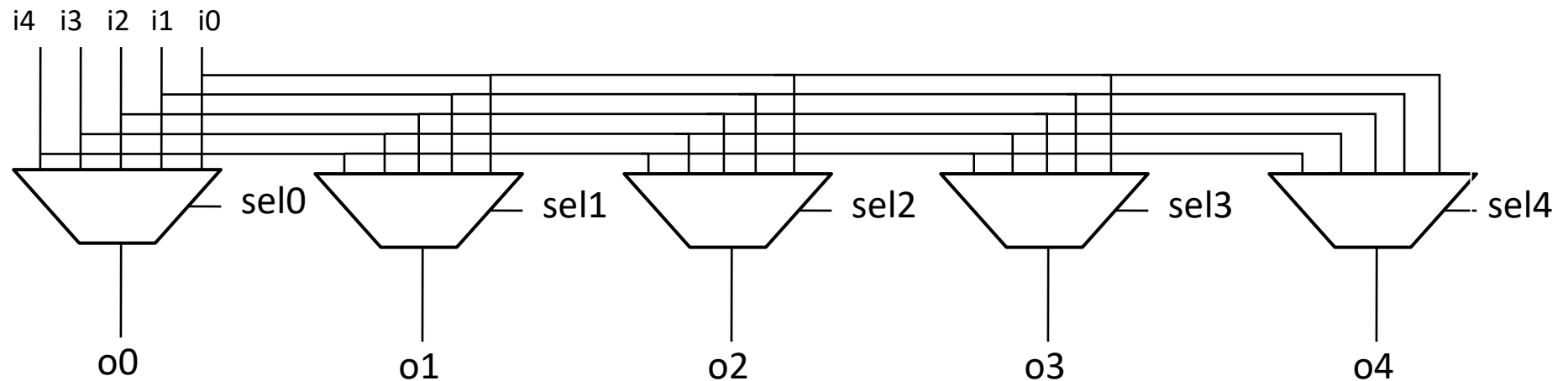
Few deep VCs

- + Less complex VC allocator
- + Smoother flow control

Switch Organization – Mux Crossbar

Crossbar composed of many multiplexers

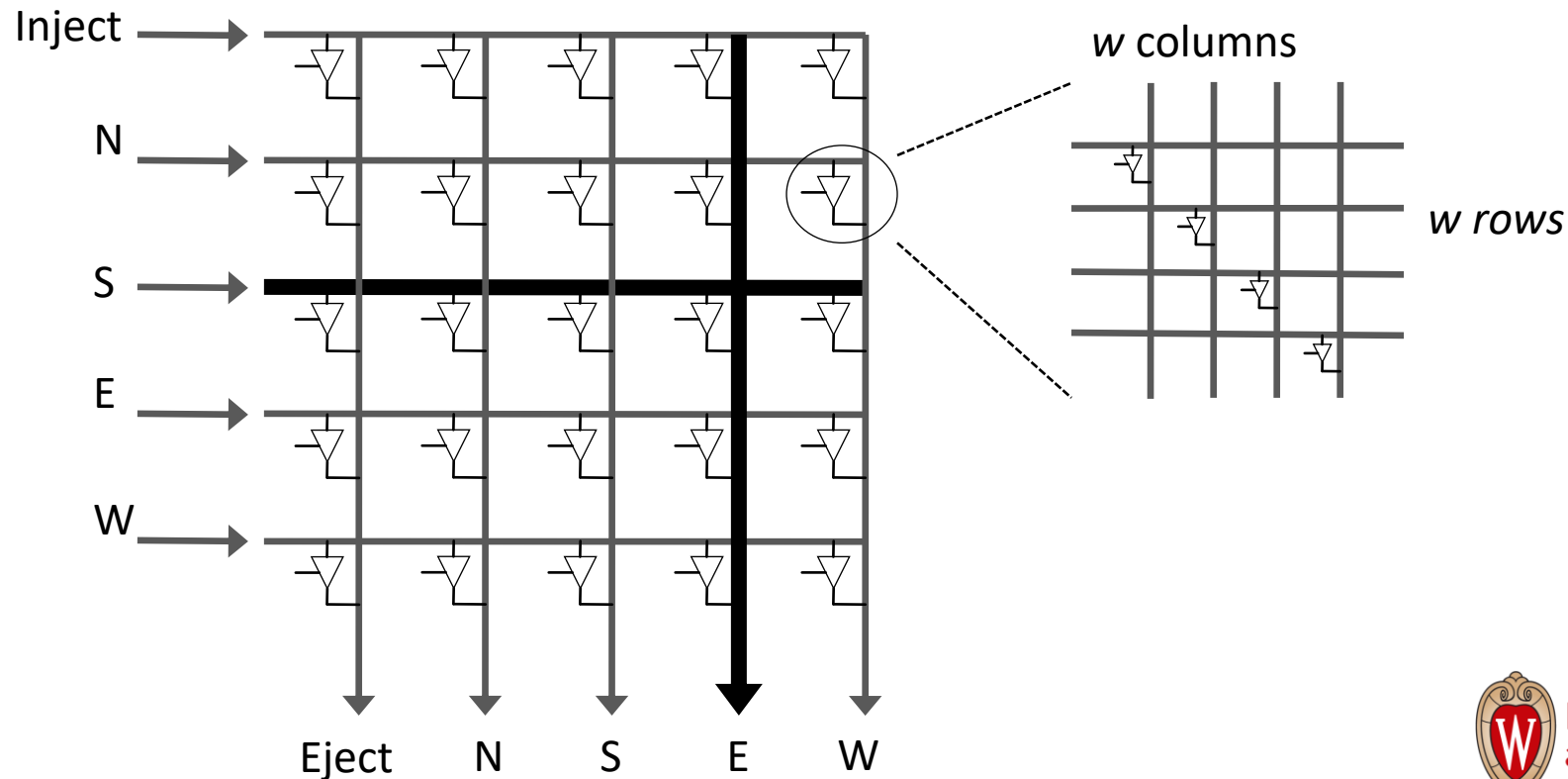
- Common in low-frequency router designs



Switch Organization – Crosspoint

Grid of tristate buffers

- w : port width in bits, determines phit size



Switch Organization – Crossbar Speedup

Design crossbar with more ports

- Increases internal switch bandwidth
- Allows VCs of the same input port to use crossbar at the same time

